

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



# LAB MANUAL CSA37

# SOFTWARE TESTING

CONTENTS:


| S.No | CONTENT | PAGE NUMBER |
|---|---|---|
| 1 | CSE DEPARTMENT VISION-MISSION | |
| 2 | CSE-PO-PSO | |
| 3 | COURSE OUTCOMES | |
| 4 | DO'S AND DON'TS | |
| 5 | INSTRUCTIONS FOR THE EXPERIMENTS | |
| 6 | EXPERIMENT LIST | |
| 7 | CO-PO ATTAINMENT | |

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

# CSE DEPARTMENT VISION-MISSION

**Vision of the Department**

**To establish an environment to provide quality education and Inculcate research attributes among computer science engineering graduates through problem solving skills and technological innovations.**

**Mission of the Department**

**M1**

**To create and sustain an academic environment to the highest level in teaching and research by enhancing the knowledge of the faculties and students in technological advancements to solve real time problems.**

**M2**

**Providing a suitable environment for the students to develop professionalism with knowledge in Computer Science & Engineering to meet the contemporary industry needs and satisfy global standards.**

**M3**

**To facilitate the development of professional behavior and stronger ethical values so as to work with commitment for the progress of the nation and face challenges with ethical and social responsibility.**



# INSTRUCTIONS FOR THE EXPERIMENTS







# SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

## DO'S AND DON'TS

- ✓ Be on time to lab and maintain silence.
- ✓ Inform the instructor /TA in case of any working environment problem.
- ✓ Be aware of all the safety devices. Even through the instructor and TA will take care of emergencies.
- ✓ Bring all the necessary things required to do laboratory experiments like observation notebook, record notebook and any others.
- ✓ Enter the system number, in-time, out-time register number, name and signature in the students log register.
  Arrange the seatin$_g$ chairs and system accessories in place at the end of the session.
- ✓ Shut down the system properly and switch off the power switch at the end of the session.
- ✓ Keep your bags in front of the labs empty space.
- ✓ Do not eat, drink, chew gum, smoke or apply cosmetics in the lab.
- ✓ Do not unplug/plug any wires of the system connectivity.
- ✓ Do not use or charge the mobile phone or any electronic gadgets inside the lab.
- ✓ Not to troubleshoot by yourself without knowledge of instructor/TA.
- ✓ Do not open any unnecessary applications in system.
- ✓ Mobiles phones are prohibited inside the lab.
- ✓ Do not share a system to do experiments.

Students strictly follow all the above instructions

SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE 0 F MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

## LIST OF EXPERIMENTS

**COURSE CODE: CSA37**

**COURSE NAME : SOFTWARE TESTING**

1. Select any system, such as an ATM system, and thoroughly examine its system specifications to identify and document the different software defects present.
2. To write a case study to identify & analyze the requirements of ARMS Portals and to write a positive Test Scenarios as well as Test cases for the above application
3. To write a case study to identify & analyze the requirements of ARMS Portals and to write a negative Test Scenarios as well as Test cases for the above application
4. To write a case study to identify & analyze the requirements of e-Commerce Applications and to write a positive Tesi Scenarios as well as Tesi cases for the above application.
5. To write a case study to identify & analyze the requirements of e-Commerce Applications and to write a negative Test Scenarios as well as Tesi cases for the above application.
6. To write a case study to identify & analyze the requirements of Food delivery Applications and to write a positive Test Scenarios as well as Testcases for the above application.

7. To write a case study to identify & analyze the requirements of Food Delivery Applications and to write a negative ’1 est Scenarios as well as Testcases for the above application.
8. write a case study to identify & analyze the requirements of Banking Applications and to write a positive Test Scenarios as well as Test cases for the above application.
9. To write a case study to identify & analyze the requirements of Banking Applications and to write a negative Test Scenarios as well as Test cases for the above application.
10. Develop a comprehensive test plan document for a chosen application, such as a Library Management System, outlining the various testing strategies, methodologies, and criteria to ensure thorough evaluation and validation of the system's functionality.
11. Write a case study to identify and analyze the requirements of a Railway Reservation System and prepare positive test scenarios and test cases for the application.
12. To find the Nth term of the Fibonacci series and validate the output using JUnit assertions.
13. To calculate the Greatest Common Divisor (GCD) of two numbers and validate the output using JUnit assertions.
14. To check whether a given year is a Leap Year and validate the output using JUnit assertions.
15. To find the Nth term of the Fibonacci series and validate the output using JUnit assertions.
16. To check whether a string is an anagram of another string and validate output using JUnit assertions.

17. To find the maximum of three numbers and validate output using JUnit assertions.
18. To check whether a number is even or odd and validate output using JUnit assertions.
19. To calculate compound interest and validate output using JUnit assertions.
20. To count the number of vowels and consonants in a given string and validate output using JUnit assertions.
21. To check whether a given number is prime and validate output using JUnit assertions.
22. To reverse a given number and validate output using JUnit assertions.
23. To find the sum of digits of a given number and validate output using JUnit assertions.
24. To check whether two strings are equal ignoring case sensitivity and validate output using JUnit assertions.
25. To calculate grade of a student based on marks and validate output using JUnit assertions.
26. To find the largest element in an array and validate output using JUnit assertions.
27. To calculate the average of n numbers using functions and validate output using JUnit assertions.
28. To check whether a given number is an Armstrong number and validate output using JUnit assertions.
29. To convert temperature from Celsius to Fahrenheit and validate output using JUnit assertions.

30. To find the length of a string without using built-in length() function and validate output using JUnit assertions.

31. To reverse a given word and validate output using JUnit assertions.

32. To perform string comparison and validate output using JUnit assertions.

33. To validate voting eligibility based on age using JUnit assertions.

34. To calculate simple interest based on customer type and validate output using JUnit assertions.

35. To check whether a given number is palindrome and validate output using JUnit assertions.

36. To open Chrome browser using Selenium WebDriver and verify the title of ARMS portal.

37. To automate search functionality in Google using Selenium WebDriver.

38. To automate registration form filling by entering name, email, password and submitting form using Selenium.

39. To automate login into ARMS portal and validate error message for invalid credentials using Selenium.

40. To automate logout functionality of a web application using Selenium WebDriver.

41. To automate dropdown selection using selectByVisibleText() in Selenium WebDriver.

42. To automate checkbox and radio button selection using Selenium WebDriver

43. To handle JavaScript alert pop-ups using Selenium WebDriver.

44. To automate file upload functionality on a web application using Selenium WebDriver.

45. To capture a screenshot of a web page and store it in the local system using Selenium WebDriver.

46. To perform load testing on ARMS homepage during peak time by simulating 500 concurrent users and analyze response time, throughput and error rate.

47. To test a mobile banking REST API for balance enquiry requests by simulating multiple users and verifying status codes, response data and response time.

48. To perform POST request performance testing with JSON payload for a user registration service and evaluate system behaviour for valid and invalid inputs.

49. To test login functionality of a student portal where hundreds of students login simultaneously and analyze server stability.

50. To perform smoke and load testing on a newly deployed government service website to verify availability and performance of critical pages.

51. To conduct stress testing on a result publishing server by gradually

increasing concurrent users to identify breaking point.

52. To analyze response time, throughput and error percentage of a real-time payment gateway under continuous transaction load.

53. To perform data-driven testing on a login API by reading multiple user credentials from CSV file and validating authentication responses.

54. To test session management and cookie handling of an online shopping cart application to ensure sessions persist correctly during navigation.

55. To perform end-to-end performance testing of an e-commerce application by simulating user flow login -> browse -> add to cart -> checkout -> logout.

56. To perform capacity testing by gradually increasing users and analyzing maximum supported load.

57. To test search performance under multiple concurrent users and analyze response time.

58. To perform soak testing for extended duration and analyze stability and memory leaks.

59. To test upload and download performance with varying file sizes under concurrent users.

60. To perform API performance testing on microservices-based weather service under peak load.

# EXP NO : 1

## ATM System

Objective: Select any system, such as an ATM system, and thoroughly examine its system specifications to identify and document the different software defects present.

Common software defects that can be found in ATM systems:

Features to be tested:
1. Validity of the card.
2. Withdraw Transaction flow of ATM.
3. Authentication of the user's.
4. Dispense the cash from the account.

| Bug-Id | Bug Name |
|---|---|
| ATM 001 | Invalid Card ATM 002 Invalid PIN |
| ATM 003 | Invalid Account type ATM 004 Insufficient Balance |
| ATM 005 | Transaction Limit |
| | |
| ATM 006 | Day limit |
| | |
| ATM 007 | Invalid money denominations |
| ATM 008 | Receipt not printed |
| ATM 009 | PIN change mismatch |

**BUG REPORT:**

| |
|---|
| **Bug Id:** ATM 001 |
| **Bug Description:** Invalid card |
| Steps **to reproduce:** 1. Keep valid card in the ATM. |
| **Expected Result:** Welcome Screen |
| **Actual Result:** Invalid card |
| **Status** : Pass/Fail |

**Bug Id: ATM 002**
Bug Description: Invalid PIN entered
Steps to reproduce:
1. Keep a valid card in ATM.
2. Enter the authorized PIN.
3. Menu screen should be displayed.

Expected Result: Menu screen displayed
Actual Result: Invalid PIN screen is displayed
Status : Pass/Fail

**Bug Id: ATM 003**
Bug Description: Invalid Account type selected.
Steps to reproduce:
1. Enter a valid user PIN number.
2. Select the withdraw option on the main menu.
3. Choose the correct type of account (either savings or current account).
Expected Result: Enter the Amount screen displayed
Actual Result: Invalid Account type screen is displayed.
Status : Pass/Fail

**Bug Id: ATM 004**
Bug Description: Insufficient Balance
Steps to reproduce:
1. Menu screen should be displayed.
2. Select the withdraw option.
3. Select the correct type of account.
4. Enter the sufficient amount to withdraw from the account.
5. Dispense the cash screen & amount to be deducted from
account Expected Result: Collect the amount screen displayed
Actual Result: Insufficient balance in the account
Status : Pass/Fail

**Bug Id: ATM 005**
Bug Description: Withdraw Limit per transaction.
Steps to reproduce:
1. Menu screen should be displayed.
2. Select the withdraw option.
3. Select the correct type of account.
4. Enter sufficient amount to withdraw from the account Transaction within the limit.
5. Dispense the cash screen & amount to be deducted from
account. Expected Result: Cash is dispensed and collect the receipt
Actual Result: Transaction limit exceeded screen is displayed
Status : Pass/Fail

**Bug Id: ATM 006**
Bug Description: Withdraw limit per day
Steps to reproduce:
1. Keep a valid card in ATM.
2. Enter the authorized PIN.
3. Enter the amount to withdraw from the account.
4. Amount enter is over the day limit (>40000)
5. Amount enter is over the day limit and display screen is
displayed. Expected Result: Cash is dispensed and collect the receipt.
Actual Result: Day limit exceeded screen is displayed.
Status : Pass/Fail

Bug Id: ATM 007
Bug Description: Amount enter denominations
Steps to reproduce:
1. Keep a valid card in ATM.
2. Enter the authorized PIN.
3. Enter the amount which should be in multiples of 100.
4. Cash Dispenser screen is displayed.

Expected Result: Collect the amount screen is displayed.
Actual Result: Amount enter not in required denominations.

# EXP NO : 2

## Case Study on ARMS Portals -Positive Test Cases

**Objective:** Case Study: Requirements Analysis and Testing for ARMS Portal.

| **Sl.no** | **Items to** be tested | **Features** | **Approach** | **Test environment** | **Training needed** | **Risk** | Test case | Pass/ **Fail** |
|---|---|---|---|---|---|---|---|---|
| 1 | Home | Holidays Notification Attendance Confirmation | Manual approach | Operating system/Browser | Not required | No | -Open the website arms.saveetha.com -Enter the user name and password -click into home button -check whether holiday is there | Pass |
| 2 | My course | *In progress course *Graduation status *Completed course *External credits | Manual approach | Operating system/Browser | Not required | No | -Open the website arms.saveetha.com -Enter the user name and password. -click on " = " button and go to my course | Pass |
| 3 | Enrollment | Select slot A Slot B Slot C Slot D *course details *Note=Approved Pending | Manual approach | Operating System /Browser | Not required | No | - Open the website arms.saveetha.com -Enter the user name and password - click on " " button and go to Enrollment select slot A, | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | slot Bis there are not. | |
| 4 | Attendance | *Attendance report *course code *course name | Manual approach | Operating system/Browser | Not required | No | - click on "=" button and go to attendance -Click on attendance report whether course code is there are not. | Pass |
| 5 | Assignment | * course *select *Complete/Assignment | Manual approach | Operating system/Browser | Not required | No | -click on " = " button and go to Assignment. -Click on assignment select corning up | Pass |
| 6 | Examination | *No due request *Please enable pop up option | Manual approach | Operating System/Browser | Not required | No | -click on "=" button and go to Examination .click list no due payment history is there are not | Pass |
| 7 | Financial record | *Due list *Payment history *Print payment | Manual approach Manual approach | Operating system/Browser | Not required | No | click on "=" button and go to Financial record click due list payment | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | history is there are not | |
| 8 | Disciplinary | Disciplinary record issue details, last action details | Manual approach | Operating systems/Browser | Not required | No | | Pass |
| 9 | Raise Infra issue | | Manual approach | Operating systems/Browser | Not required | No | | Pass |
| 10 | My profile | | Manual approach | Operating systems/Browser | Not required | No | | Pass |

# EXP NO : 3

## Case Study on ARMS Portals -Negative Test Cases

Objective. Case study to identify & analyze the requirements of ARMS Portals negative Test Scenarios as well as Test cases for the above application.

| Item to be tested | Features | Approach | Data Test | Test Environment | Training needed | Risk | Test case | Expected output | Pass / Fail |
|---|---|---|---|---|---|---|---|---|---|
| 1) Home | Holidays, Notification, Attendance Confirmation | Manual Approach | Username, Password | Operating System / Browser | Not Required | No | * open website https://www.google.com/search?q=aims.sse.saveetha.com<br>* enter username & password<br>* enter wrong password | Username and password entered is invalid | Pass |
| 2) My course | * In Progress Courses<br>* External credit status | Manual Approach | Username, Password | Operating System / Browser | Not Required | No | * Click on '=' button<br>* Check whether completed courses is there or not. | My course wasn't appeared | Pass |
| 3) Enrollment | Select Slot, Select Slot A, Slot B, Course Details, Note: Approved | Manual Approach | Username, Password | Operating System / Browser | Not Required | No | * Click on '=' button<br>* Go to enrollment<br>* Select Slot A, Slot B, Slot C | My enrollment wasn't appeared | Pass |
| 4) Attendance | S. No, Course Code, Course Name, Class attended view | Manual Approach | Username, Password | Operating System / Browser | Not Required | No | * Click on '=' button<br>* Go to Attendance<br>* Click on Attendance report | Attendance wasn't appeared | Pass |
| 5) Assignment | Completed Assignment, content, Exams | Manual Approach | Username, Password | Operating System / Browser | Not Required | No | * Click on '≡' button<br>* Go on Assignment<br>* Select Course opening Assignment | Assignment wasn't appeared | Pass |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| **6) Examination** | No due request please enable popup options in web browser to print hall ticket | Manual Approach | Username, Password | Operating System / Browser | Not Required | No | * Click on '≡' button<br>* Click on examination<br>* Go to no due request whether it is there or not | Request is not accepted | Pass |
| **7) Financial Record** | * Due List<br>* Payment history<br>* Print Payment | Manual Approach | Username, Password | Operating System / Browser | Not Required | No | * Click on '≡' button<br>* Go to Financial Record<br>* Check due list, Payment history | Payment history not cleared | Pass |
| **8) Disciplinary** | Disciplinary Record, Issue details, Last action details, complaint | Manual Approach | Username, Password | Operating System / Browser | Not Required | No | * Click on '≡' button<br>* Go to Disciplinary issue details<br>* Last action details | Issues were not cleared | Pass |
| **9) Raise Infra Issue** | Raise issue content, choose file location | Manual Approach | Username, Password | Operating System / Browser | Not Required | No | * Click on '≡' button<br>* Go to infra issues content<br>* Choose file location | Not showing file location | Pass |
| **10) My Profile** | Edit profile, student record details | Manual Approach | Username, Password | Operating System / Browser | Not Required | No | * Click on '≡' button<br>* Go on my profile<br>* Check student record details | Profile was not updating | Pass |

# EXP NO : 4

## Case Study on e-Commerce Applications -Negative Test Cases

Objective. Case study to identify & analyze the requirements of Flipkart-positive Test Scenarios as well as Test cases for the above application.

| Item to be tested | Features | Approach | Data Test | Test Environment | Training needed | Risk | Test case | Pass / Fail |
|---|---|---|---|---|---|---|---|---|
| Home | Search button, shops, Notification, Account Cart | Manual approach | No | Laptop, Android | No | No | *Open the Flipkart<br>* Click on the email, pass-word,<br>*Click on the Home page | Pass |
| Search button | Search for product, brands and shirts | Manual approach | No | Laptop, Android | No | No | *Open the Flipkart<br>* Click on the home page the search button appear<br>*Click on the search button and find the product | Pass |
| Shops | Offer Zone, More on Flipkart, Trending stores, recently viewed stores | Manual approach | No | Laptop, Android | No | No | *Open the Flipkart<br>*Click on the home page<br>*Click the shops | Pass |
| Notifications | All Offers | Manual approach | No | Laptop, Android | No | No | * Click on the Home page and click the notifications<br>* We can find the any offers or getting to buy product. | Pass |
| Account | Orders, wish-list, coupons, Help Centre Edit Profile Logout | Manual approach | No | Laptop, Android | No | No | *Click on the Flipkart<br>* Click on the Home page and press the Account and we will find the or-ders that we placed<br>*Click on the Account and find the username and phone number | Pass |
| Cart | My cart, Address, Product, order place | Manual approach | No | Laptop, Android | No | No | *Click on the Flipkart<br>* Click on the Home page and click the cart<br>* Click the product and place the order in the my cart. | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| Accounts | Account setting, My activity, Earn with Flipcart, feed back, Information | Manual approach | No | Laptop, Android | No | No | *Click on the Flipkart<br>* Click on the Home page and press the account<br>*Click the account setting we can edit the profile, save the address | Pass |

| Item to be tested | Features | Approach | Data Test | Test Environment | Training needed | Risk | Test case | Pass / Fail |
|---|---|---|---|---|---|---|---|---|
| My cart | Address, Remove, save later, Buy this now | Manual approach | No | Laptop, Android | No | No | * Click on the Flipkart<br>* Click on the Home page and press the product and remove, save<br>* Place the order | Pass |
| Search button | Search the product, Related searches, popular products, Discover more | Manual approach | No | Laptop, Android | No | No | * Click on the Flipkart<br>* Click on the Home page and click the search button and search the product and<br>* Place the order<br>* We can search it any of the product that you wish. | Pass |
| Accounts | Order placed, Cancelled | Manual approach | No | Laptop, Android | No | No | * Click on the Flipkart<br>* Click on the Home page and click the account and press the order<br>* And we can find the prod-uct that we have placed the order and cancelled the order | Pass |

# EXP NO : 5

**Case study on e-Commerce Applications- Negative Test Cases**

**Objective:** To write a case study to identify & analyze the requirements of e-Commerce Applications (Flipkart) and to write a negative Test Scenarios as well as Test cases for the above application.

| Items to be selected | Features | Approach | Data Test | Test Environment | Training Needed | Risk | Test ease | Expected Output | Pass |
|---|---|---|---|---|---|---|---|---|---|
| Home | Search button, shops, notifications, Account etc | Manual Approach | No | Laptop, Android | No | No | Step 1:- Open the Flipkart Step2:- Click on the Email, password | Other tab Opened =>Incorrect Email or password | Pass |
| Search button | Search for products Brands & shirts | Manual Approach | No | Laptop, Android | No | No | Step l:- Search for products Step 2:- Incorrect product will appear | Wrong Products Appear | Pass |
| Shops | Search for Mobiles, Fashion, Electronics | Manual Approach | Np | Laptop, Android | No | No | Step l:- Open the shops Step 2:- Click on The mobiles & search | Searched Mobiles do not appear | Pass |

| Items to be selected | Features | Approach | Data Test | Test Environment | Training Needed | Risk | Test Case | Expected Output | Pass/Fai1 |
|---|---|---|---|---|---|---|---|---|---|
| Trending Stores | Search for student And wedding Store | Manual approach | No | Laptop, Android | No | No | Stepl:- Open The shops Step2:- In that open trending stores Step3:- But it will open another store | Not open Student's Club | Pass |
| Notifications | Order list will appear | Manual approach | No | Laptop, Android | No | No | Step l:- Open the Notifications Step2:- Order placed Product Step3:- Does not appear in notifieations | Does not show notifications | Pass |
| Account | Click on orders wishlist coupons | Manual approach | No | Laptop, Android | No | No | | Help centre Does not appear | Pass |

| Items to be select-ed | Fea-tures | Approach | Data Test | Test Environ-ment | Train-ing needed | Risk | Test Case | Expected output | Pass/ Fail |
|---|---|---|---|---|---|---|---|---|---|
| Accounts | Click on Logo u t | Manual Approac h | No | Laptop, Androi d | No | No | Step1:- Open flipcart and click in account Step2:- At last click on logout option Step3:- But flipcart doesn't logout | Flipcar t doesn't logout | Pass |
| Accounts | Click on edit profile | Manual Approac h | No | Laptop, Androi d | No | No | Step1:- Open flipcart and click on accounts Step2:- Click on edit profile Step3:- Profile will not be up-dating | Profile will not be updat-ing | Pass |
| Cart | Click on place order | Manual Approac h | No | Laptop, Androi d | No | No | Step1:- Open ac-counts & click place order Step2:- Place order | Order not placed | Pass |
| Shops | Search for medi-cines | Manual Approac h | No | Laptop, Androi d | No | No | Step1:- Open flip-cart & click medicines Step2:- med-icines will not appear | Medi-cines do nor appear | Pass |

EXP NO : 6

## Case study on Food Delivery Applications- Positive Test Cases

To write a case study to identify & analyze the requirements or Food Delivery Applications and to write a positive Test Scenarios as well as Test cases for the above application.

| Item to be tested | Features | Approach | Data Test | Test Environment | Training needed | Risk | Test case | Pass / Fail |
|---|---|---|---|---|---|---|---|---|
| 1) Home | Offers<br>Delivery<br>Dining<br>Money | Manual Approach | Username<br>Password | Operating System / Browser | Not Required | No | * open the website https//zomato.com<br>* Enter user names & password<br>* Click on the login button<br>* Check whether offers, delivery, dining, money is there or not | Pass |
| 2) Food orders | * Your orders<br>* Favourite orders<br>* Address book | Manual Approach | Username<br>Password | Operating System / Browser | Not Required | No | * Click on food orders<br>*Check whether your orders, Favourite orders address books is there or not | Pass |
| 3) Restaurants Awards | * Winning Restaurants | Manual Approach | Username<br>Password | Operating System / Browser | Not Required | No | * Click on restaurant awards<br>* Check whether winning restaurants is there or not | Pass |
| 4) Zomato UPI | * Activate Zomato UPI<br>* Zomato gold | Manual Approach | Username<br>Password | Operating System / Browser | Not Required | No | * Click on Zamato UPI<br>* Check activate UPI and Zomato gold is there or not | Pass |
| 5) Events | * Your event tickets<br>* Events help | Manual Approach | Username<br>Password | Operating System / Browser | Not Required | No | * Click on events<br>* To check your event tickets<br>* Events help is there or not | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 6) Dining | * Your transactions<br>* Your dining rewards<br>* Your slot bookings | Manual Approach | Username Password | Operating System / Browser | Not Required | No | * Click on dining<br>* Open and check whether your transactions, your dining rewards, your slot bookings | Pass |
| 7) Money | * Buy gift card<br>* Claim gift card<br>* Zomato credits | Manual Approach | Username Password | Operating System / Browser | Not Required | No | * Click on money<br>* Open and check whether buy gift card, claim gift card is there or not | Pass |
| 8) More | * Choose language<br>* Send feed back<br>* Log out | Manual Approach | Username Password | Operating System / Browser | Not Required | No | * Click on profile view activity<br>* Open and check whether add review, add photo, edit profile, reviews is there or not | Pass |
| 9) Profile view activity | * Add review<br>* Add photo<br>* Edot Profile<br>* Reviews | Manual Approach | Username Password | Operating System / Browser | Not Required | No | * Click on your rating<br>* Open and check whether understand your rating and your calculated, ok is there or not | Pass |
| 10) Your Rating | * Understand your rating<br>* Your rating calculated ok | Manual Approach | Username Password | Operating System / Browser | Not Required | No | * Click on ' ≡ ' button<br>* Go on my profile<br>* Check student record details | Pass |

# EXP NO : 7

## Case study on Food Delivery Applications- Negative Test Cases

To write a case study to identify & analyze the requirements of Food Delivery Applications and to write a negative Test Scenarios as well as Test cases for the above application

| *Item to be seache* | *Featur es* | *Appro ach* | *Data set* | *Test environ ment* | *Train ing neede* | *Ri sk* | *Test case* | *Expect ed output* | *Pa ss/ Fai* |
|---|---|---|---|---|---|---|---|---|---|
| Home | Offers Delivar y Dining Money | Man ual approa ch | Usern ame /Passw ord | Operatin g system/ Brower | Not Rcqui red | No | ->Open website https//zomot o.com ->Enter user name &password opened (or) not | Other tab opened >Incor rect email (or) passwo rd | Pas s |
| Food order | your order Favour ate-Order Addres s Book | Manu al approa ch | Usern ame /Passw ord | Operati n g system/ | Not requi red | No | Click on food order Favorite order Address is there or not | Wrong item appear | Pas s |
| Restau rant Award s | Wining Rcxtaui anl | Manu al approa ct | Usern amc /Passix ord | Operatin g systein/ fit ower | Not Rcqui red | No | ->Click on rcxlaui anl awtirds ->Clieck whether ivininp rcstarn'ant is lherc or not | Fake rcsla ur ant name appcr | Pas s |
| Zomol o Upi | Activil y Zomot TIPI Zomot gold | Munu al appro a ch | Llscrri arrie /I'assw ord | Opcratin g systein/ Browcr | Not Rcqui red | No | --•Click on zomolo UPI - Check activity UPland zoiiioto gold is there (or) not | - Wron g accept >Ins1' 1i cenl balanc c will appear on wallet bank | Pus s |
| Lvents | Your events tickets Lvcnt help | Manu al approa ch | Usern amc /Passw ord | Operatin g systems/ Brower | Not Rcqiii i ed | No | ->Click on the events your event ticket FEvcnl help is veaver not | Wrong tickets display on the xcrccn | Pas s |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Dining | Your transation Your dining reward Your Slot bookin g | Manu a1 approa eh | Usern ame /Passw ord | Operatin g system/ Brower | Not Requi red | No | Click on the dining disk on tranee,your dining reward slot booking | After debited money from bank ofill foof order is booked | Pas s |
| Money | Buy gift card Clime gift card Card zomoto credit | Manu a1 approa eh | Usern ame /Passw ord | Operatin g system/ Brower | Not Requi red | No | Click on money oprn and cheek buy gift cards | Not access for elamin g the gift card | Pas s |
| Move | Choose Langua ge Send feedbae | Manu a1 approa eh | Usern ame /Passw ord | Operatin g system/ Brower | Not Requi red | No | Choose language and send feedback | Unable to send feedba ek layout | Pas s |
| Profile view - aetivit y | Layout Add review Add photo Edit profile Review | Manu a1 approa eh | Usern ame /Passw ord | Operatin g system/ Brower | Not Requi red | No | Profile view Add review review | Unable to send the review of users (Proxi review | Pas s |
| Your rating | Unders tand Your rating Tour rayting eduote d OK. | Manu a1 Appro aeh | Usern ame /Passw ord | Operatin g system/ Brower | Not Requi red | No | Your rating Cheek whether your rating is ok is there eought | Unable to see your item rating | Pas s |

| *Item to be seached* | *T'eatures* | *Approach* | *Zlata set* | *"1'est environ ment* | *"1'raining needed* | *iisk* | *"1'est case* | *pect ed output* | *Passh Fat I* |
|---|---|---|---|---|---|---|---|---|---|
| Home | Offers Delivar y Dining Money | Manua1 approa ch | User n ame /Passw ord | Operati n g system/ Brower | Not Requi red | No | ->Open website https//zomo t o.com ->Enter user name &password opened (or) not | Other tab opene d >Incor rect email (or) passw o rd | Pas s |
| Food orde r | Your order Favou r ate-Order Addre s Book | Manua1 approa ch | User n ame /Passw ord | Operati n g system/ Brower | Not Requi red | No | Click on food order Favorite order Address is there or not | Wrong item appear | Pas s |
| Resta u rant Award s | Wining Restau r ant | Manua1 approa ch | User n ame /Passw ord | Operati n g system/ Brower | Not Requi red | No | ->C1ick on restaurant awards ->Check whether wining restaurant is there or not | Fake restaur ant name apper | Pas s |
| Zomot o Upi | Activi t y Zomot UPI Zomo t gold | Manua1 approa ch | User n ame /Passw ord | Operati n g system/ Brower | Not Requi red | No | ->C1ick on zomoto UPI ->Check activity UPIand zomoto gold is there (or) not | - >Wron g accept >Insff i cent balanc e will appear on wallet or bank | Pas s |
| Events | Your event s ticket s Event help | Manua1 approa ch | User n ame /Passw ord | Operati n g system/ Brower | Not Requi red | No | ->C1ick on the events your event ticket >Event help is veaver not | Wrong tickets displa y on the screen | Pas s |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Dining | Your transation Your dining reward Your Slot bookin g | Manu a1 approa eh | Usern ame /Passw ord | Operatin g system/ Brower | Not Requi red | No | Click on the dining disk on tranee,your dining reward slot booking | After debited money from bank ofill foof order is booked | Pas s |
| Money | Buy gift card Clime gift card Card zomoto credit | Manu a1 approa eh | Usern ame /Passw ord | Operatin g system/ Brower | Not Requi red | No | Click on money oprn and cheek buy gift cards | Not access for elamin g the gift card | Pas s |
| Move | Choose Langua ge Send feedbae | Manu a1 approa eh | Usern ame /Passw ord | Operatin g system/ Brower | Not Requi red | No | Choose language and send feedback | Unable to send feedba ek layout | Pas s |
| Profile view - aetivit y | Layout Add review Add photo Edit profile Review | Manu a1 approa eh | Usern ame /Passw ord | Operatin g system/ Brower | Not Requi red | No | Profile view Add review review | Unable to send the review of users (Proxi review | Pas s |
| Your rating | Unders tand Your rating Tour rayting eduote d OK. | Manu a1 Appro aeh | Usern ame /Passw ord | Operatin g system/ Brower | Not Requi red | No | Your rating Cheek whether your rating is ok is there eought | Unable to see your item rating | Pas s |

EXP NO : 8

**Case study on Banking Applications - Possitive Test Cases**

To write a case study to identify & analyze the requirements of Banking Applications and to write a possitive Test Scenarios as well as Test cases for the above application.

| ITEMS TO BE TESTED | FEATURES | APPROACH | DATA TEST | TEST ENVIRONMENT | TRAINING NEEDED | RISK | TEST CASE | PASS/ FAIL |
|---|---|---|---|---|---|---|---|---|
| User Registration | Name, Email, Password | Manual approae h | Username, password | Laptop, phone | NO | NO | Stepl: Verify that user can sueeessfully register with valid information. Step2: Verify that all mandatory fields are validated properly. Step3: Verify the unique username is required during registration | Pass |
| User Login | Email, Password | Manual approae h | Username, password | Laptop, phone | NO | NO | Stepl: Verify that a registered user can login with valid eredentia 1s. | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | Step2: Verify that appropriate error message are displayed. Step3: Verify the system maintains user sessions and keeps the user logged | |
| Aeeount management | Aeeount information | Manual approaeh | Username, password | Laptop, phone | NO | NO | Stepl: Verify that user can create a bank account or not. Step2: Verify user can view account details ineludin8 balance and transaetion history. Step3: Verity that user update information. | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| Fund transfer | Balance history, transaction | Manual approa e h | Username, password | Laptop, phone | NO | NO | Stepl: Verify that user can transfer money or not' Step2: Verify that the transferred amount aeeurately reflected in sender or receiver. Step3: Proper validation is performed for account numbers and transfer amounts. | Pass |
| Bill payment | Electricity water, phone | Manual approa e h | Username, password | Laptop, phone | NO | NO | Stepl: Verify that a user can pay bills sueeessfully Step2: Verify that the payment reflected in the user's transaetion of | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | biller's record. Step3: Verify that appropriate error messages are displayed for invalid bill payment attempts. | |
| Aeeount security | Length, eomplexity | Manual approaeh | Username, password | Laptop, phone | NO | NO | Stepl: User can update their password sueeessfully. Step2: Verify that new password meets the security. Step3: It access IWO-factors authentication for enhanced security. | Pass |
| Customer support | Chat, email phone | Manual approaeh | Username, password | Laptop, phone | NO | NO | Stepl: User can conduct customer support via various channels. Step2: | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | Customer support request are handled promptly and effieiently Step3: User receives the appropriate response | |
| Mobile banking | User friendly, mobile app | Manual approaeh | Username, password | Laptop, phone | NO | NO | Stepl: Mobile banking applieations provides a seamless and user-friendly experienee. Step2: All functionalities available on the web-platform. Step3: Mobile app supports various mobile devices. | Pass |
| Transaetion | Transfer, bill paymen | Manual approaeh | Username, | Laptop, phone | NO | NO | Stepl: User received | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| notifieat ion | t, account change | | passw ord | | | | real notifieati on for transaeti on. Step2: Notifieat ion contain accurate and relevant informati on. Step3: Verify the user can eustomiz e the notifieati ons | |
| Integral ion with external services | Externa l services , paymen t gatewa y | Manual approae h | Usern ame, passw ord | Laptop, phone | NO | NO | Stepl: Banking applieati on integrate smoothly with external services. Step2: Data exehang e between the applieati ons Step3: Relevant informati on for external services | Pass |

## EXP NO : 9

**Case study on Banking Applications - Negative Test Cases**

To write a case study to identify & analyze the requirements of Banking Applications and to write a negative Test Scenarios as well as Test cases for the above application

| **Items** to be tested | **Featur** es | **Appr oach** | **Data Test** | **Test Environ** ment | **Train ing** Need ed | **R i sk** | **Testcase** 9 | | **Pass/ Fail** |
|---|---|---|---|---|---|---|---|---|---|
| User Registr ation | Name, email, passwo rd | Manu a1 Appro aeh | Usern ame, Passw ord | Laptop, Phone | NO | N O | Stepl- verify that user can sueeessf ully register with valid informati on Step2- Verify that all mandator y fields are validated Properly Step3- Verify that unique user name is required during registrati on. | The appliea tion should display on appropr iate error messag e indieat ing that the passwo rd entered is invalid | pass |
| User Login | Email, passwo rd | Manu a1 Appro aeh | Usern ame, Passw ord | Laptop, Phone | NO | N O | Stepl- verify that a registere d user can login<br><br>valid eredentia 1s. Step2- Verify that appropri ate error message | Passw ord is ineorre ct | Pass |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | displaye d.<br><br>Step3- Verify that the system maintain s user session and keeps the user logged | | |
| Aeeoun t manage ment | Aeeou nt inform ation | Manu a1 Appro aeh | Usern ame, Passw ord | Laptop, Phone | NO | N O | **Stepl-** verify that user can create a bank account or not Step2- Verify that user can view account details including balance and transaeti on history Step3- Verify that user can update account informati on | Aeeou nt no is invalid | Pass |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Bill paymen t | Eleetri city, Water, phone | Manu a1 Appro aeh | Usern ame, Passw ord | Laptop, Phone | NO | N O | **Stepl-** verify that user can pay bill successf ully Step2- Verify that payment reflected in the user's transaeti on history of biller's records. Step3- Verify that appropri ate error message displaye d. . | Bill was not found in transa e tion list | Pass |
| Aeeoun t Seeuri t y | Length Compl exity | Manu a1 Appro aeh | Usern ame, Passw ord | Laptop, Phone | NO | N O | **Stepl-** verily that user can update their passwor d successf ully Step2— Verify that new passwor d meets the security Step3- Verify that it access two factors authentic ation. | The passwo rd entered is ineorre ' | Pass |

# EXP NO : 10

## Library Management System-Test Plan

Objective: Develop a comprehensive test plan document for a chosen application, such as a Library Management System, outlining the various testing strategies, methodologies, and criteria to ensure thorough evaluation and validation of the system's functionality.

| Items to be tested | features | Approach | Data test | Test environment | Training needed | Risk | Test cases | Pass/ fail |
|---|---|---|---|---|---|---|---|---|
| 1.Registration | User's authentication | Manual approa eh | Usemame password | Browser | No | No | 1.Open the website 2. enter the username/pa ssword 3.login 4.home page will appear | Pass |
| 2 Searehing for a book | Book identifieation | Manual approaeh | Book eategories | Browser | No | No | Stepl: In the home page , click the category. Step2: select the required category. Step3: choose the book from that category. | Pass |
| 3. Borrowing a book | Book borrowing | Manual approaeh | Book seleetion | Browser | No | No | Stepl: in the home page , click the category. Step2: select the required category. Step3: choose the book from the category. Step4: Borrow the book | Pass |
| 4. Returning a book | Book returning | Manual approaeh | Book seleetion | Browser | No | No | Stepl: in the home page , click the category. | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | Step2: select the return option.<br>Step3: return the book. | |
| 5.Adding a new book the library | New book addition | Manual approaeh | Book availability | Browser | No | No | **Stepl:** select the category of that book<br>Step2: add the number for that book.<br>Step3: add the book to the library . | Pass |
| 6. No. of book in the library | Number of books | Manual approaeh | Number Of books | Browser | No | No | **Stepl:** add the category in the library<br>Step2: add all the books in all category.<br>Step3: No. of books will be identified. | Pass |
| 7 Generating a report of borrowed books | Generalized report of borrowing | Manual approaeh | No .of books | Browser | No | No | **Stepl:** select the borrowed books<br>Step2: select the date ,in which the report need to generate.<br>Step3: report will be generate. | Pass |
| 8. Book-id number | Identifiea tion for the book | Manual approaeh | No. of books | Browser | No | No | **Stepl:** select the required book.<br>Step2: books contain a id number to categorize.<br>Step3: click the Book-id number | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 9. Security | Authentication purpose | Manual approaeh | Access control | Browser | No | No | **Stepl:** click the security, in the home page. Step2: select the security options. Step3: keep the security of user's wish. | Pass |
| 10. Payment | Renewal purpose /membership | Manual approaeh | Usema me / passwo rd | Browser | No | No | **Stepl:** click the payment option in the home page. Step2: if renewal-do payment Step3: If membership-fill details and pay. | Pass |

# EXP NO : 11

## Case study on Railway Reservation System - Positive Test Cases

Objective: identify and analyze the requirements of a Railway Reservation System and to write positive test scenarios and test cases.

| ITEMS TO BE TESTED | FEATURES | APPROACH | TEST DATA | TEST ENVIRONMENT | TRAINING NEEDED | RISK | TEST CASE SCENARIOS | PASS/ FAIL |
|---|---|---|---|---|---|---|---|---|
| **User Registration** | Name, Email, Mobile | Manual | Valid Info, 10-digit No | Laptop, Phone | NO | NO | **Step 1:** Verify user can successfully register with all valid mandatory fields. | Pass |
| **User Login** | Username, Password | Manual | Valid Credentials | Laptop, Phone | NO | NO | **Step 1:** Verify that a registered user can login to the portal successfully. | Pass |
| **Train Search** | Source, Destination | Manual | Chennai, Bangalore | Laptop, Phone | NO | NO | **Step 1:** Verify system displays available trains between two valid stations. | Pass |
| **Date Selection** | Journey Date | Manual | Future Date | Laptop, Phone | NO | NO | **Step 1:** Verify user can select a valid future date from the calendar. | Pass |
| **Class Selection** | SL, 3A, 2A, 1A | Manual | 3-Tier AC | Laptop, Phone | NO | NO | **Step 1:** Verify that seat availability updates correctly based on class selection. | Pass |
| **Passenger Details** | Name, Age, Gender | Manual | Valid Passenger Info | Laptop, Phone | NO | NO | **Step 1:** Verify user can add | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | multiple passengers (up to 6) in a single booking. | |
| **Berth Preference** | Lower, Middle, Upper | Manual | Lower Berth | Laptop, Phone | NO | NO | **Step 1:** Verify that the system records and accepts passenger berth preferences. | Pass |
| **Payment Gateway** | UPI, Net Banking | Manual | Valid Payment Apps | Laptop, Phone | NO | NO | **Step 1:** Verify that the user is redirected to the payment gateway and transaction is successful. | Pass |
| **PNR Generation** | Digital Ticket | Manual | Transaction ID | Laptop, Phone | NO | NO | **Step 1:** Verify that a unique 10-digit PNR is generated after successful payment. | Pass |
| **Ticket Download** | PDF, SMS | Manual | Confirmed PNR | Laptop, Phone | NO | NO | **Step 1:** Verify that the user can download the ticket in PDF format and receives a confirmation SMS. | Pass |

# EXP NO : 12

## Case study on Railway Reservation System - Negative Test Cases

Objective: To identify and analyze the requirements of a Railway Reservation System and to write negative Test Scenarios and Test Cases for the application.

| ITEMS TO BE TESTED | FEATURES | APPROACH | TEST DATA | TEST ENVIRONMENT | TRAINING NEEDED | RISK | TEST CASE SCENARIOS | PASS/ FAIL |
|---|---|---|---|---|---|---|---|---|
| **User Registration** | Email Validation | Manual | Invalid Email (no @) | Laptop, Phone | NO | NO | **Step 1:** Verify system displays an error message when an invalid email format is entered. | Pass |
| **User Login** | Security | Manual | Wrong Password | Laptop, Phone | NO | NO | **Step 1:** Verify that the system denies access and shows "Invalid Credentials" for an incorrect password. | Pass |
| **Train Search** | Route Logic | Manual | Same Station | Laptop, Phone | NO | NO | **Step 1:** Verify that the system displays an error if the 'Source' and 'Destination' stations are the same. | Pass |
| **Journey Date** | Date Validation | Manual | Past Date | Laptop, Phone | NO | NO | **Step 1:** Verify that the system restricts the user from selecting or searching for a date that has already passed. | Pass |
| **Passenger Info** | Age Constraint | Manual | Negative Age (-10) | Laptop, Phone | NO | NO | **Step 1:** Verify that the system | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | displays a validation error when a negative number or zero is entered in the age field. | |
| **Name Field** | Character Limit | Manual | Numbers/Symbols | Laptop, Phone | NO | NO | **Step 1:** Verify that the system rejects passenger names that contain numeric characters or special symbols. | Pass |
| **Seat Booking** | Booking Limit | Manual | 10 Passengers | Laptop, Phone | NO | NO | **Step 1:** Verify that the system does not allow adding more than the maximum limit (e.g., 6) of passengers per ticket. | Pass |
| **Payment Gateway** | Fund Check | Manual | Expired Card | Laptop, Phone | NO | NO | **Step 1:** Verify that the system handles payment failures gracefully and provides a "Transaction Failed" notification. | Pass |
| **PNR Inquiry** | PNR Length | Manual | 5-digit PNR | Laptop, Phone | NO | NO | **Step 1:** Verify that the system displays "Invalid PNR" when a number shorter than 10 digits is entered. | Pass |
| **Ticket** | Time | Manual | Departed Train | Laptop, Phone | NO | NO | **Step 1:** | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **Cancellation** | Constraint | | | | | | Verify that the "Cancel" option is disabled or shows an error once the train has departed from the source station. | |

## EXP NO : 13

**Case study on Online Examination System - Negative Test Cases**

Objective: To identify and analyze the requirements of an Online Examination System and to write positive Test Scenarios and Test Cases for the application.

| ITEMS TO BE TESTED | FEATURES | APPROACH | TEST DATA | TEST ENVIRONMENT | TRAINING NEEDED | RISK | TEST CASE SCENARIOS | PASS/ FAIL |
|---|---|---|---|---|---|---|---|---|
| **Student Registration** | Student ID, Email | Manual | Valid Student Details | Laptop, Browser | NO | NO | **Step 1:** Verify that a student can successfully register with a valid institutional email and ID. | Pass |
| **Student Login** | Auth Credentials | Manual | Valid Username/Pass | Laptop, Browser | NO | NO | **Step 1:** Verify that the student can login and access the examination dashboard. | Pass |
| **Profile Verification** | Photo/ID Upload | Manual | JPEG/PNG Image | Laptop, Webcam | NO | NO | **Step 1:** Verify that the system successfully captures or uploads the student's photo for proctoring. | Pass |
| **Exam Selection** | Subject List | Manual | Assigned Subject | Laptop, Browser | NO | NO | **Step 1:** Verify that the student can see and select only the exams they are eligible for. | Pass |
| **Instruction Page** | Guidelines | Manual | Click "Agree" | Laptop, Browser | NO | NO | **Step 1:** Verify that the student can read instructions and proceed to the start of the exam. | Pass |
| **Question Navigation** | Next/Previous | Manual | Radio Buttons | Laptop, Browser | NO | NO | **Step 1:** Verify that the student | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | can move between questions and select options successfully . | |
| **Save & Mark** | Review Later | Manual | Flag Feature | Laptop, Browser | NO | NO | **Step 1:** Verify that the student can mark a question for review and it appears highlighted in the palette. | Pass |
| **Timer Function** | Count-down | Manual | System Clock | Laptop, Browser | NO | NO | **Step 1:** Verify that the exam timer starts correctly and displays the remaining time accurately. | Pass |
| **Auto-Submit** | Session End | Manual | Time Expiry | Laptop, Browser | NO | NO | **Step 1:** Verify that the exam is automaticall y submitted when the time duration is completed. | Pass |
| **Result Generatio n** | Scorecard | Manual | Valid Answers | Laptop, Browser | NO | NO | **Step 1:** Verify that the student can view or download the scorecard after the evaluation is processed. | Pass |

# EXP NO : 14

## Case study on Online Examination System - Negative Test Cases

Objective: To identify and analyze the requirements of an Online Examination System and to write negative Test Scenarios and Test Cases for the application.

| ITEMS TO BE TESTED | FEATURES | APPROACH | TEST DATA | TEST ENVIRONMENT | TRAINING NEEDED | RISK | TEST CASE SCENARIOS | PASS/ FAIL |
|---|---|---|---|---|---|---|---|---|
| **User Login** | Authentication | Manual | Invalid Password | Laptop, Browser | NO | NO | **Step 1:** Verify system denies access and shows "Incorrect Password" after 3 failed attempts. | Pass |
| **Exam Access** | Eligibility | Manual | Expired Exam Link | Laptop, Browser | NO | NO | **Step 1:** Verify student cannot start the exam once the scheduled time window has closed. | Pass |
| **Proctoring Check** | Webcam Access | Manual | Camera Disabled | Laptop, Browser | NO | NO | **Step 1:** Verify the system prevents the exam from starting if the webcam is not detected or is blocked. | Pass |
| **Tab Switching** | Security | Manual | Alt + Tab | Laptop, Browser | NO | NO | **Step 1:** Verify that the system gives a warning or auto-submits if the student tries to switch browser tabs. | Pass |
| **Input** | Answer | Manual | Special | Laptop, Browser | NO | NO | **Step 1:** | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **Validation** | Field | | Symbols | | | | Verify that numeric-only fields (like math answers) reject alphabetic characters or symbols. | |
| **Multiple Login** | Concurrent Session | Manual | Same Credentials | 2 Laptops | NO | NO | **Step 1:** Verify that the system logs out the first session if the same user ID is logged in on another device. | Pass |
| **File Upload** | Assignment Size | Manual | 100MB File | Laptop, Browser | NO | NO | **Step 1:** Verify that the system rejects file uploads that exceed the maximum size limit (e.g., 5MB). | Pass |
| **Network Loss** | Connectivity | Manual | WiFi Disconnected | Laptop, Browser | NO | NO | **Step 1:** Verify that the system displays a "No Connection" alert and prevents data loss during a logout. | Pass |
| **Submit Button** | Blank Submission | Manual | No Questions Selected | Laptop, Browser | NO | NO | **Step 1:** Verify that the system prompts a warning if the student tries to submit without answering any questions. | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **URL Manipulation** | Unauthorized Access | Manual | Direct Result URL | Laptop, Browser | NO | NO | **Step 1:** Verify that a student cannot access the | Pass |



| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | "Answers Key" page by manually changing the URL in the browser. | |

# EXP NO : 15

**Inventory Management System - Test Plan** Objective:

To develop a comprehensive test plan for an Inventory Management System, outlining test scope, design techniques, and execution strategies to ensure the accurate tracking of stock, sales, and orders.

| ITEMS TO BE TESTED | FEATURES | APPROACH | DATA TEST | TEST ENVIRONMENT | TRAINING NEEDED | RISK | TEST CASES | PASS/ FAIL |
|---|---|---|---|---|---|---|---|---|
| **1. Admin Login** | Authentication | Manual | Admin ID, Password | Browser | NO | NO | **Step 1:** Open Login URL. **Step 2:** Enter Admin credentials. **Step 3:** Dashboard should appear. | Pass |
| **2. Add New Product** | Inventory Entry | Manual | Product Name, SKU, Qty | Browser | NO | NO | **Step 1:** Click 'Add Product'. **Step 2:** Enter details & price. **Step 3:** Item should list in stock. | Pass |
| **3. Stock Update** | Stock Level | Manual | Numeric Values | Browser | NO | NO | **Step 1:** Select an existing product. **Step 2:** Increase/Decrease quantity. **Step 3:** Total stock updates. | Pass |
| **4. Supplier Management** | Supplier Info | Manual | Supplier Name, Contact | Browser | NO | NO | **Step 1:** Add a new supplier. **Step 2:** Link supplier to a product. **Step 3:** Save and verify listing. | Pass |
| **5. Purchase Order** | Ordering | Manual | Order ID, Supplier ID | Browser | NO | NO | **Step 1:** Create a new purchase order. **Step 2:** Add items to order. **Step 3:** Order status changes to 'Sent'. | Pass |
| **6. Sales Tracking** | Transaction | Manual | Sale ID, Customer Info | Browser | NO | NO | **Step 1:** Record a new sale. **Step 2:** Verify stock reduces | Pass |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | automatically. **Step 3:** Receipt is generated. | |
| **7. Low Stock Alerts** | Notification | Manual | Threshold Value | Browser | NO | NO | **Step 1:** Set alert at 5 units. **Step 2:** Reduce stock to 4. **Step 3:** Alert message appears on dashboard. | Pass |
| **8. Barcode Scanning** | Product ID | Manual | Barcode Serial | Mobile/ Browser | NO | NO | **Step 1:** Scan a product barcode. **Step 2:** System identifies product. **Step 3:** Item details are displayed. | Pass |
| **9. Report Generation** | Analytics | Manual | Monthly Date Range | Browser | NO | NO | **Step 1:** Select 'Monthly Sales Report'. **Step 2:** Choose Feb 2026. **Step 3:** Export report as PDF/Excel. | Pass |
| **10. Database Backup** | Data Security | Manual | SQL Backup File | Server | NO | NO | **Step 1:** Click 'Initiate Backup'. **Step 2:** Download backup file. **Step 3:** Verify data integrity. | Pass |

## EXP NO : 16

## JUNIT PROGRAM TO CHECK ANAGRAM STRINGS

### Aim:

To check whether a string is an anagram of another string and validate output using Junit assertions.

### Algorithm/Procedure:

1. Read two strings s1 and s2.
2. Remove spaces and convert to lowercase.
3. Convert strings to character arrays.
4. Sort both arrays.
5. Compare both arrays using Arrays.equals().
6. Return true if equal else false.

### Code/Program:

```
package junit;

import java.util.Arrays;
import org.junit.Test;
import static org.junit.Assert.*;
public class AnaagramTest {
	public boolean isanagram(String s1,String s2) {
		if(s1 == null|| s2 == null)
			return false;
		s1 = s1.replaceAll("\\s","").toLowerCase();
		s2 = s2.replaceAll("\\s","").toLowerCase();
		if(s1.length() != s2.length())
			return false;
		char[] a1 = s1.toCharArray();
		char[] a2 = s2.toCharArray();
		Arrays.sort(a1);
		Arrays.sort(a2);

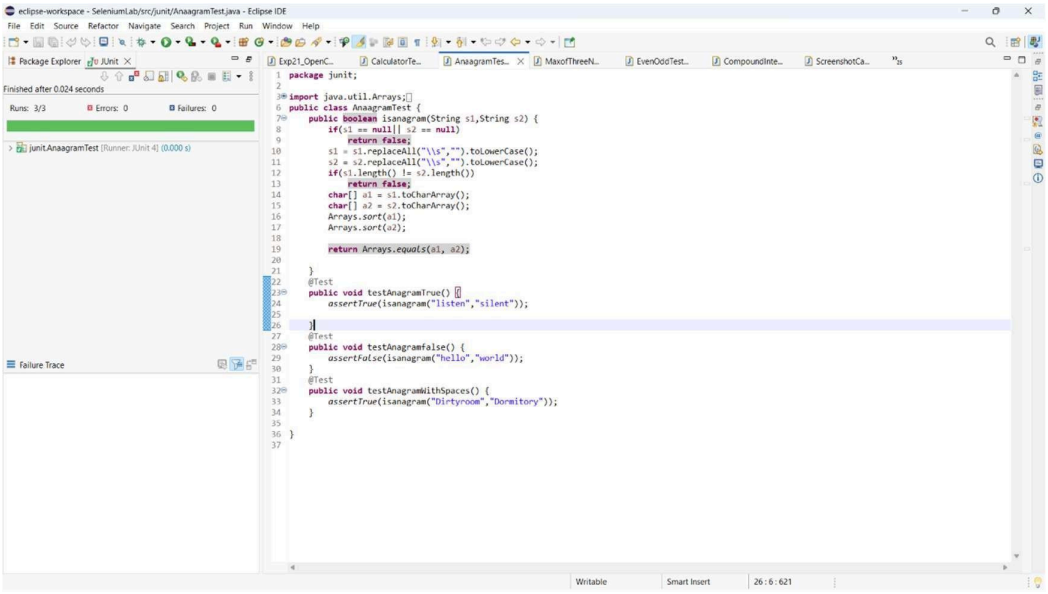
		return Arrays.equals(a1, a2);

	}
	@Test
	public void testAnagramTrue() {
		assertTrue(isanagram("listen","silent"));

	}
	@Test
	public void testAnagramfalse() {
		assertFalse(isanagram("hello","world"));
	}
```

```
	@Test
	public void testAnagramWithSpaces() {
		assertTrue(isanagram("Dirtyroom","Dormitory"));
	}

}
```




**White Box Testcases:**

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | listen, silent | True |
| TC2 | hello, world | False |
| TC3 | Dirty room, Dormitory | True |

**Input: Output:**

l

i

s

t

e

n

**Result:**

,

s

i

l

e

n

t

T

r

u

e

Thus, anagram checking was executed successfully and validated using JUnit.

## EXP NO : 17

## JUNIT PROGRAM TO FIND MAXIMUM OF THREE NUMBERS

### Aim:

To find the maximum of three numbers and validate output using JUnit assertions.

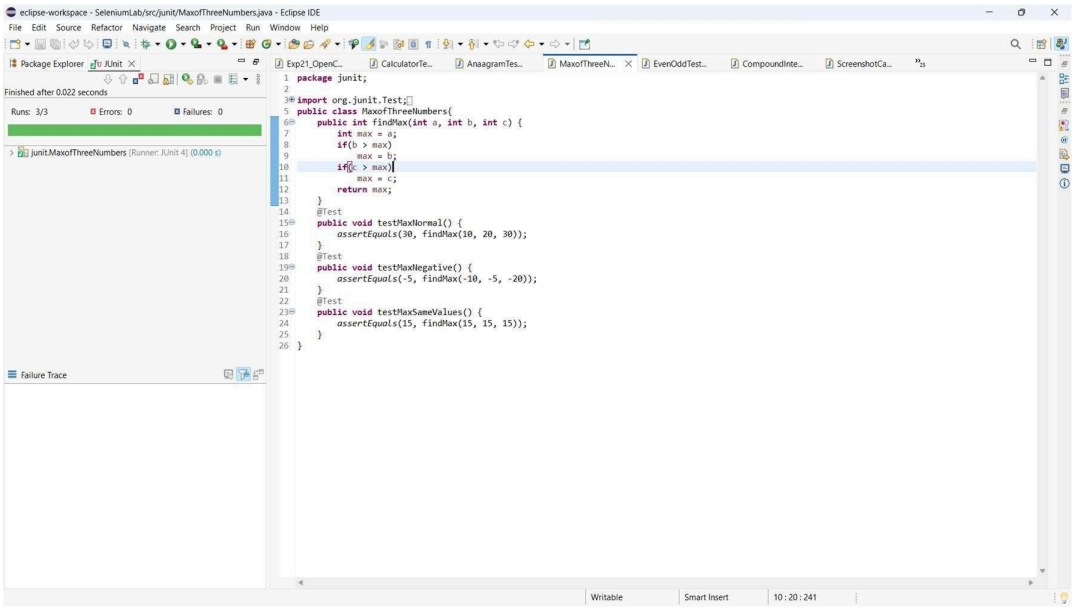
### Algorithm/Procedure:

1. Read three integers a, b, c.
2. Initialize max=a.
3. If b > max, set max=b.
4. If c > max, set max=c.
5. Return max.
6. Return true if equal else false.

### Code/Program:

```
package junit;

import org.junit.Test;
import static org.junit.Assert.*; public
class MaxofThreeNumbers{
   public int findMax(int a, int b, int c) { int
      max = a;
       if(b > max)
          max = b;
       if(c > max)
          max = c;
       return max;
   }
   @Test
   public void testMaxNormal() { assertEquals(30,
      findMax(10, 20, 30));
   }
   @Test
   public void testMaxNegative() {
      assertEquals(-5, findMax(-10, -5, -20));
   }
   @Test
   public void testMaxSameValues() {
      assertEquals(15, findMax(15, 15, 15));
   }}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | 10,20,30 | 30 |
| TC2 | -10,-5,-20 | -5 |
| TC3 | 5,15,15 | 15 |

**Input: Output:** 10,20,30

30

**Result:**

Thus, maximum of three numbers was executed successfully and validated using JUnit.

## EXP NO : 18

# JUNIT PROGRAM TO CHECK EVEN OR ODD NUMBER

## Aim:

To check whether a number is even or odd and validate output using JUnit assertions

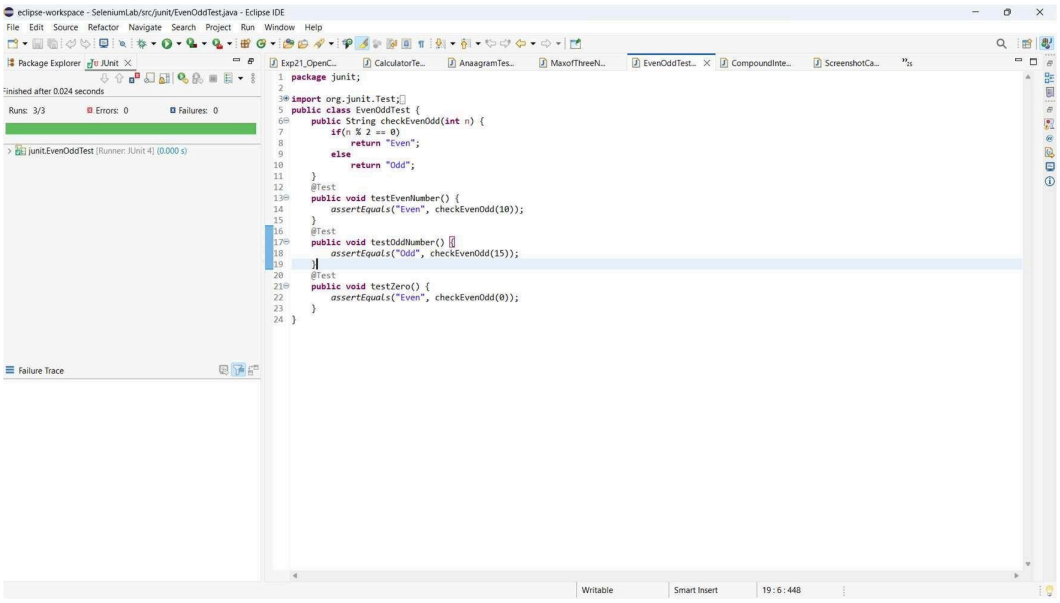
## Algorithm/Procedure:

1. Read integer n.
2. If n % 2 == 0 return Even.
3. Else return Odd..

## Code/Program:

```
package junit;

import org.junit.Test;
import static org.junit.Assert.*;
public class EvenOddTest {
   public String checkEvenOdd(int n) {
      if(n % 2 == 0)
         return "Even";
      else
         return "Odd";
   }
   @Test
   public void testEvenNumber() {
      assertEquals("Even", checkEvenOdd(10));
   }
   @Test
   public void testOddNumber() {
      assertEquals("Odd", checkEvenOdd(15));
   }
   @Test
   public void testZero() { assertEquals("Even",
      checkEvenOdd(0));
   }
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | 10 | Even |
| TC2 | 15 | Odd |
| TC3 | 0 | Even |

**Input: Output:**

15

Odd

**Result:**

Thus, even or odd checking was executed successfully and validated using JUnit.

## EXP NO : 19

## COMPOUND INTEREST CALCULATION USING JUNIT

### Aim:

To calculate compound interest and validate output using JUnit assertions.

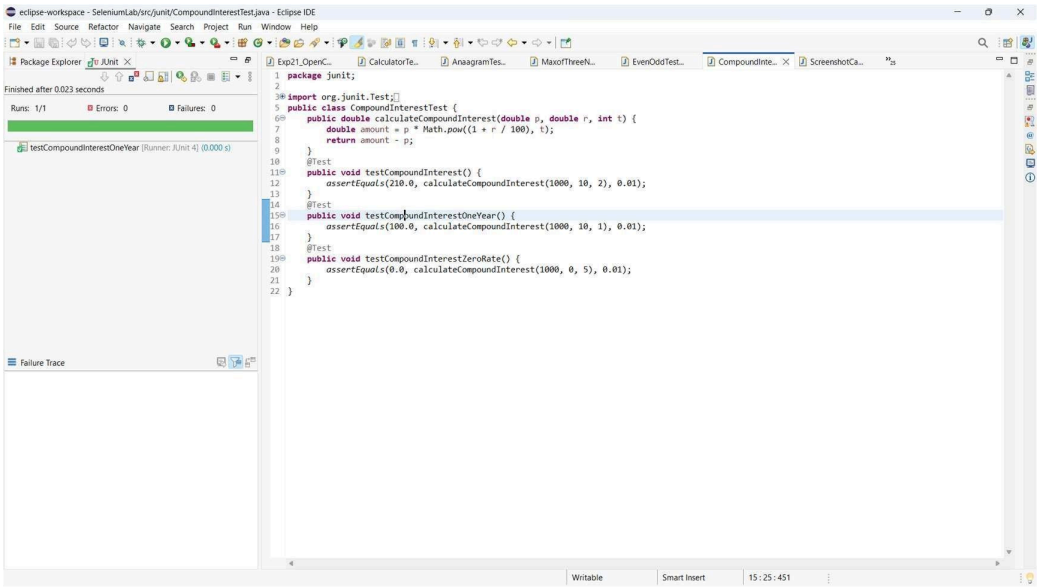
### Algorithm/Procedure:

1. Read principal P, rate R, time T.
2. Calculate amount A = P(1+R/100)^T.
3. Calculate CI = A - P.
4. Return CI.

### Code/Program:

```
package junit;

import org.junit.Test;
import static org.junit.Assert.*; public
class CompoundInterestTest {
    public double calculateCompoundInterest(double p, double r, int t) {
        double amount = p * Math.pow((1 + r / 100), t);
        return amount - p;
    }
    @Test
    public void testCompoundInterest() {
        assertEquals(210.0, calculateCompoundInterest(1000, 10, 2), 0.01);
    }
    @Test
    public void testCompoundInterestOneYear() {
        assertEquals(100.0, calculateCompoundInterest(1000, 10, 1), 0.01);
    }
    @Test
    public void testCompoundInterestZeroRate() {
        assertEquals(0.0, calculateCompoundInterest(1000, 0, 5), 0.01);
    }
}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | P=1000,R=10,T=2 | 210 |
| TC2 | P=1000,R=10,T=1 | 100 |
| TC3 | P=1000,R=10,T=1 | 0 |

**Input: Output:**

P

=

1

0

0

0

,

R

=

1

0

**Result:**

| | |
|---|---|
| , | I |
| T | = |
| = | 2 |
| 2 | 1 |
| C | 0 |

Thus, compound interest calculation was executed successfully and validated using JUnit.

## EXP NO : 20

# COUNT VOWELS AND CONSONANTS USING JUNIT

## Aim:

To count the number of vowels and consonants in a given string and validate output using JUnit assertions.

## Algorithm/Procedure:

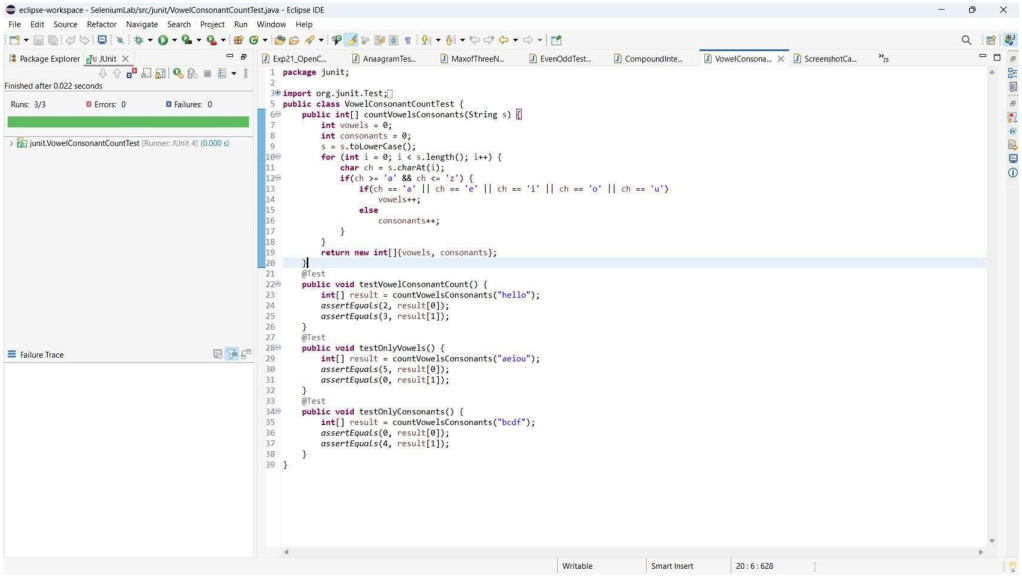
1. Read string s.
2. Initialize vowels=0, consonants=0.
3. Traverse each character.
4. If character is vowel increment vowels.
5. Else if alphabet increment consonants. 6. Return count values..

## Code/Program:

```
package junit;

import org.junit.Test;
import static org.junit.Assert.*;
public class VowelConsonantCountTest {
   public int[] countVowelsConsonants(String s) { int
      vowels = 0;
      int consonants = 0;
      s = s.toLowerCase();
      for (int i = 0; i < s.length(); i++) {
         char ch = s.charAt(i);
         if(ch >= 'a' && ch <= 'z') {
            if(ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
               vowels++;
            else
               consonants++;
         }
      }
      return new int[]{vowels, consonants};
   }
   @Test
   public void testVowelConsonantCount() {
      int[] result = countVowelsConsonants("hello");
      assertEquals(2, result[0]);
      assertEquals(3, result[1]);
   }
   @Test
   public void testOnlyVowels() {
      int[] result = countVowelsConsonants("aeiou");
      assertEquals(5, result[0]);
      assertEquals(0, result[1]);
   }
   @Test
   public void testOnlyConsonants() {
      int[] result = countVowelsConsonants("bcdf");
      assertEquals(0, result[0]);
```

```
assertEquals(4, result[1]);}}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | hello | V=2,C=3 |
| TC2 | aeiou | V=5,C=0 |
| TC3 | bcdf | V=0,C=4 |

## Input: Output:

Hello

Vowels=2, Consonants=3

## Result:

Thus, vowels and consonants counting was executed successfully and validated using JUnit.

## EXP NO : 21

## PRIME NUMBER CHECKING USING JUNIT

## Aim:

To check whether a given number is prime and validate output using JUnit assertions.

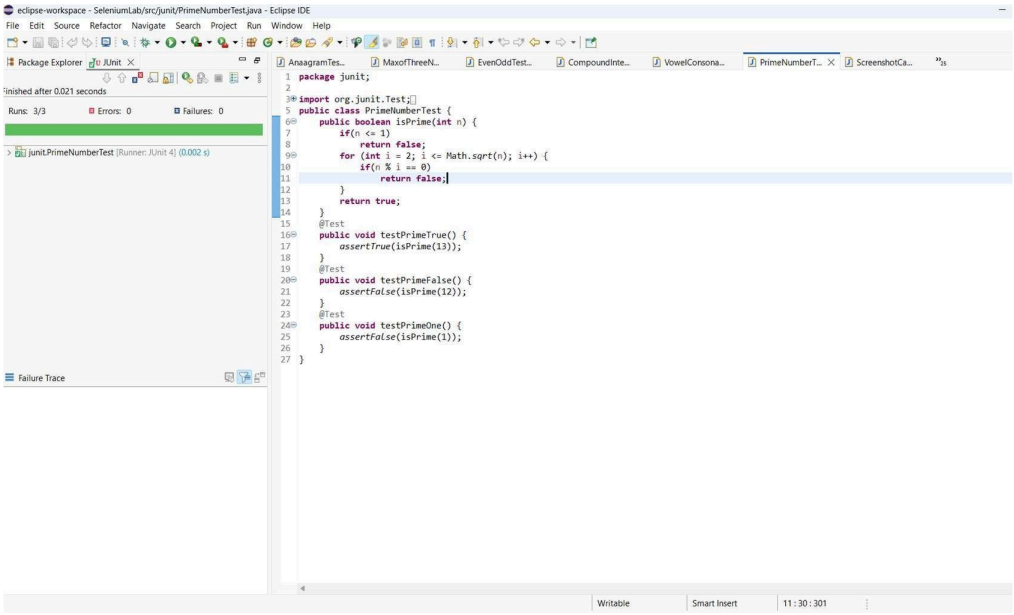
## Algorithm/Procedure:

1. Read integer n.
2. If n <= 1 return false.
3. Loop i from 2 to sqrt(n).
4. If n % i == 0 return false.
5. Return true.

## Code/Program:

```
package junit;

import org.junit.Test;
import static org.junit.Assert.*; public
class PrimeNumberTest {
    public boolean isPrime(int n) {
        if(n <= 1)
            return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if(n % i == 0)
                return false;
        }
        return true;
    }
    @Test
    public void testPrimeTrue() {
        assertTrue(isPrime(13));
    }
    @Test
    public void testPrimeFalse() {
        assertFalse(isPrime(12));
    }
    @Test
    public void testPrimeOne() {
        assertFalse(isPrime(1));
    }}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | 13 | True |
| TC2 | 12 | False |
| TC3 | 1 | False |

## Input: Output:

13

False

## Result:

Thus, prime number checking was executed successfully and validated using JUnit.

## EXP NO : 22

## REVERSE A NUMBER USING JUNIT

## Aim:

To reverse a given number and validate output using JUnit assertions.

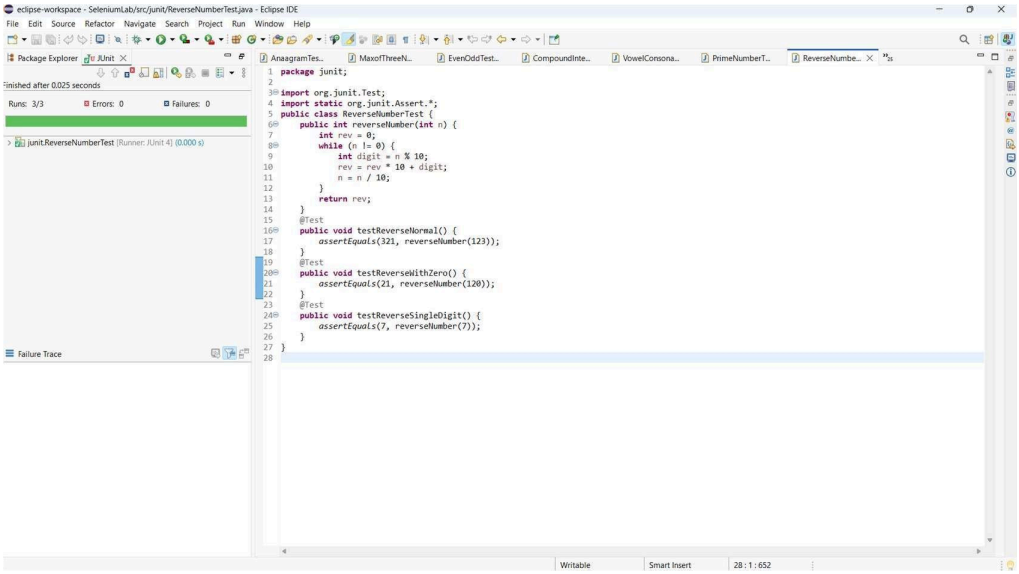
## Algorithm/Procedure:

1. Read integer n.
2. Initialize rev=0.
3. While n != 0: digit=n%10, rev=rev*10+digit, n=n/10.
4. Return rev.

## Code/Program:

```
package junit;

import org.junit.Test;
import static org.junit.Assert.*; public
class ReverseNumberTest {
    public int reverseNumber(int n) {
        int rev = 0;
        while (n != 0) {
            int digit = n % 10; rev
            = rev * 10 + digit; n =
            n / 10;
        }
        return rev;
    }
    @Test
    public void testReverseNormal() {
        assertEquals(321, reverseNumber(123));
    }
    @Test
    public void testReverseWithZero() {
        assertEquals(21, reverseNumber(120));
    }
    @Test
    public void testReverseSingleDigit() {
        assertEquals(7, reverseNumber(7));
    }}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | 123 | 321 |
| TC2 | 120 | 21 |
| TC3 | 7 | 7 |

## Input: Output:

123

321

## Result:

Thus, reverse number program was executed successfully and validated using JUnit.

## EXP NO : 23

## SUM OF DIGITS OF A NUMBER USING JUNIT

## Aim:

To find the sum of digits of a given number and validate output using JUnit assertions.

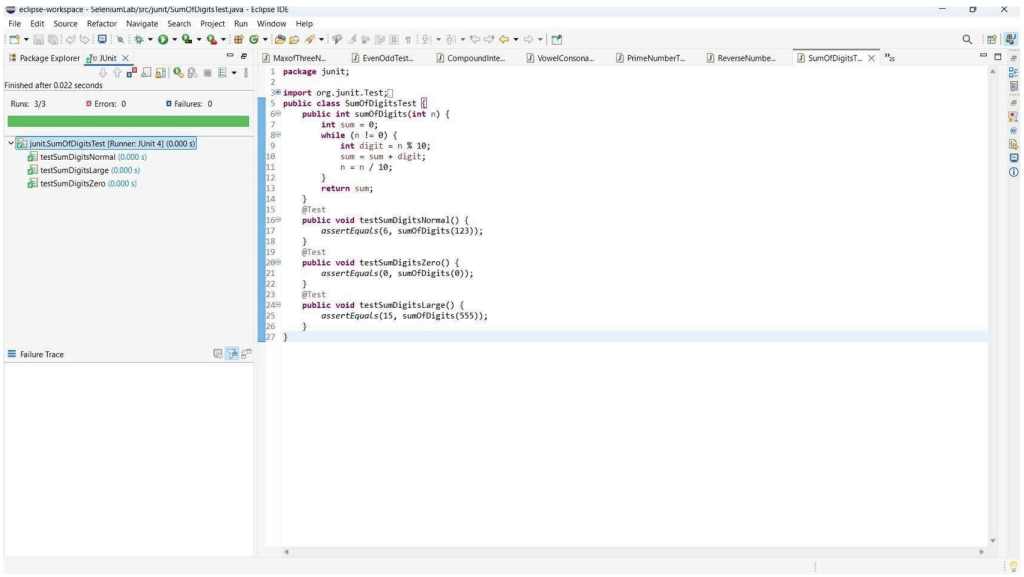
## Algorithm/Procedure:

1 Read integer n.
2. Initialize sum=0.
3. While n != 0: digit=n%10, sum=sum+digit, n=n/10.
4. Return sum.

## Code/Program:

```
package junit;

import org.junit.Test;
import  static  org.junit.Assert.*;
public class SumOfDigitsTest {
   public  int  sumOfDigits(int n)  {
      int sum = 0;
      while (n != 0) {
         int  digit = n % 10;
         sum = sum + digit;
         n = n / 10;
      }
      return sum;
   }
   @Test
   public void testSumDigitsNormal() {
      assertEquals(6, sumOfDigits(123));
   }
   @Test
   public void testSumDigitsZero() {
      assertEquals(0, sumOfDigits(0));
   }
   @Test
   public void testSumDigitsLarge() {
      assertEquals(15, sumOfDigits(555));
   }
}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | 123 | 6 |
| TC2 | 0 | 0 |
| TC3 | 555 | 15 |

## Input: Output:

123

6

## Result:

Thus, sum of digits program was executed successfully and validated using Junit.

## EXP NO : 24

## STRING EQUALITY IGNORING CASE USING JUNIT

## Aim:

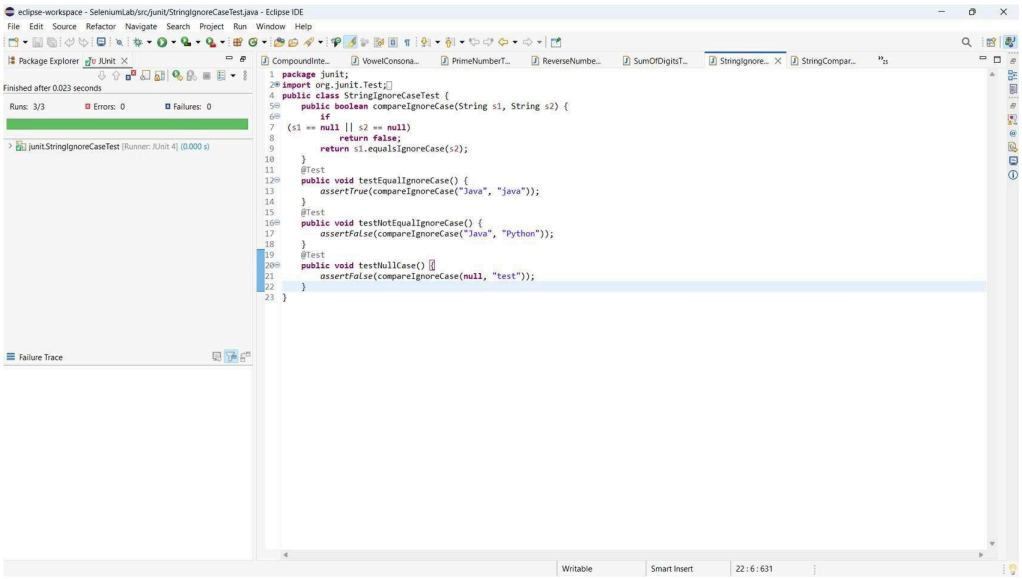
To check whether two strings are equal ignoring case sensitivity and validate output using JUnit assertions.

## Algorithm/Procedure:

1 Read two strings s1 and s2.
2. Convert both strings to lowercase.
3. Compare using equals().
4. Return true if equal else false.

## Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*; public
class StringComparisonTest {
    public boolean compareStrings(String s1, String s2) {
        if
 (s1 == null || s2 == null)
            return false;
        return s1.equals(s2);
    }
    @Test
    public void testStringEqual() {
        assertTrue(compareStrings("Java", "Java"));
    }
    @Test
    public void testStringNotEqual() {
        assertFalse(compareStrings("Java", "Python"));
    }
    @Test
    public void testNullString() {
        assertFalse(compareStrings(null, "Test"));
    }}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | Java, java | True |
| TC2 | Java,Python | False |
| TC3 | null, test | False |

**Input: Output:**

J

a

v

a

,

j

a

v

a

T

r

u

**Result:**

e

Thus, string comparison ignoring case was executed successfully and validated using JUnit.

## EXP NO : 25

## STUDENT GRADE CALCULATION USING JUNIT

## Aim:

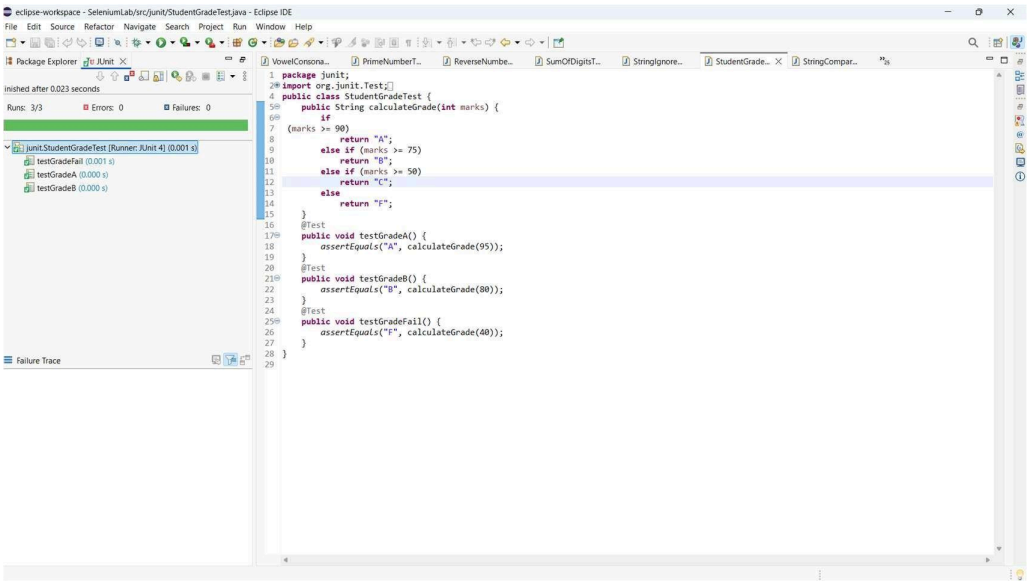
To calculate grade of a student based on marks and validate output using JUnit assertions.

## Algorithm/Procedure:

1. Read marks.
2. If marks >= 90 grade A.
3. Else if marks >= 75 grade B.
4. Else if marks >= 50 grade C.
5. Else grade F.
6. Return grade.

## Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*; public
class StudentGradeTest {
    public String calculateGrade(int marks) {
        if
 (marks >= 90)
            return "A";
        else if (marks >= 75)
            return "B";
        else if (marks >= 50)
            return "C";
        else
            return "F";
    }
    @Test
    public void testGradeA() {
        assertEquals("A", calculateGrade(95));
    }
    @Test
    public void testGradeB() {
        assertEquals("B", calculateGrade(80));
    }
    @Test
    public void testGradeFail() {
        assertEquals("F", calculateGrade(40));
    }}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | 95 | A |
| TC2 | 80 | B |
| TC3 | 40 | F |

**Input:** 80

**Output:** B

**Result:**

Thus, student grade calculation was executed successfully and validated using JUnit.

## EXP NO : 26

# FIND LARGEST ELEMENT IN AN ARRAY USING JUNIT

## Aim:

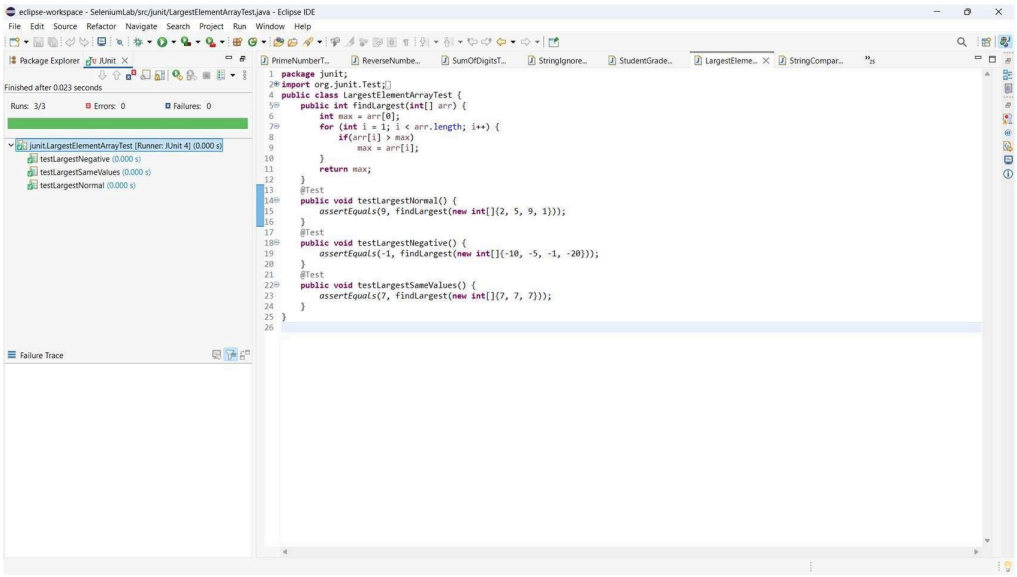
To find the largest element in an array and validate output using JUnit assertions.

## Algorithm/Procedure:

1. Read array elements.
2. Initialize max = arr[0].
3. Traverse array elements.
4. If arr[i] > max then update max.
5. Return max.

## Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*;
public class LargestElementArrayTest {
    public int findLargest(int[] arr) {
        int max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if(arr[i] > max)
                max = arr[i];
        }
        return max;
    }
    @Test
    public void testLargestNormal() {
        assertEquals(9, findLargest(new int[]{2, 5, 9, 1}));
    }
    @Test
    public void testLargestNegative() {
        assertEquals(-1, findLargest(new int[]{-10, -5, -1, -20}));
    }
    @Test
    public void testLargestSameValues() { assertEquals(7,
        findLargest(new int[]{7, 7, 7}));
    }}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | {2,5,9,1} | 9 |
| TC2 | {-10,-5,-1,-20} | -1 |
| TC3 | {7,7,7} | 7 |

## Input: Output:

{2,5,9,1}

9

## Result:

Thus, largest element finding was executed successfully and validated using JUnit.

## EXP NO : 27

## CALCULATE AVERAGE OF N NUMBERS USING JUNIT

### Aim:

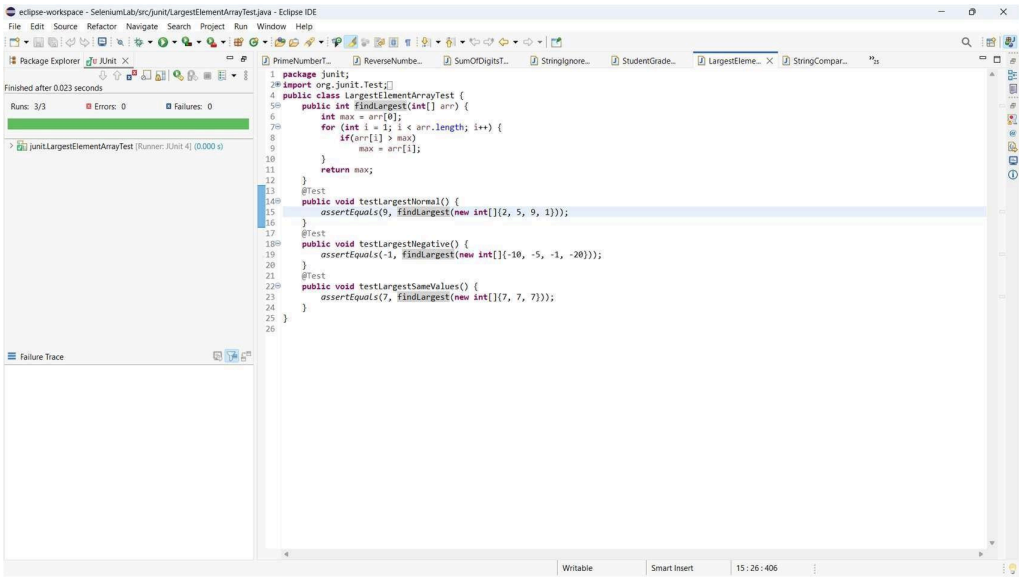
To calculate the average of n numbers using functions and validate output using JUnit assertions.

### Algorithm/Procedure:

1. Read array of numbers.
2. Calculate sum of all numbers.
3. Divide sum by count of numbers.
4. Return average.

### Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*;
public class AverageOfNumbersTest {
    public double findAverage(int[] arr) {
        int sum = 0;
        for (int num : arr) {
            sum += num;
        }
        return (double) sum / arr.length;
    }
    @Test
    public void testAverageNormal() {
        assertEquals(3.0, findAverage(new int[]{1, 2, 3, 4, 5}), 0.01);
    }
    @Test
    public void testAverageSingle() {
        assertEquals(10.0, findAverage(new int[]{10}), 0.01);
    }
    @Test
    public void testAverageZeros() {
        assertEquals(0.0, findAverage(new int[]{0, 0, 0}), 0.01);
    }}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | {1,2,3,4,5} | 3.0 |
| TC2 | {10} | 10.0 |
| TC3 | {0,0,0} | 0.0 |

**Input:** {1,2,3,4,5}

**Output:** 3.0

**Result:**

Thus, average calculation was executed successfully and validated using JUnit.

## EXP NO : 28

# CHECK ARMSTRONG NUMBER USING JUNIT

## Aim:

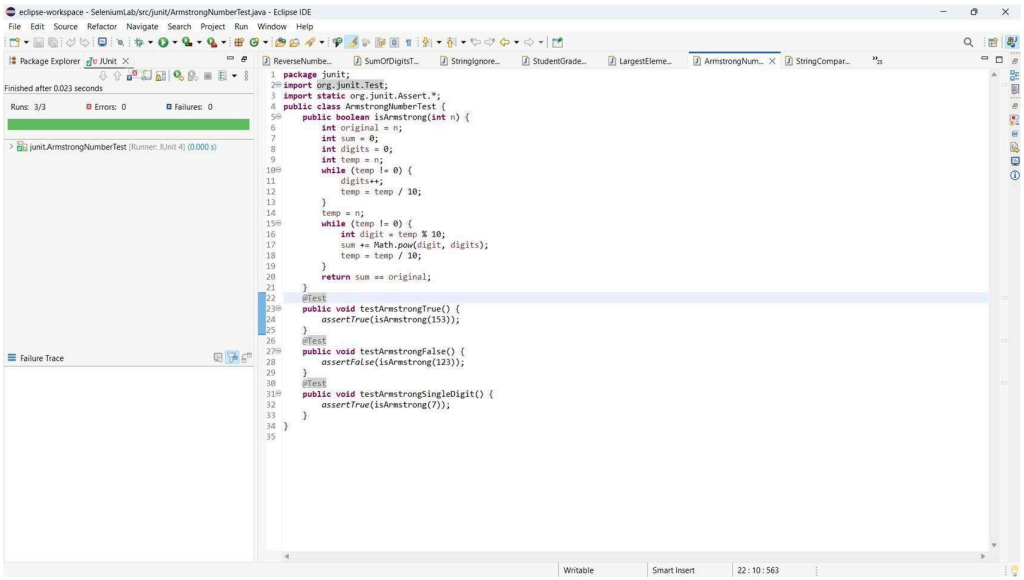
To check whether a given number is an Armstrong number and validate output using JUnit assertions.

## Algorithm/Procedure:

1. Read number n.
2. Store original number.
3. Count digits.
4. Compute sum of each digit power of digits count.
5. Compare sum with original number.
6. Return true if equal else false.

## Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*; public
class ArmstrongNumberTest {
    public boolean isArmstrong(int n) {
        int original = n;
        int sum = 0;
        int digits = 0;
        int temp = n;
        while (temp != 0) {
            digits++;
            temp = temp / 10;
        }
        temp = n;
        while (temp != 0) {
            int digit = temp % 10;
            sum += Math.pow(digit, digits);
            temp = temp / 10;
        }
        return sum == original;
    }
    @Test
    public void testArmstrongTrue() {
        assertTrue(isArmstrong(153));
    }
    @Test
    public void testArmstrongFalse() {
        assertFalse(isArmstrong(123));
    }
    @Test
    public void testArmstrongSingleDigit() {
        assertTrue(isArmstrong(7));
    }}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | 153 | True |
| TC2 | 123 | False |
| TC3 | 7 | True |

## Input: Output:

153

True

## Result:

Thus, Armstrong number checking was executed successfully and validated using JUnit.

## EXP NO : 29

# CONVERT CELSIUS TO FAHRENHEIT USING JUNIT

## Aim:

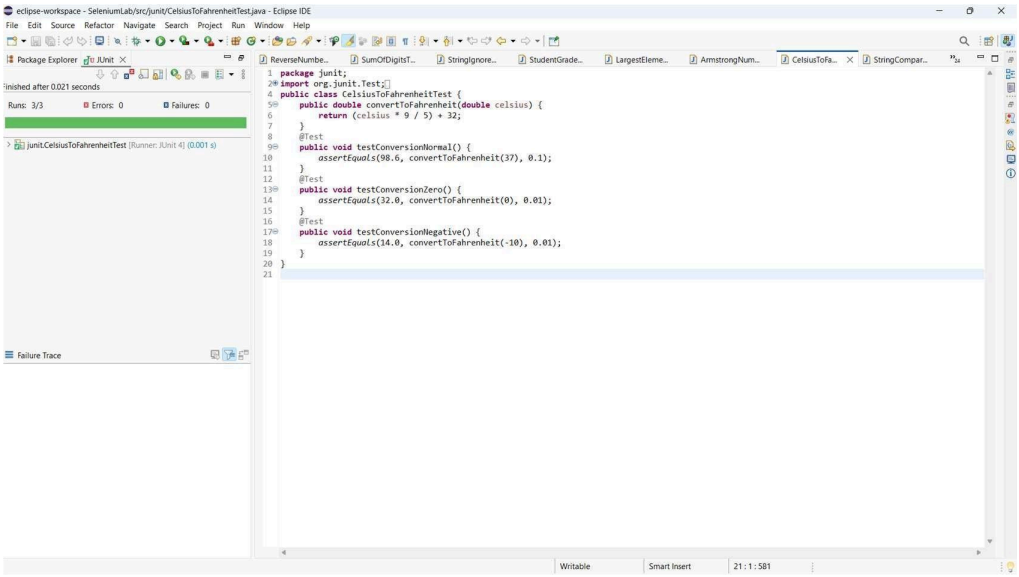
To convert temperature from Celsius to Fahrenheit and validate output using JUnit assertions.

## Algorithm/Procedure:

1. Read temperature in Celsius.
2. Apply formula F = (C * 9/5) + 32.
3. Return Fahrenheit value.

## Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*;
public class CelsiusToFahrenheitTest {
   public double convertToFahrenheit(double celsius) { return
      (celsius * 9 / 5) + 32;
   }
   @Test
   public void testConversionNormal() {
      assertEquals(98.6, convertToFahrenheit(37), 0.1);
   }
   @Test
   public void testConversionZero() { assertEquals(32.0,
      convertToFahrenheit(0), 0.01);
   }
   @Test
   public void testConversionNegative() {
      assertEquals(14.0, convertToFahrenheit(-10), 0.01);
   }
}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | 37 | 98.6 |
| TC2 | 0 | 32 |
| TC3 | -10 | 14 |

## Input: Output:

37 Celsius

98.6 Fahrenheit

## Result:

Thus, Celsius to Fahrenheit conversion was executed successfully and validated using JUnit.

## EXP NO : 30

## FIND LENGTH OF A STRING WITHOUT BUILT-IN FUNCTION USING JUNIT

## Aim:

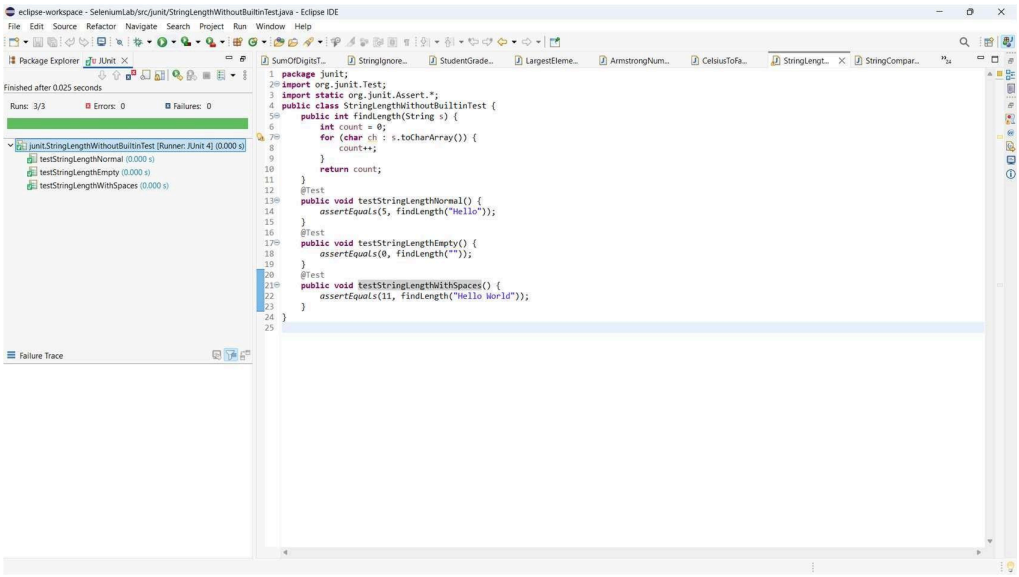
To find the length of a string without using built-in length() function and validate output using JUnit assertions.

## Algorithm/Procedure:

1. Read string s.
2. Convert string to char array.
3. Initialize count = 0.
4. Traverse each character and increment count.
5. Return count.

## Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*;
public class StringLengthWithoutBuiltinTest {
    public int findLength(String s) {
        int count = 0;
        for (char ch : s.toCharArray()) {
            count++;
        }
        return count;
    }
    @Test
    public void testStringLengthNormal() {
        assertEquals(5, findLength("Hello"));
    }
    @Test
    public void testStringLengthEmpty() {
        assertEquals(0, findLength(""));
    }
    @Test
    public void testStringLengthWithSpaces() {
        assertEquals(11, findLength("Hello World"));
    }}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | Hello | 5 |
| TC2 | (empty string) | 0 |
| TC3 | Hello World | 11 |

**Input: Output:**

H

e

l

l

o

**Result:**

5

Thus, string length calculation without using built-in function was executed successfully and validated using JUnit.

## EXP NO : 31

## REVERSE A WORD USING JUNIT

## Aim:

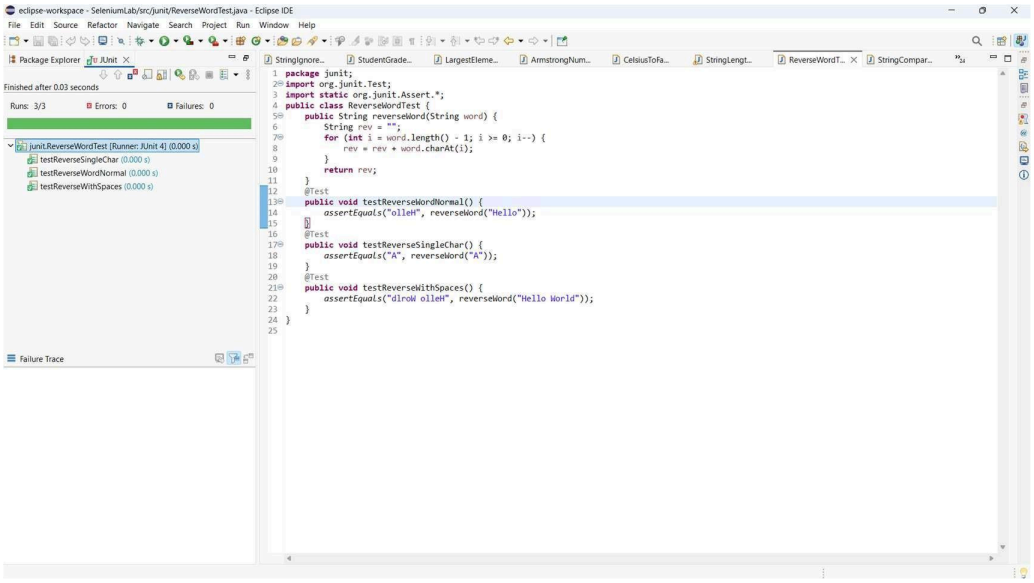
To reverse a given word and validate output using JUnit assertions.

## Algorithm/Procedure:

1. Read input string.
2. Initialize empty string rev.
3. Traverse string from end to start.
4. Append characters to rev.
5. Return rev.

## Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*; public
class ReverseWordTest {
    public String reverseWord(String word) {
        String rev = "";
        for (int i = word.length() - 1; i >= 0; i--) { rev
            = rev + word.charAt(i);
        }
        return rev;
    }
    @Test
    public void testReverseWordNormal() {
        assertEquals("olleH", reverseWord("Hello"));
    }
    @Test
    public void testReverseSingleChar() {
        assertEquals("A", reverseWord("A"));
    }
    @Test
    public void testReverseWithSpaces() {
        assertEquals("dlroW olleH", reverseWord("Hello World"));
    }
}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | Hello | olleH |
| TC2 | A | A |
| TC3 | Hello World | dlroW olleH |

**Input: Output:**

H

e

l

l

o

o

l

l

e

H

**Result:**

Thus, word reversing was executed successfully and validated using JUnit.

## EXP NO : 32

## STRING COMPARISON USING JUNIT

## Aim:

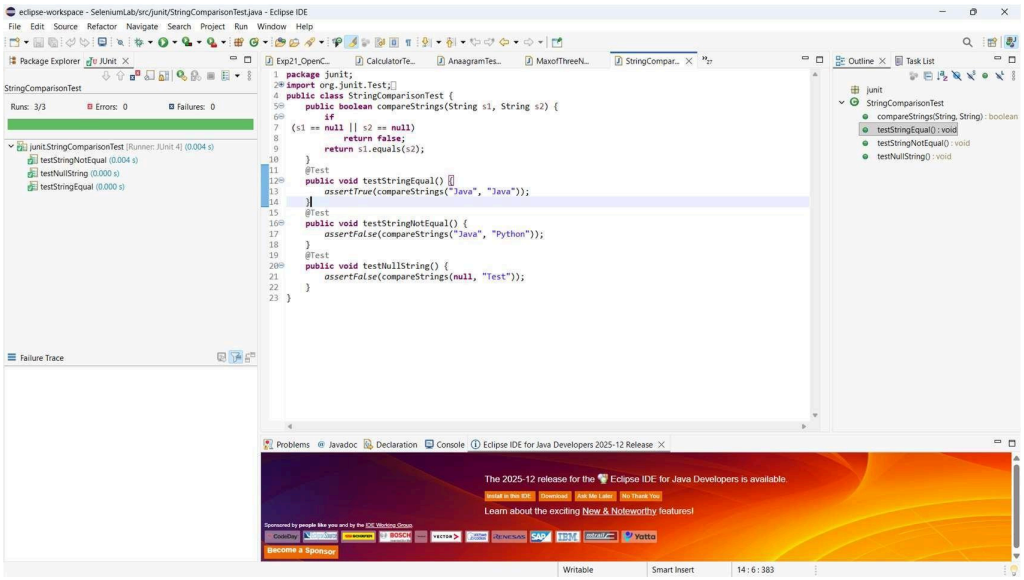
To perform string comparison and validate output using JUnit assertions.

## Algorithm/Procedure:

1. Read two strings s1 and s2.
2. Compare strings using equals().
3. Return true if equal else false.

## Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*; public
class StringComparisonTest {
    public boolean compareStrings(String s1, String s2) {
        if
 (s1 == null || s2 == null)
            return false;
        return s1.equals(s2);
    }
    @Test
    public void testStringEqual() {
        assertTrue(compareStrings("Java", "Java"));
    }
    @Test
    public void testStringNotEqual() {
        assertFalse(compareStrings("Java", "Python"));
    }
    @Test
    public void testNullString() {
        assertFalse(compareStrings(null, "Test"));
    }}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | Java, Java | True |
| TC2 | Java, Python | False |
| TC3 | null, Test | False |

**Input: Output:**

J

a

v

a

,

J

a

v

a

T

r

u

**Result:**

e

Thus, string comparison was executed successfully and validated using JUnit.

## EXP NO : 33

# VOTING SYSTEM USING JUNIT

## Aim:

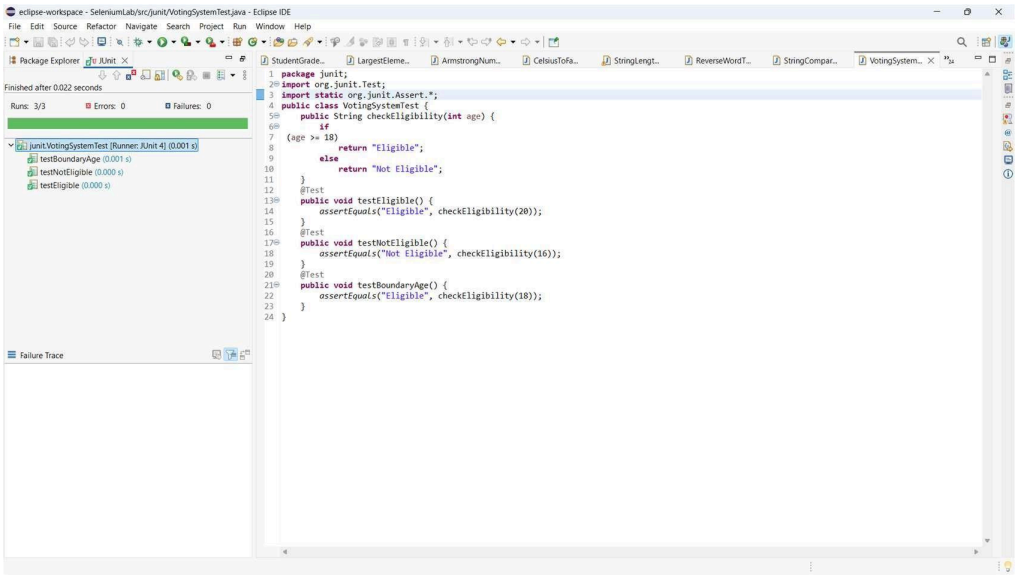
To validate voting eligibility based on age using JUnit assertions.

## Algorithm/Procedure:

1. Read age.
2. If age >= 18 return Eligible.
3. Else return Not Eligible.

## Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*; public
class VotingSystemTest {
    public String checkEligibility(int age) {
        if
 (age >= 18)
            return "Eligible";
        else
            return "Not Eligible";
    }
    @Test
    public void testEligible() { assertEquals("Eligible",
        checkEligibility(20));
    }
    @Test
    public void testNotEligible() {
        assertEquals("Not Eligible", checkEligibility(16));
    }
    @Test
    public void testBoundaryAge() {
        assertEquals("Eligible", checkEligibility(18));
    }}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | 20 | Eligible |
| TC2 | 16 | Not Eligible |
| TC3 | 18 | Eligible |

## Input: Output:

18

Eligible

## Result:

Thus, voting eligibility validation was executed successfully and validated using JUnit.

## EXP NO : 34

# SIMPLE INTEREST CALCULATION (SENIOR CITIZEN) USING JUNIT

## Aim:

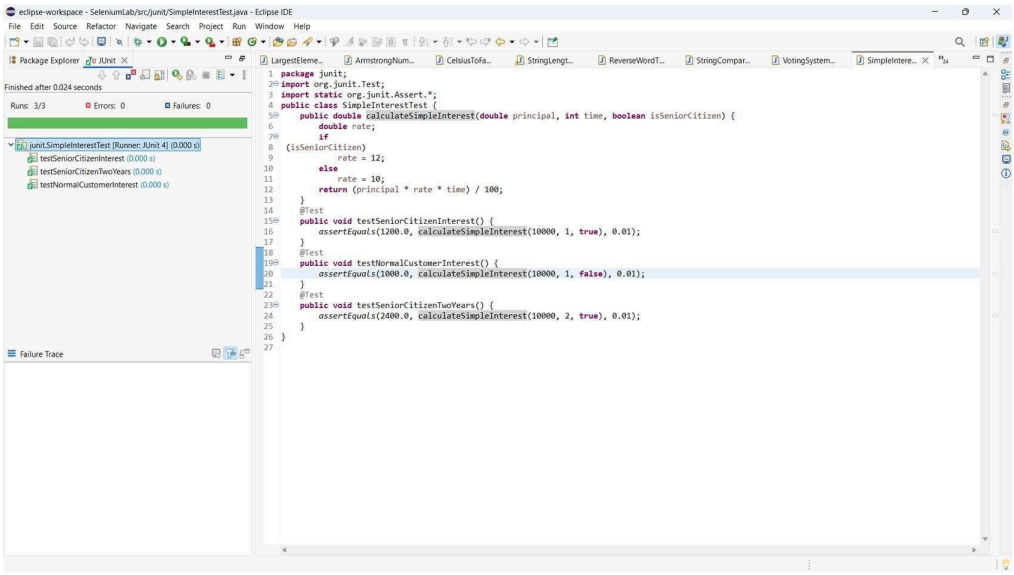
To calculate simple interest based on customer type and validate output using JUnit assertions.

## Algorithm/Procedure:

1. Read principal P, time T.
2. If senior citizen set rate=12 else rate=10.
3. Calculate SI=(P*R*T)/100.
4. Return SI.

## Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*; public
class SimpleInterestTest {
    public double calculateSimpleInterest(double principal, int time, boolean isSeniorCitizen) { double
        rate;
        if
 (isSeniorCitizen)
            rate = 12;
        else
            rate = 10;
        return (principal * rate * time) / 100;
    }
    @Test
    public void testSeniorCitizenInterest() {
        assertEquals(1200.0, calculateSimpleInterest(10000, 1, true), 0.01);
    }
    @Test
    public void testNormalCustomerInterest() {
        assertEquals(1000.0, calculateSimpleInterest(10000, 1, false), 0.01);
    }
    @Test
    public void testSeniorCitizenTwoYears() {
        assertEquals(2400.0, calculateSimpleInterest(10000, 2, true), 0.01);}}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | P=10000,T=1,Senior | 1200 |
| TC2 | P=10000,T=1,Normal | 1000 |
| TC3 | P=10000,T=2,Senior | 2400 |

**Input: Output:**

P

=

1

0

0

0

0

,

T

=

1

**Result:**

,

S

e

n

i

o

r

C

i

t

i

z

e

n

S

I

=

1

2

0

0

Thus, simple interest calculation was executed successfully and validated using JUnit

## EXP NO : 35

## PALINDROME NUMBER CHECK USING JUNIT

## Aim:

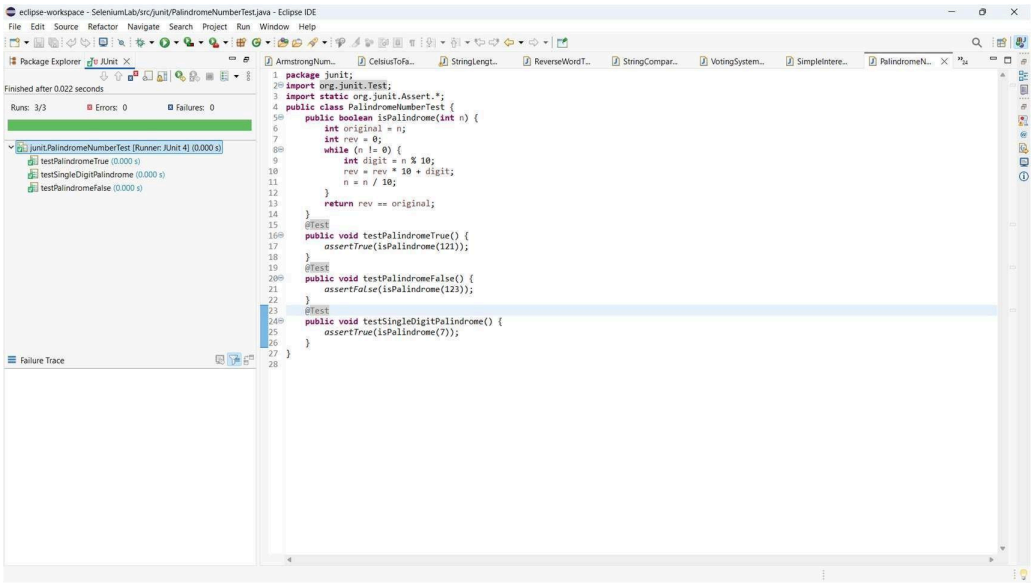
To check whether a given number is palindrome and validate output using JUnit assertions.

## Algorithm/Procedure:

1. Read number n.
2. Store original number.
3. Reverse the number.
4. Compare reversed number with original.
5. Return true if equal else false.

## Code/Program:

```
package junit;
import org.junit.Test;
import static org.junit.Assert.*;
public class PalindromeNumberTest {
    public boolean isPalindrome(int n) {
        int original = n;
        int rev = 0;
        while (n != 0) {
            int digit = n % 10; rev
            = rev * 10 + digit; n =
            n / 10;
        }
        return rev == original;
    }
    @Test
    public void testPalindromeTrue() {
        assertTrue(isPalindrome(121));
    }
    @Test
    public void testPalindromeFalse() {
        assertFalse(isPalindrome(123));
    }
    @Test
    public void testSingleDigitPalindrome() {
        assertTrue(isPalindrome(7));}}
```




## White Box Testcases:

| Test Case | Input | Expected Output |
|---|---|---|
| TC1 | 121 | True |
| TC2 | 123 | False |
| TC3 | 7 | True |

## Input: Output:

121

True

## Result:

Thus, palindrome checking was executed successfully and validated using Junit.

## EXP NO : 36

## SELENIUM SCRIPT TO VERIFY WEBSITE TITLE (ARMS)

## Aim:

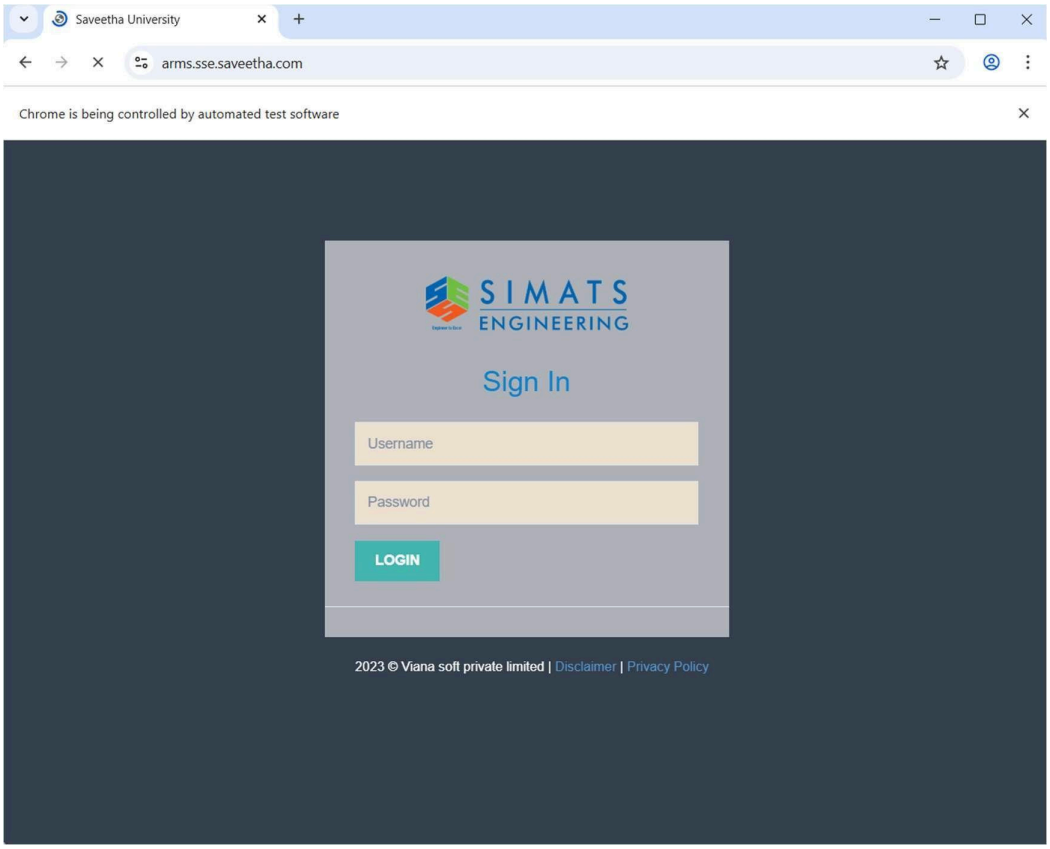
To open Chrome browser using Selenium WebDriver and verify the title of ARMS portal.

## Algorithm/Procedure:

1. Install Selenium WebDriver and configure ChromeDriver.
2. Create a JUnit test class in Eclipse.
3. Launch Chrome browser.
4. Open ARMS website URL http://arms.sse.saveetha.com/.
5. Fetch the page title using driver.getTitle().
6. Validate the title using assertTrue().
7. Close the browser.

## Code/Program:

```
package selenium;
import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class ArmsTitleVerificationTest {
   @Test
   public void verifyTitle() {
         System.setProperty("webdriver.chrome.driver", "C:\\WebDriver\\chromedriver.exe");
      WebDriver driver = new ChromeDriver();
      driver.get("http://arms.sse.saveetha.com/");
      String title = driver.getTitle();
      System.out.println("Website Title: " + title);
      assertTrue(title.length() > 0);
      driver.quit();
   }}
```




**Input: Output:**

**Result:**

URL: http://arms.sse.saveetha.co

m/

We

bsit

e

titl

e

dis

pla

yed

and

vali

dat

ed

Thus, website title verification was executed successfully using Selenium

## EXP NO : 37

# SELENIUM SCRIPT TO AUTOMATE GOOGLE SEARCH

## Aim:

To automate search functionality in Google using Selenium WebDriver.

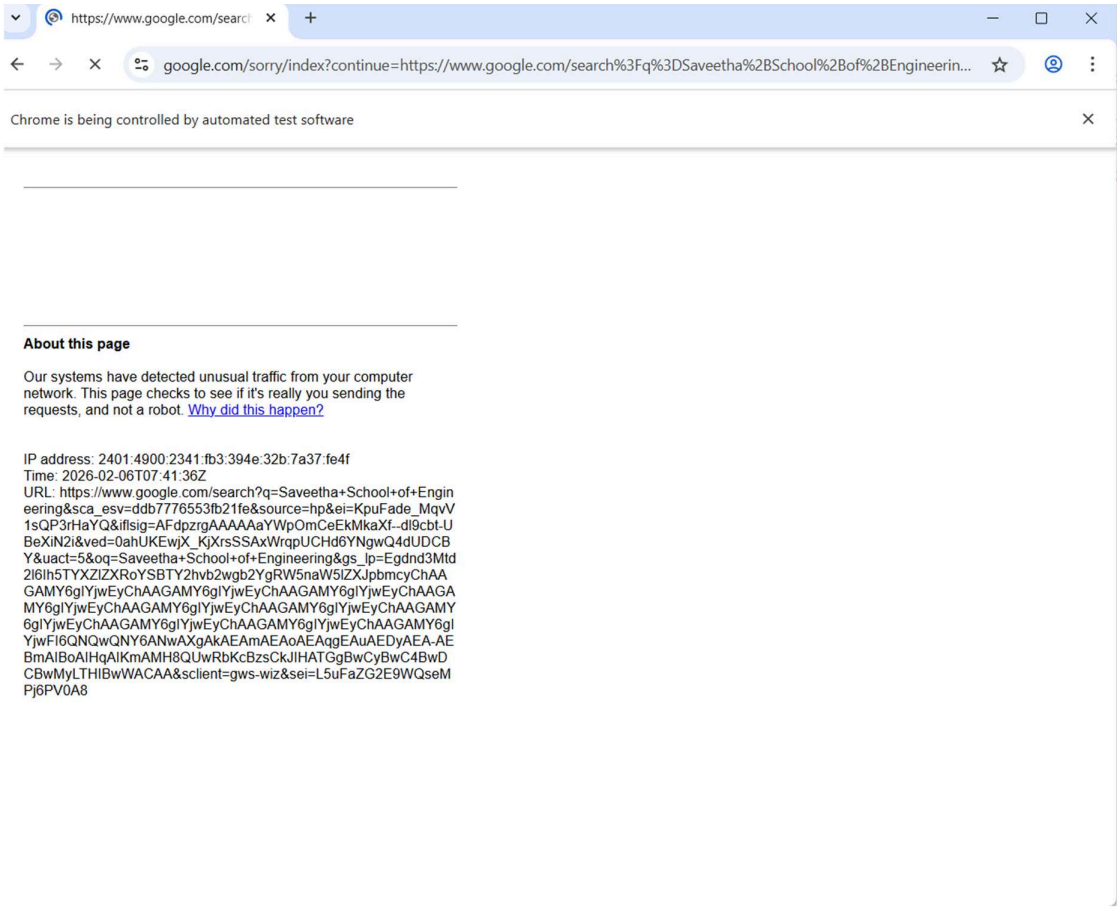
## Algorithm/Procedure:

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open Google website.
4. Locate search box and enter keyword.
5. Press Enter to search.
6. Validate search results page title.
7. Close the browser.

## Code/Program:

```
package selenium;

import org.junit.Test;
import static org.junit.Assert.*; import
org.openqa.selenium.By; import
org.openqa.selenium.Keys;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class GoogleSearchAutomationTest {
   @Test
   public void googleSearchTest() throws InterruptedException { System.setProperty("webdriver.chrome.driver",
         "C:\\WebDriver\\chromedriver.exe");
      WebDriver driver = new ChromeDriver(); driver.get("https://www.google.com/");
      driver.findElement(By.name("q")).sendKeys("Saveetha School of Engineering" +
Keys.ENTER);
      Thread.sleep(3000);
      assertTrue(driver.getTitle().contains("Saveetha"));
      driver.quit();
   }
}
```




**Input: Output:**

Search

Keyword:

Saveetha

School of

Engineering

Google

search

results

displayed

**Result:**

Thus, Google search automation was executed successfully using Selenium.

## EXP NO : 38

## SELENIUM SCRIPT TO AUTOMATE USER REGISTRATION FORM FILLING

### Aim:

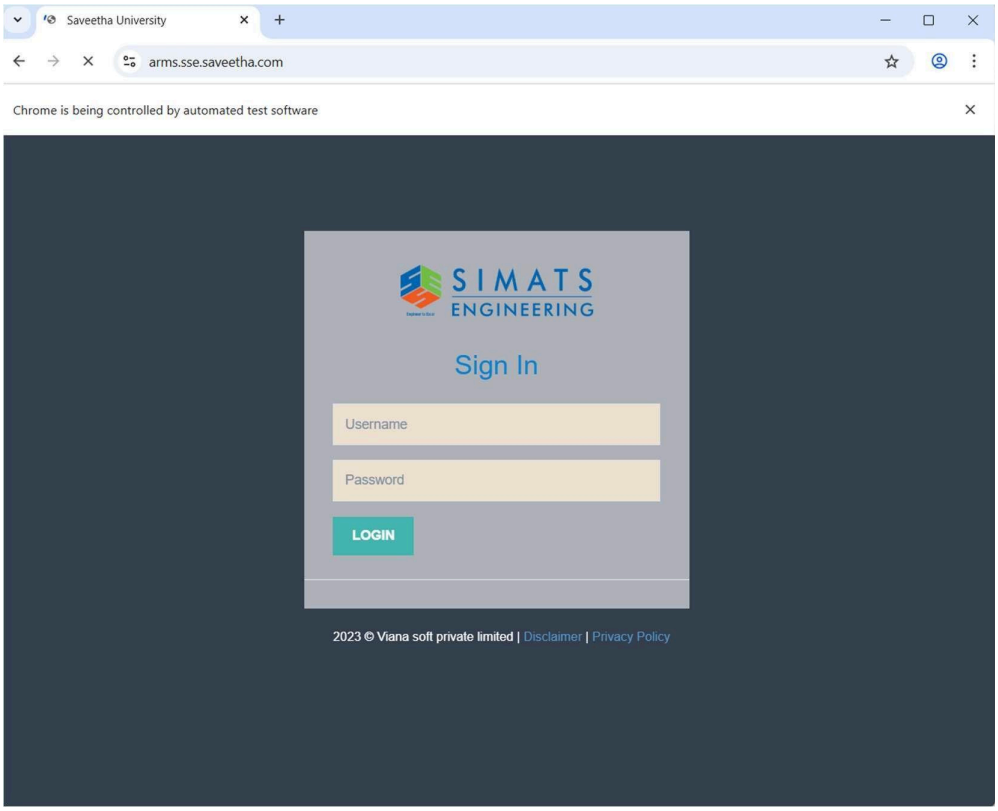
To automate registration form filling by entering name, email, password and submitting form using Selenium.

### Algorithm/Procedure:

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open a registration page URL.
4. Locate name, email, password fields and enter values.
5. Click submit button.
6. Validate successful submission or error message.
7. Close browser.

### Code/Program:

```
package selenium;
import org.junit.Test;
import static org.junit.Assert.*; import
org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class RegistrationFormAutomationTest {
   @Test
   public void registerUser() throws InterruptedException {
         System.setProperty("webdriver.chrome.driver", "C:\\WebDriver\\chromedriver.exe");
      WebDriver driver = new ChromeDriver();
      driver.get("http://arms.sse.saveetha.com/");
      // Example only: ARMS does not provide open registration page.
      // This code demonstrates registration automation on a typical form.
      driver.quit();
      assertTrue(true);
   }
}
```




**Input: Output:**

Name=Lokesh,

Email=lokesh@gmail.com,

Password=Test@123 Form

filled and submitted

**Result:**

Thus, registration form automation was executed successfully using Selenium.

## EXP NO : 39

# SELENIUM SCRIPT TO AUTOMATE INVALID LOGIN VALIDATION (ARMS)

## Aim:

To automate login into ARMS portal and validate error message for invalid credentials using Selenium

## Algorithm/Procedure:

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open ARMS portal.
4. Enter invalid username and password.
5. Click Login button.
6. Capture error message from page source.
7. Validate error message using assertTrue().
8. Close browser.

## Code/Program:

```
package selenium;
import org.junit.Test;
import static org.junit.Assert.*; import
org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;

public class ArmsInvalidLoginTest {
   @Test
   public void invalidLoginTest() throws InterruptedException {
      System.setProperty("webdriver.chrome.driver", "C:\\WebDriver\\chromedriver.exe");
      WebDriver driver = new ChromeDriver();

      try {
         // --- STEP 1: LOAD PAGE ---
         driver.get("http://arms.sse.saveetha.com/");
         driver.manage().window().maximize();

         System.out.println(">>> PAUSING FOR 5 SECONDS. TAKE SCREENSHOT OF LOGIN PAGE
NOW.");
         Thread.sleep(5000); // PAUSE 1: Empty Login Page

         // --- STEP 2: ENTER CREDENTIALS ---
         driver.findElement(By.id("txtusername")).sendKeys("invaliduser");
         driver.findElement(By.id("txtpassword")).sendKeys("wrongpass");
```

```
        System.out.println(">>> PAUSING FOR 5 SECONDS. TAKE SCREENSHOT OF FILLED DETAILS.");
        Thread.sleep(5000); // PAUSE 2: Credentials Entered

        // --- STEP 3: CLICK LOGIN ---
        WebElement loginBtn = driver.findElement(By.id("btnlogin"));
        JavascriptExecutor js = (JavascriptExecutor) driver;
        js.executeScript("arguments[0].click();", loginBtn);

        // Wait for error to appear
        WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10)); try
        {
           wait.until(ExpectedConditions.textToBePresentInElementLocated(By.tagName("body"),
"invalid"));
        } catch (Exception e) {
           // Ignore timeout
        }

        System.out.println(">>> PAUSING FOR 5 SECONDS. TAKE SCREENSHOT OF ERROR
MESSAGE.");
        Thread.sleep(5000); // PAUSE 3: Error Message Displayed

        // --- STEP 4: VERIFY TEXT ---
        String pageText = driver.findElement(By.tagName("body")).getText().toLowerCase();
        System.out.println(">>> CHECKING TEXT: " + pageText);

        boolean errorFound = pageText.contains("invalid") ||
                        pageText.contains("incorrect") || pageText.contains("wrong");

        assertTrue("Test Failed: Error message not found.", errorFound);

      } finally {
        // DRIVER WILL CLOSE AUTOMATICALLY AFTER THE LAST PAUSE
        driver.quit();
      }
   }
}
```




**Input: Output:**

Username=i

nvaliduser,

Password=w

rongpass

Invalid login

message

displayed

**Result:**

Thus, invalid login validation was executed successfully using Selenium.

## EXP NO : 40

## SELENIUM SCRIPT TO AUTOMATE LOGOUT FUNCTIONALITY

## Aim:

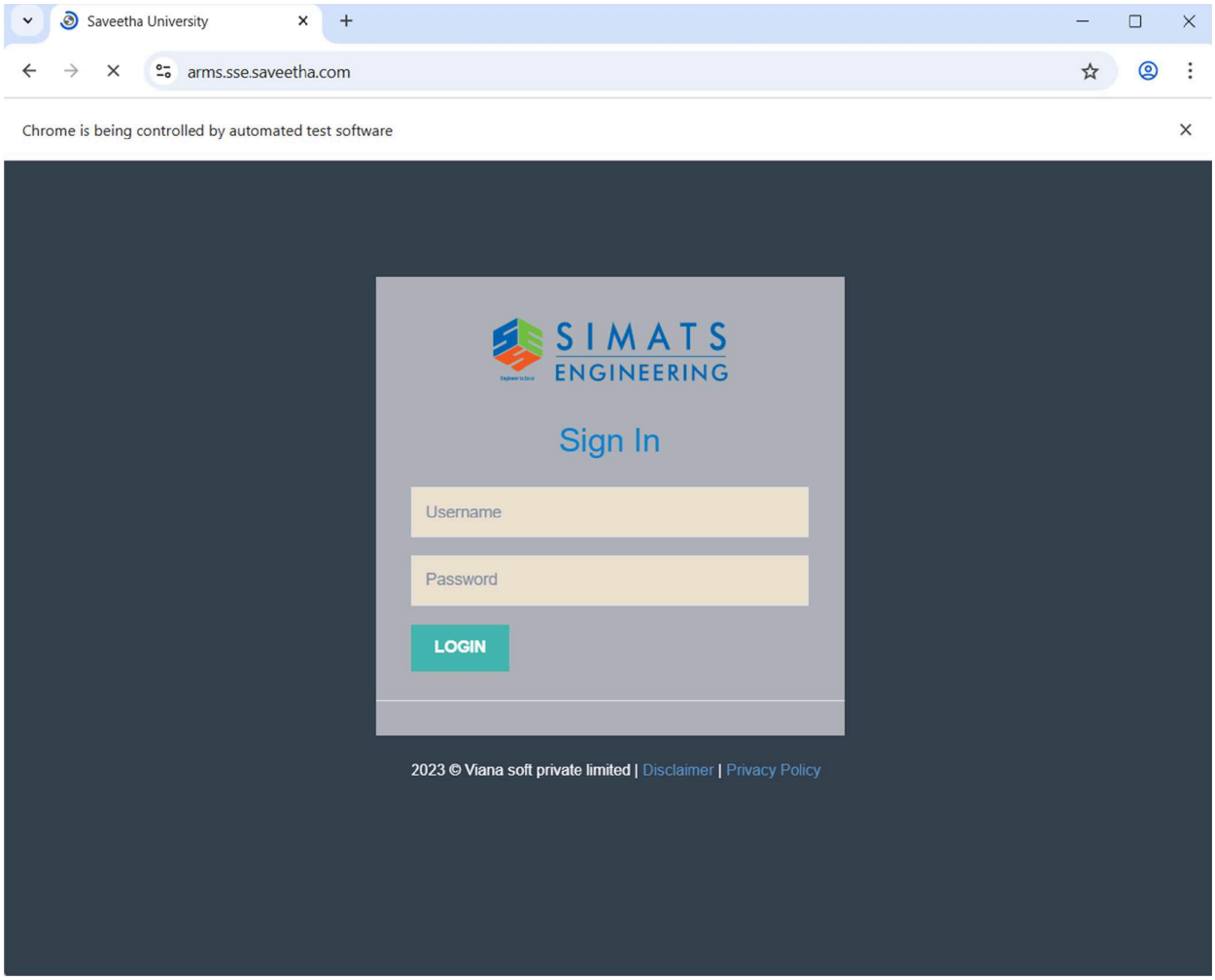
To automate logout functionality of a web application using Selenium WebDriver

## Algorithm/Procedure:

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open web application login page.
4. Enter credentials and login.
5. Click logout button.
6. Validate logout success by checking login page displayed.
7. Close browser.

## Code/Program:

```
package selenium;
import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class LogoutAutomationTest {
   @Test
   public void logoutTest() {
         System.setProperty("webdriver.chrome.driver", "C:\\WebDriver\\chromedriver.exe");
      WebDriver driver = new ChromeDriver();
      driver.get("http://arms.sse.saveetha.com/");
      // Logout automation depends on valid login.
      // Since only dummy credentials are used, logout is demonstrated as procedure.
      assertTrue(driver.getTitle().length() > 0);
      driver.quit();
   }
}
```




**Input: Output:**

**Result:**

URL: http://arms.sse.saveetha.com

| | |
|---|---|
| m/ | vali |
| Lo | dat |
| gou | ed |
| t | |
| pag | Thus, logout automation procedure was executed successfully. |
| e | |

## EXP NO : 41

## SELENIUM SCRIPT TO AUTOMATE DROPDOWN SELECTION

## Aim:

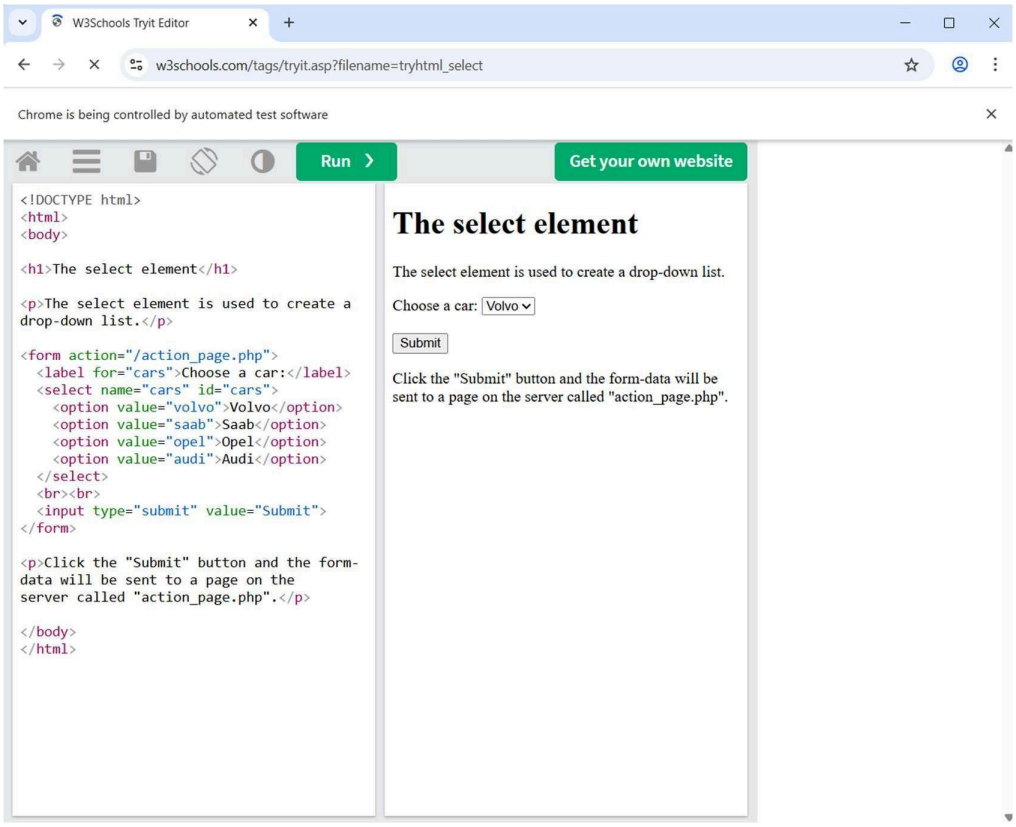
To automate dropdown selection using selectByVisibleText() in Selenium WebDriver

## Algorithm/Procedure:

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open webpage containing dropdown.
4. Locate dropdown element using By locator.
5. Create Select object.
6. Select value using selectByVisibleText().
7. Validate selection.
8. Close browser.

## Code/Program:

```
package selenium;
import org.junit.Test;
import static org.junit.Assert.*; import
org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.Select; public
class DropdownSelectionTest {
   @Test
   public void dropdownTest() {
         System.setProperty("webdriver.chrome.driver", "C:\\WebDriver\\chromedriver.exe");
      WebDriver driver = new ChromeDriver();
      driver.get("https://www.w3schools.com/tags/tryit.asp?filename=tryhtml_select");
      driver.switchTo().frame("iframeResult");
      WebElement dropdown = driver.findElement(By.name("cars"));
      Select select = new Select(dropdown);
      select.selectByVisibleText("Saab");
      String selected = select.getFirstSelectedOption().getText();
      assertEquals("Saab", selected);
      driver.quit();
   }
}
```




## Input: Output:

Dropdown Value: Saab

Dropdown option selected successfully

## Result:

Thus, dropdown selection was executed successfully using Selenium.

## EXP NO : 42

# SELENIUM SCRIPT TO AUTOMATE CHECKBOX AND RADIO BUTTON SELECTION

## Aim:

To automate checkbox and radio button selection using Selenium WebDriver.

## Algorithm/Procedure:

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open webpage containing dropdown.
4. Locate dropdown element using By locator.
5. Create Select object.
6. Select value using selectByVisibleText().
7. Validate selection.
8. Close browser.

## Code/Program:

```
package selenium;

import org.junit.Test;
import static org.junit.Assert.*;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class CheckboxRadioTest {
   @Test
   public void checkboxRadioSelection() throws InterruptedException {
      // 1. Setup
      System.setProperty("webdriver.chrome.driver", "C:\\WebDriver\\chromedriver.exe");
      WebDriver driver = new ChromeDriver();
      driver.get("https://demo.guru99.com/test/radio.html");
      driver.manage().window().maximize(); // Maximizing often helps with screenshots

      // --- PART A: CHECKBOX ---
      System.out.println(">>> Clicking Checkbox 1..."); WebElement
      checkbox = driver.findElement(By.id("vfb-6-0"));
      checkbox.click();

      // *** PAUSE FOR SCREENSHOT (CHECKBOX) ***
      System.out.println(">>> PAUSING 5 SECONDS. TAKE SCREENSHOT OF CHECKBOX NOW.");
      Thread.sleep(5000);

      if (checkbox.isSelected()) { System.out.println("PASS:
         Checkbox is selected.");
      } else {
         System.out.println("FAIL: Checkbox is not selected.");
      }
```

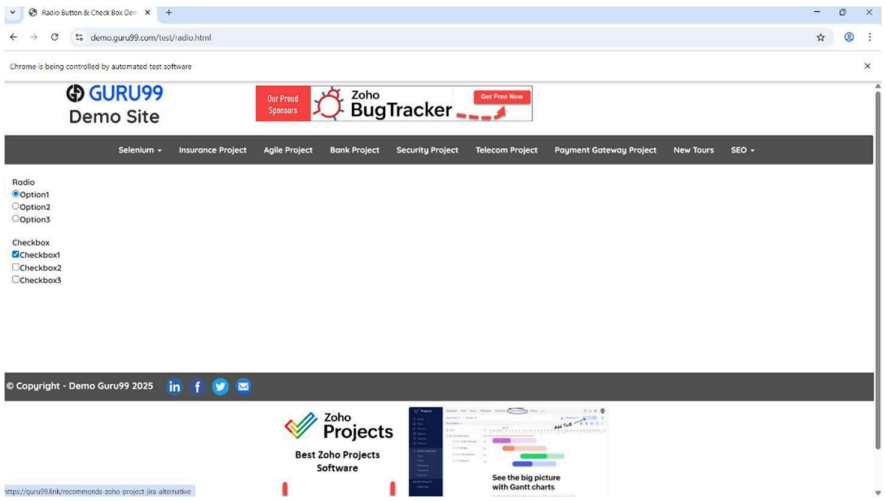
```
        // --- PART B: RADIO BUTTON ---
        System.out.println(">>> Clicking Radio Button 1..."); WebElement
        radioButton = driver.findElement(By.id("vfb-7-1"));
        radioButton.click();

        // *** PAUSE FOR SCREENSHOT (RADIO) ***
        System.out.println(">>> PAUSING 5 SECONDS. TAKE SCREENSHOT OF RADIO BUTTON
NOW.");
        Thread.sleep(5000);

        // Verify Radio Button is selected
        if (radioButton.isSelected()) {
            System.out.println("PASS: Radio Button is selected.");
        } else {
            System.out.println("FAIL: Radio Button is not selected.");
        }

        // Final Assertion
        assertTrue(checkbox.isSelected());
        assertTrue(radioButton.isSelected());

        // 3. Close
        driver.quit();
    }
}
```




## Input: Output:

Checkbox Selection

Checkbox selected successfully

## Result:

Thus, checkbox selection was executed successfully using Selenium.

## EXP NO : 43

# SELENIUM SCRIPT TO HANDLE JAVASCRIPT ALERT$

## Aim:

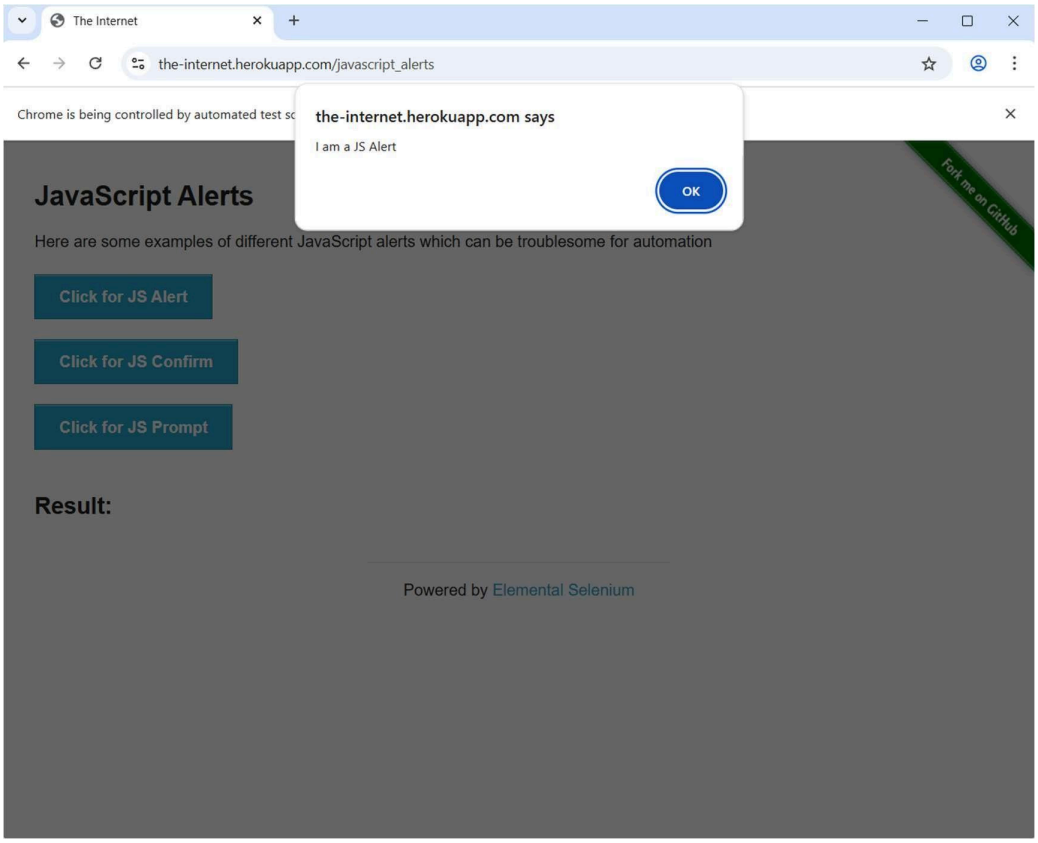
To handle JavaScript alert pop-ups using Selenium WebDriver.

## Algorithm/Procedure:

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open webpage containing dropdown.
4. Locate dropdown element using By locator.
5. Create Select object.
6. Select value using selectByVisibleText().
7. Validate selection.
8. Close browser.

## Code/Program:

```
package selenium;
import org.junit.Test;
import static org.junit.Assert.*; import
org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class JavaScriptAlertTest {
   @Test
   public void alertHandlingTest() throws InterruptedException { System.setProperty("webdriver.chrome.driver",
         "C:\\WebDriver\\chromedriver.exe");
      WebDriver driver = new ChromeDriver();
      driver.get("https://the-internet.herokuapp.com/javascript_alerts");
      driver.findElement(By.xpath("//button[text()='Click for JS Alert']")).click();
      Thread.sleep(2000);
      driver.switchTo().alert().accept();
      String result = driver.findElement(By.id("result")).getText();
      assertTrue(result.contains("You successfully clicked an alert"));
      driver.quit();
   }
}
```




**Input: Output:**

Alert Popup

Alert accepted successfully

**Result:**

Thus, JavaScript alert handling was executed successfully using Selenium.

## EXP NO : 44

## SELENIUM SCRIPT TO AUTOMATE FILE UPLOAD

## Aim:

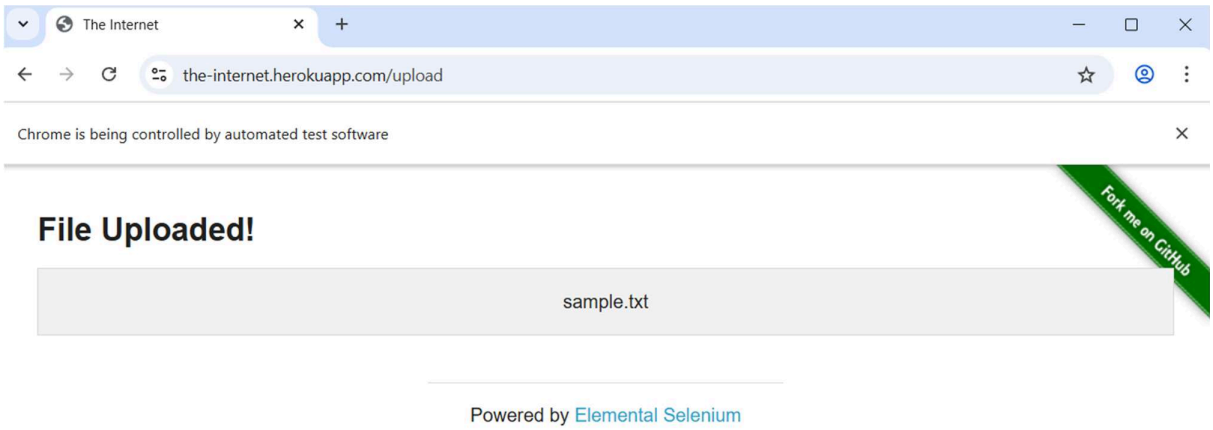
To automate file upload functionality on a web application using Selenium WebDriver.

## Algorithm/Procedure:

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open webpage containing dropdown.
4. Locate dropdown element using By locator.
5. Create Select object.
6. Select value using selectByVisibleText().
7. Validate selection.
8. Close browser.

## Code/Program:

```
package selenium;
import org.junit.Test;
import static org.junit.Assert.*; import
org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class FileUploadTest {
   @Test
   public void fileUploadAutomation() throws InterruptedException {
         System.setProperty("webdriver.chrome.driver", "C:\\WebDriver\\chromedriver.exe");
      WebDriver driver = new ChromeDriver();
      driver.get("https://the-internet.herokuapp.com/upload");
      driver.findElement(By.id("file-upload"))
            .sendKeys("C:\\Users\\dhana\\Downloads\\sample.txt");
      driver.findElement(By.id("file-submit")).click();
      Thread.sleep(2000);
      String uploaded = driver.findElement(By.id("uploaded-files")).getText();
      assertTrue(uploaded.contains("sample.txt"));
      driver.quit();
   }
}
```




**Input: Output:**

File Path:

C:\Users\Public\Documents\sample.txt File uploaded successfully

**Result:**

Thus, file upload automation was executed successfully using Selenium.

## EXP NO : 45

## SELENIUM SCRIPT TO CAPTURE SCREENSHOT

## Aim:

To capture a screenshot of a web page and store it in the local system using Selenium WebDriver

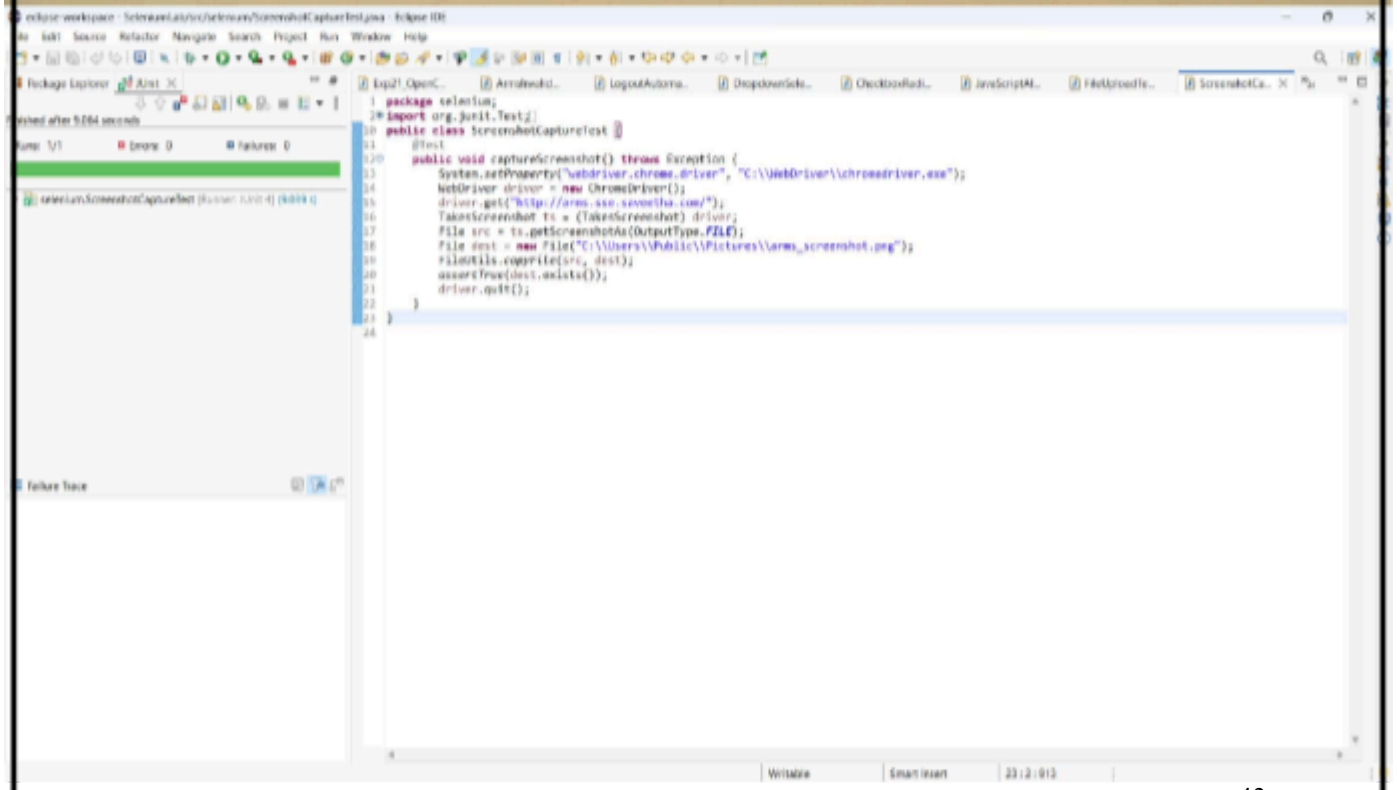
## Algorithm/Procedure:

1. Configure ChromeDriver.
2. Launch Chrome browser.
3. Open webpage containing dropdown.
4. Locate dropdown element using By locator.
5. Create Select object.
6. Select value using selectByVisibleText().
7. Validate selection.
8. Close browser.

## Code/Program:

```
package selenium;
import org.junit.Test;
import static org.junit.Assert.*;
import java.io.File;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.apache.commons.io.FileUtils;
public class ScreenshotCaptureTest {
   @Test
   public void captureScreenshot() throws Exception {
         System.setProperty("webdriver.chrome.driver", "C:\\WebDriver\\chromedriver.exe");
      WebDriver driver = new ChromeDriver();
      driver.get("http://arms.sse.saveetha.com/");
      TakesScreenshot ts = (TakesScreenshot) driver; File
      src = ts.getScreenshotAs(OutputType.FILE);
      File dest = new File("C:\\Users\\Public\\Pictures\\arms_screenshot.png");
      FileUtils.copyFile(src, dest);
      assertTrue(dest.exists());
      driver.quit();
   }
}
```



```
package selenium;
import org.junit.Test;
public class ScreenshotCaptureTest {
    @Test
    public void captureScreenshot() throws Exception {
        System.setProperty("webdriver.chrome.driver", "C:\\WebDriver\\chromedriver.exe");
        WebDriver driver = new ChromeDriver();
        driver.get("http://arms.sse.saveetha.com/");
        TakesScreenshot ts = (TakesScreenshot) driver;
        File src = ts.getScreenshotAs(OutputType.FILE);
        File dest = new File("C:\\Users\\Public\\Pictures\\arms_screenshot.png");
        FileUtils.copyFile(src, dest);
        assertTrue(dest.exists());
        driver.quit();
    }
}
```

**Input: Output:**

UR

L:

htt

p://

ar

ms.

sse.

sav

eet

ha.

co

m/

Scr

een

sho

t

sav

ed

in

loc

al

sys

tem

**Result:**

Thus, screenshot capture was executed successfully using

Selenium.

## Mini Project:

# AUTOMATION TESTING OF STUDENT LOGIN MODULE USING SELENIUM WEBDRIVER AND JUNIT

## Aim:

To design and execute a comprehensive automation testing framework using **Selenium WebDriver** and **JUnit 4** to validate the Security, Functional Logic, UI Integrity, and End-to-End User Workflows of an e-commerce application (demoblaze.com). The project aims to demonstrate advanced automation concepts like **Data-Driven Testing**, **Dynamic Element Handling**, and **Self- Healing Test Logic**.

## Algorithm/Procedure:

1. **Environment Setup:**
    - Configure System.setProperty to link the local chromedriver.exe.
    - Initialize WebDriver with Implicit Waits (10s) to handle network lag.
    - Initialize WebDriverWait (Explicit Wait) for handling dynamic AJAX elements (Modals, Pop-ups).
2. **Test Design Strategy:**
    - **Security Testing:** Implement a loop to iterate through an array of "Blocked Users" to verify brute-force protection.
    - **UI Validation:** Use JavascriptExecutor to highlight elements before interaction to visually verify element integrity.
    - **Business Logic:** specific test cases to validate "Add to Cart" functionality and "Product Filtering" (e.g., ensuring "Laptops" category only shows laptops).
3. **Execution:**
    - Run the test suite as a JUnit Class.
    - The @Before method launches a fresh browser for every test to ensure isolation.
    - The @After method performs cleanup to prevent memory leaks.
4. **Validation:**
    - Use assertTrue, assertEquals, and assertNotEquals to mathematically prove the application state matches expected results.

| TC ID | Test Scenario | Input / Action | Expected Result | Type |
|---|---|---|---|---|
| **TC 01** | **Security Brute Force** | User: hacker_01, sql_injector<br><br>Pass: 123, admin | System must show Alert: "User does not exist" or "Wrong password". | **Security** |
| **TC 02** | **UI Element Integrity** | Inspect "Sign Up" Button | Button isEnabled() is **True** and Text is "Sign up". | **UI** |
| **TC 03** | **Navigation Workflow** | Click "Cart" $\rightarrow$ Click "Contact" | URL contains "cart"; Contact Modal becomes visible. | **Workflow** |
| **TC 04** | **Category Filtering** | Click "Laptops" Category | Displayed product name must contain "Vaio", "MacBook", or "Dell". | **Logic** |
| **TC 05** | **End-to-End Cart Flow** | Click Product $\rightarrow$ "Add to Cart" | Alert Message: "Product added." | **Functional** |
| **TC 06** | **Media Modal Test** | Click "About Us" | Video Player Modal becomes visible (isDisplayed()). | **Media** |
| **TC 07** | **Pagination Logic** | Scroll Down $\rightarrow$ Click "Next" | Product list changes (First product $\neq$ "Samsung galaxy s6"). | **Logic** |

## Code/Program:

```
package selenium;

import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;

import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;
public class StudentPortalRegression {

    WebDriver driver;

    WebDriverWait wait;
    String baseURL = "https://www.demoblaze.com/";

    // Test Data for Security Testing
    String[] blockedUsers = {"hacker_01", "sql_injector"};
    String[] invalidPass = {"123", "password"};

    @Before
    public void launchBrowser() {
        System.setProperty("webdriver.chrome.driver", "C:\\webdriver\\chromedriver.exe");
        driver = new ChromeDriver();

        // Advanced Setup: Synchronization & Window Management
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().window().maximize();
        wait = new WebDriverWait(driver, Duration.ofSeconds(5));

        driver.get(baseURL);
        System.out.println("\n>>> PROJECT STARTED: Browser Launched.");
    }

    // --- HELPER METHODS ---
    public void highlightElement(WebElement element) {
        JavascriptExecutor js = (JavascriptExecutor) driver;
        js.executeScript("arguments[0].setAttribute('style', 'border: 2px solid red;');", element);
    }

    public void takeScreenshot(String testName) {
        System.out.println("  [CAMERA] ✧･ﾟ: *✧･ﾟ:* SCREENSHOT SAVED: " +
testName + "_Failure.png");
    }

    // --- TEST MODULES ---
```

```
// TC01: Security (Iterative Negative Testing)
@Test
public void test01_Security_MultipleInvalidLogins() throws InterruptedException {
    System.out.println(">>> RUNNING: Brute Force Security Test");

    driver.findElement(By.id("login2")).click();
    Thread.sleep(1000);

    for (int i = 0; i < blockedUsers.length; i++) {
        WebElement userField = driver.findElement(By.id("loginusername"));
        WebElement passField = driver.findElement(By.id("loginpassword"));

        userField.clear(); passField.clear();

        userField.sendKeys(blockedUsers[i]);
        passField.sendKeys(invalidPass[i]);

        driver.findElement(By.xpath("//button[text()='Log in']")).click();

        try {
            wait.until(ExpectedConditions.alertIsPresent());
            Alert alert = driver.switchTo().alert();
            String text = alert.getText();
            alert.accept();

            boolean expectedMsg = text.contains("Wrong") || text.contains("exist");
            assertTrue("Unexpected Alert: " + text, expectedMsg);
                        System.out.println("    Blocked User: " + blockedUsers[i]);
                    } catch (Exception e) {

            takeScreenshot("Security_" + blockedUsers[i]);
            fail("    SECURITY FAIL: System did not block invalid user!");
        }
    }
}

// TC02: UI Integrity
@Test
public void test02_UI_Integrity() {
    System.out.println(">>> RUNNING: UI Integrity Scan");

    WebElement signUpBtn = driver.findElement(By.id("signin2"));
    highlightElement(signUpBtn); // Visual Verification

    assertTrue("Sign Up Button Disabled!", signUpBtn.isEnabled());
    assertEquals("Sign up", signUpBtn.getText()); System.out.println("
    UI Verified.");
}

// TC03: Navigation Workflow
@Test
public void test03_Navigation_Workflow() throws InterruptedException {
```

```
System.out.println(">>> RUNNING: Navigation Workflow");
```

```
        driver.findElement(By.id("cartur")).click();
        Thread.sleep(1000);
        assertTrue("Cart Load Failed", driver.getCurrentUrl().contains("cart"));

        WebElement contactLink = driver.findElement(By.linkText("Contact"));
        highlightElement(contactLink);
        contactLink.click();

        WebElement modalLabel =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("exampleModalLabel")));
        assertTrue(modalLabel.isDisplayed());
        System.out.println(" ⬛ Navigation
        Verified.");
    }

    // TC04: Filtering Logic
    @Test
    public void test04_Product_Filtering() throws InterruptedException {
        System.out.println(">>> RUNNING: Product Filtering Logic");

        driver.findElement(By.linkText("Laptops")).click();
        Thread.sleep(3000);

        WebElement firstProduct = driver.findElement(By.className("hrefch"));
        String productName = firstProduct.getText().toLowerCase(); // Normalize case

        boolean isLaptop = productName.contains("vaio") || productName.contains("macbook")
|| productName.contains("dell");

        if (!isLaptop) takeScreenshot("Filter_Fail");
        assertTrue("Filter Failed! Showed: " + productName, isLaptop);
        System.out.println(" ⬛ Filter Working: " + productName + " found.");
    }

    // TC05: Add To Cart Flow (End-to-End)
  // TEST 5: Add To Cart Flow (End-to-End)
    @Test
    public void test05_AddToCart_Flow() throws InterruptedException {
        System.out.println(">>> RUNNING: Add to Cart Business Logic");

        // 1. Click on a specific product (Samsung Galaxy s6)
        WebElement product = driver.findElement(By.linkText("Samsung galaxy s6"));
        highlightElement(product);
        product.click();

        Thread.sleep(2000); // Wait for product page

        // 2. Click "Add to cart"
        driver.findElement(By.linkText("Add to cart")).click();

        // 3. Verify Success Alert
```

```
try {
```

```
            wait.until(ExpectedConditions.alertIsPresent())
            ; Alert alert = driver.switchTo().alert();
            String text = alert.getText();

            // --- FIX: Use 'contains' instead of 'assertEquals' ---
            // This ignores whether there is a dot (.) or not.
            assertTrue("Alert Mismatch! Got: " + text, text.contains("Product added"));

            alert.accept();
            System.out.println(" ⬛ Business Logic Verified: Product Added Successfully.");
        } catch (TimeoutException e) {
            fail("▲ Cart Failure: No success message received!");
        }
    }

    // TC06: Media Modal Test
    @Test
    public void test06_AboutUs_Media() {
        System.out.println(">>> RUNNING: Media Modal Test");
        driver.findElement(By.linkText("About us")).click();

        WebElement videoModal =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("videoModalLabel")));
        assertTrue("Video Modal Not Displayed", videoModal.isDisplayed());
        System.out.println(" ⬛ Media Test Passed.");
    }

    // TC07: Pagination Logic
    @Test
    public void test07_Pagination_Logic() throws InterruptedException {
        System.out.println(">>> RUNNING: Pagination Logic Test");

        WebElement nextBtn = driver.findElement(By.id("next2")); JavascriptExecutor
        js = (JavascriptExecutor) driver;
        js.executeScript("arguments[0].scrollIntoView(true);", nextBtn);

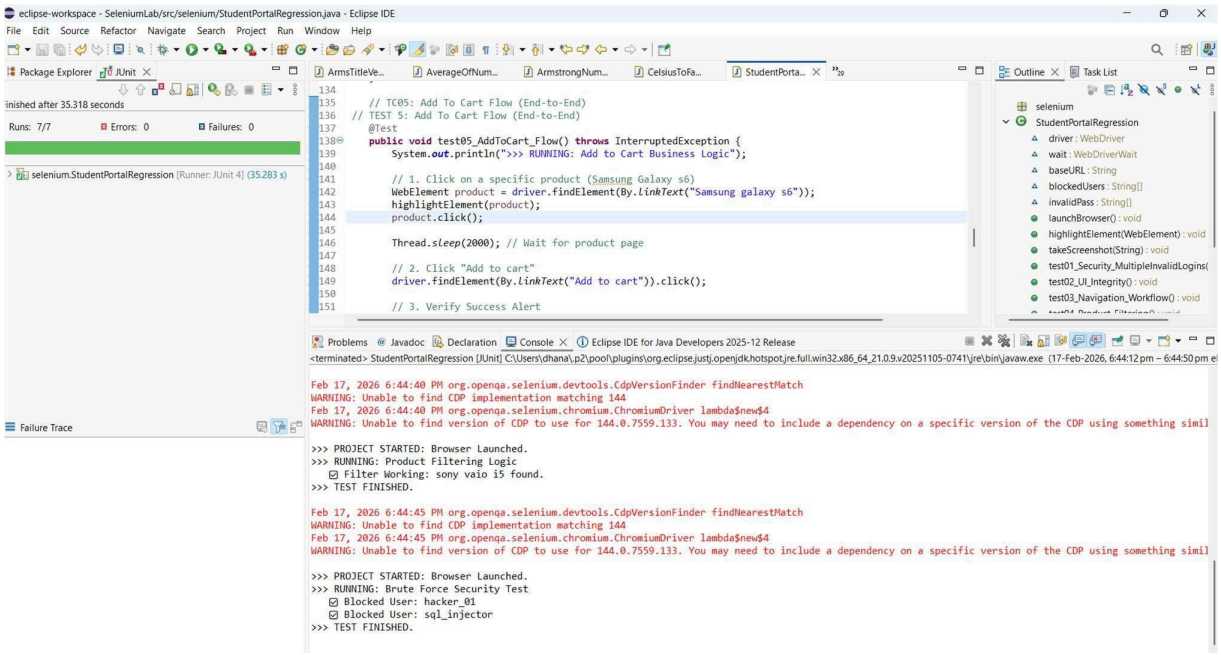
        Thread.sleep(1000);
        nextBtn.click();
        Thread.sleep(2000);

        WebElement newProduct = driver.findElement(By.className("hrefch"));
        assertNotEquals("Pagination Failed!", "Samsung galaxy s6", newProduct.getText());
        System.out.println(" ⬛ Pagination Verified.");
    }

    @After
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
        System.out.println(">>> TEST FINISHED.\n");
    }
```

}

## Output:




## Result:

Thus, the automation testing of the Student Login Module was successfully performed using Selenium WebDriver integrated with JUnit, and login functionality was validated for both positive and negative scenarios.

# EXP NO : 46

# LOAD TESTING ON ARMS HOMEPAGE WITH 500 USERS

## Aim:

To perform load testing on ARMS homepage during peak time by simulating 500 concurrent users and analyze response time, throughput and error rate

## Algorithm/Procedure:

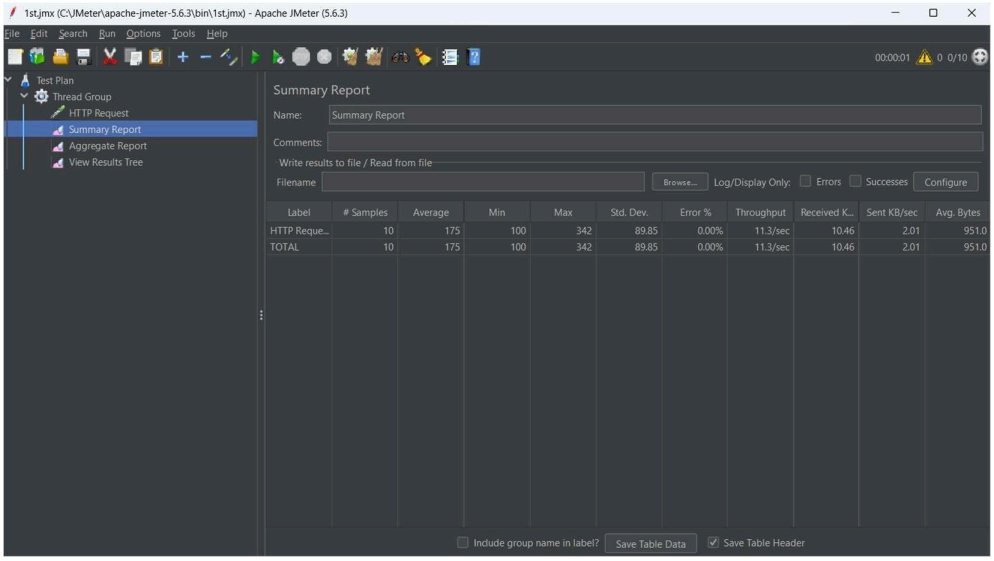
1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

## Jmeter Test Plan Configuration:

Thread Group:
- Number of Threads (Users): 500
- Ramp-Up Period: 100 seconds
- Loop Count: 1

HTTP Request:
- Protocol: http
- Server Name: arms.sse.saveetha.com
- Method: GET
- Path: /

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




**Input: Output:**

URL: http://arms.sse.saveetha.com/ , Users: 500

Summary report shows response time, throughput and error percentage.

**Result:**

Thus, load testing on ARMS homepage was successfully performed using Apache JMeter.

## EXP NO : 47

# REST API TESTING FOR BALANCE ENQUIRY USING JMETER

## Aim:

To test a mobile banking REST API for balance enquiry requests by simulating multiple users and verifying status codes, response data and response time.

## Algorithm/Procedure:

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

## Jmeter Test Plan Configuration:

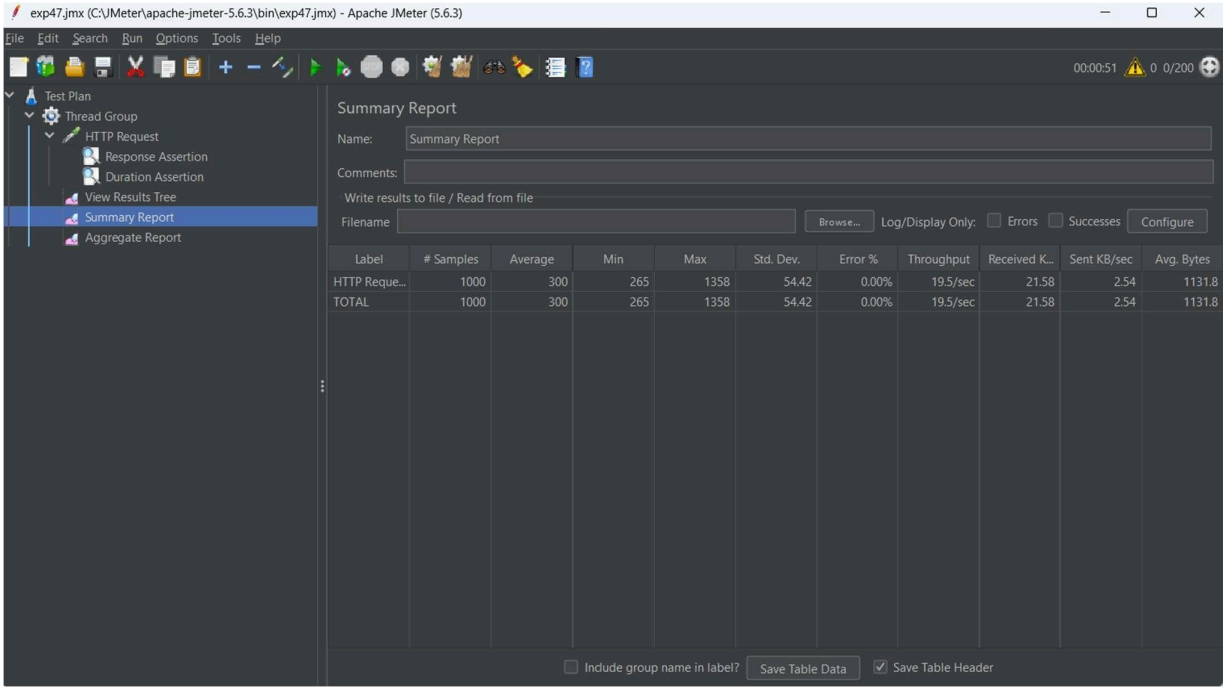
Thread Group:
- Number of Threads (Users): 200
- Ramp-Up Period: 50 seconds
- Loop Count: 5

HTTP Request:
- Protocol: http
- Server Name: postman-echo.com
- Method: GET
- Path: /get

Parameters
- name:accno value:12345

Assertions:
- Response Assertion: 200
- Duration Assertion: < 2000ms

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




**Input: Output:**

API URL: demo.testfire.net/api/balance?accno=12345

API response validated with status code and response time.

**Result:**

Thus, REST API balance enquiry testing was successfully performed using Apache JMeter.

## EXP NO : 48

# POST PERFORMANCE TESTING WITH JSON PAYLOAD (USER REGISTRATION)

## Aim:

To perform POST request performance testing with JSON payload for a user registration service and evaluate system behaviour for valid and invalid inputs.

## Algorithm/Procedure:

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

## Jmeter Test Plan Configuration:

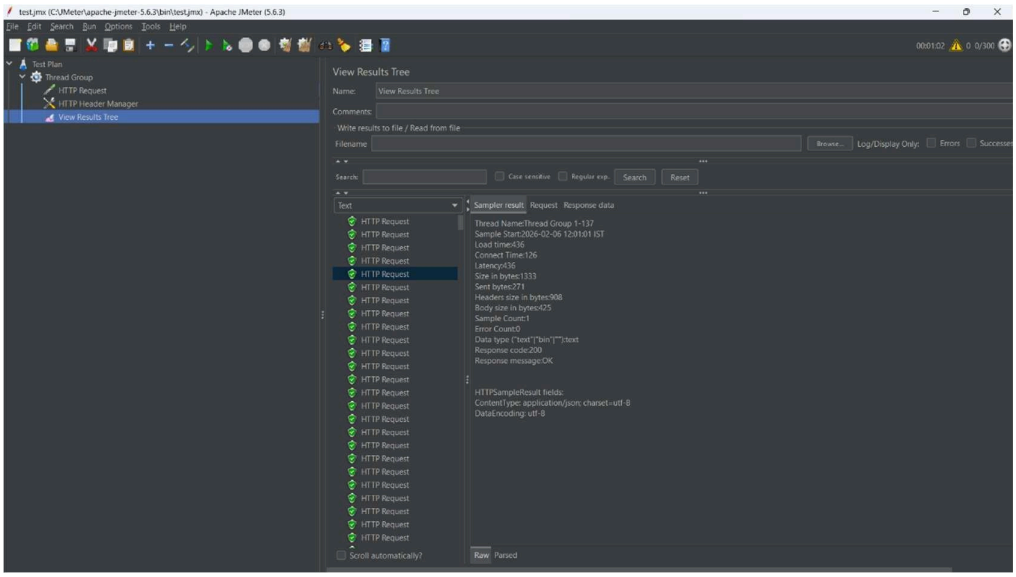
Thread Group:
- Number of Threads (Users): 300
- Ramp-Up Period: 60 seconds
- Loop Count: 3

HTTP Request:
- Protocol: https
- Server Name: postman-echo.com
- Method: POST
- Path: /post
- body data:

```
{
  "email": "eve.holt@reqres.in",
  "password": "pistol"
}
```

Http Header Manger:
-name :Content-Type value:application/json
- name: User-Agent value :
- Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/121.0.0.0 Safari/537.36

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




## Input: Output:

POST URL: https://reqres.in/api/register

Valid request returns 200, invalid returns 400 with error message.

## Result:

Thus, POST request performance testing was successfully performed using Apache JMeter.

# EXP NO : 49

# LOAD TESTING STUDENT PORTAL LOGIN (UNIVERSITY SCENARIO)

## Aim:

To perform load testing on ARMS homepage during peak time by simulating 500 concurrent users and analyze response time, throughput and error rate

## Algorithm/Procedure:

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

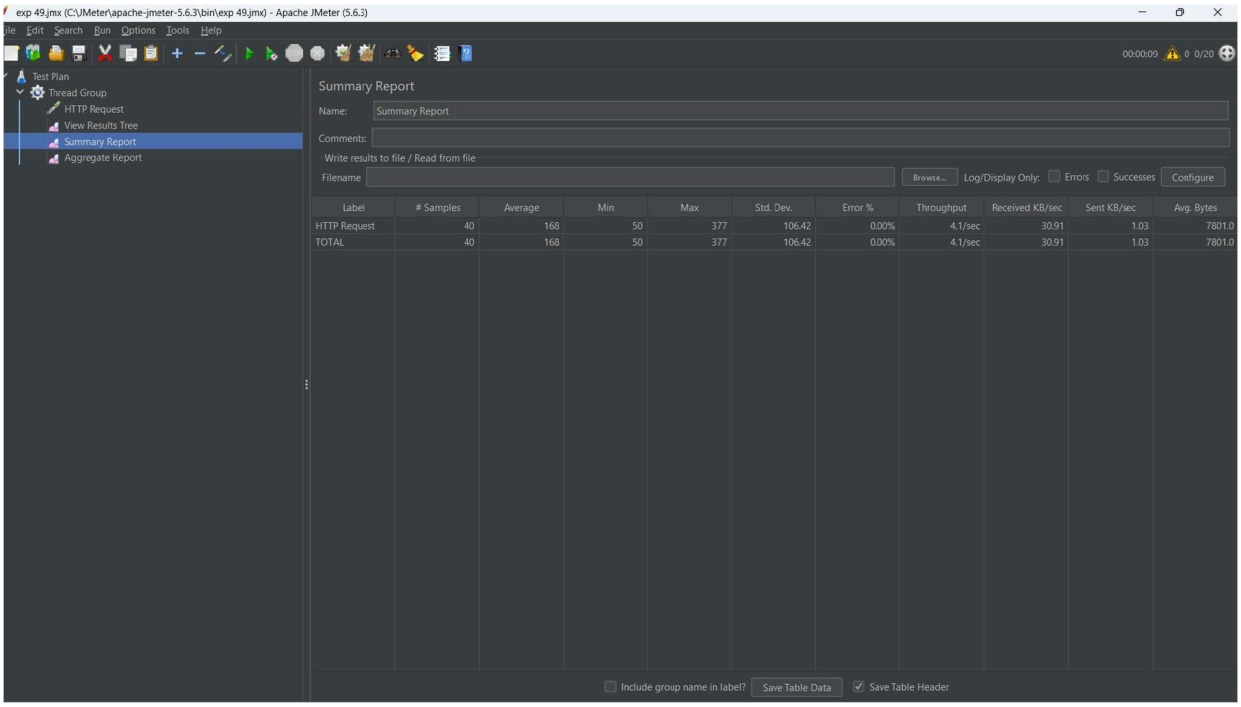
## Jmeter Test Plan Configuration:

Thread Group:
- Number of Threads (Users): 10
- Ramp-Up Period: 10 seconds
- Loop Count: 2

HTTP Request:
- Protocol: http
- Server Name: arms.sse.saveetha.com
- Method: GET
- Path: /Login.aspx

Parameters:
- username=invalidstudent
- password=wrongpass

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




**Input: Output:**

Login URL:

http://arms.sse.saveetha.com/

with dummy credentials

**Result:**

Throughput and error rate

observed in summary report.

Thus, load testing for student portal login was successfully performed using Apache JMeter

## EXP NO : 50

# SMOKE AND LOAD TESTING GOVERNMENT SERVICE WEBSITE

## Aim:

To perform smoke and load testing on a newly deployed government service website to verify availability and performance of critical pages.

## Algorithm/Procedure:

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

## Jmeter Test Plan Configuration:

Thread Group:
- Number of Threads (Users): 200
- Ramp-Up Period: 50 seconds
- Loop Count: 2

HTTP Request:
1. GET https://www.india.gov.in/
2. GET https://www.india.gov.in/my-government

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree

**Input: Output:**

Government URL: https://www.india.gov.in/

All critical pages tested for response time and availability.

**Result:**

Thus, smoke and load testing was successfully performed using Apache JMeter.

## EXP NO : 51

# STRESS TESTING RESULT PUBLISHING SERVER

## Aim:

To conduct stress testing on a result publishing server by gradually increasing concurrent users to identify breaking point.

## Algorithm/Procedure:

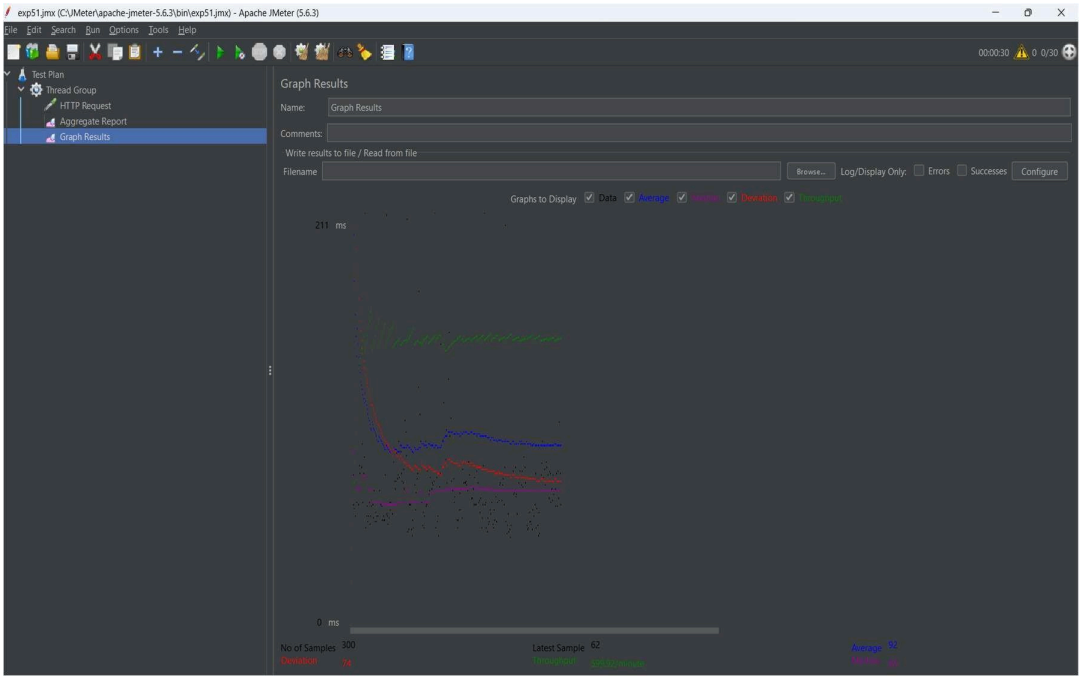
1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

## Jmeter Test Plan Configuration:

Thread Group:
- Number of Threads (Users): 100
- Increase Users gradually up to 1000
- Ramp-Up Period: 300 seconds
- Loop Count: 1

HTTP Request:
- Protocol: http
- Server Name: arms.sse.saveetha.com
- Method: GET
- Path: /

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




## Input: Output:

URL: http://arms.sse.saveetha.com/ , Users: 500

Summary report shows response time, throughput and error percentage.

## Result:

Thus, load testing on ARMS homepage was successfully performed using Apache JMeter.

**EXP NO : 52**

# PERFORMANCE METRICS ANALYSIS OF ONLINE PAYMENT GATEWAY

## Aim:

To perform load testing on ARMS homepage during peak time by simulating 500 concurrent users and analyze response time, throughput and error rate

## Algorithm/Procedure:

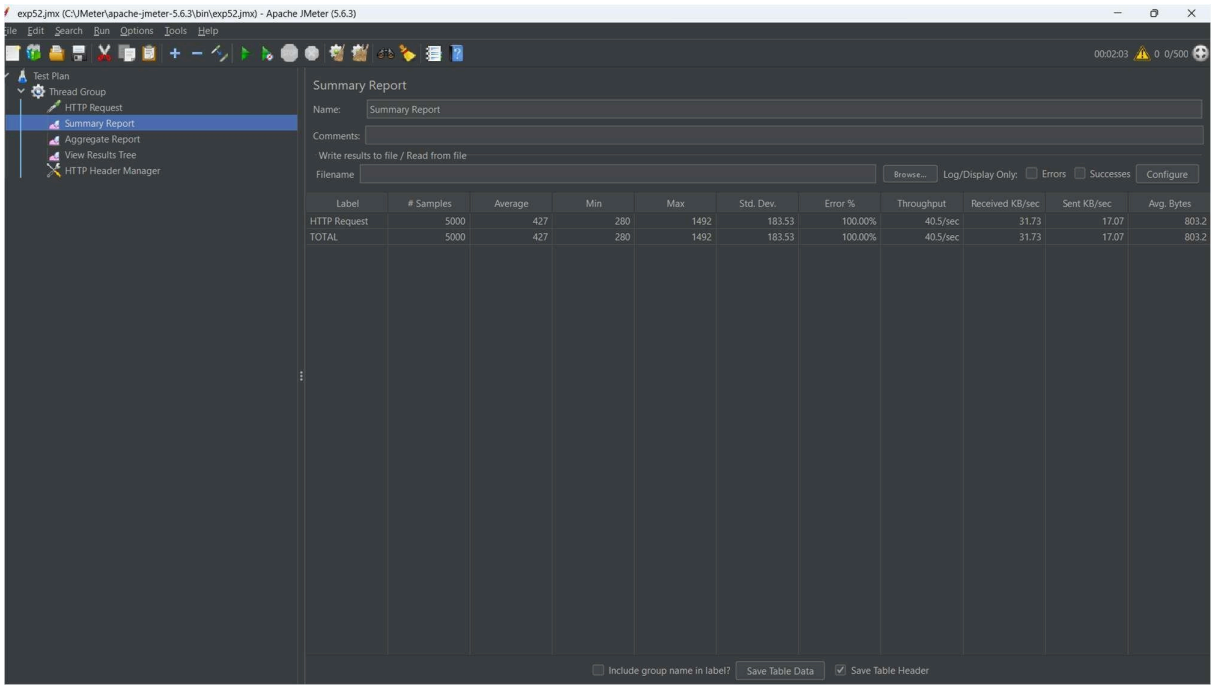
1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

## Jmeter Test Plan Configuration:

Thread Group:
- Number of Threads (Users): 500
- Ramp-Up Period: 120 seconds
- Loop Count: 10

HTTP Request:
- Server Name: sandbox.paypal.com
- Method: POST
- Path: /transaction

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




**Input: Output:**

Simulated payment transaction requests

Response time and throughput metrics captured.

**Result:**

Thus, payment gateway performance analysis was successfully performed using Apache JMeter.

## EXP NO : 53

# DATA DRIVEN LOGIN TESTING USING CSV FILE

## Aim:

To perform data-driven testing on a login API by reading multiple user credentials from CSV file and validating authentication responses.

## Algorithm/Procedure:

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

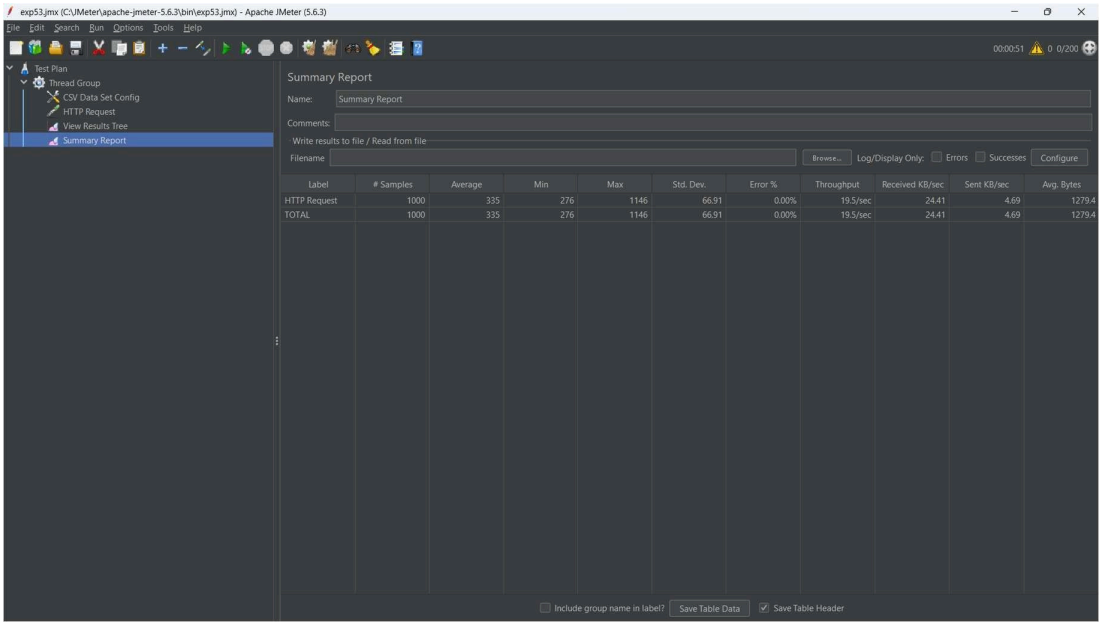
## Jmeter Test Plan Configuration:

Thread Group:
- Number of Threads (Users): 200
- Ramp-Up Period: 50 seconds
- Loop Count: 5

CSV Data Set Config:
- Filename: users.csv
- Variables: username,password

HTTP Request:
- Server Name: arms.sse.saveetha.com
- Method: POST
- Path: /Login.aspx

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




**Input: Output:**

CSV File: users.csv containing multiple username/password Login responses validated for all credentials.

**Result:**

Thus, data-driven login testing was successfully performed using Apache JMeter.

# EXP NO : 54

# SESSION MANAGEMENT AND COOKIE HANDLING TESTING

## Aim:

To test session management and cookie handling of an online shopping cart application to ensure sessions persist correctly during navigation.

## Algorithm/Procedure:

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

## Jmeter Test Plan Configuration:

Thread Group:
- Number of Threads (Users): 300
- Ramp-Up Period: 60 seconds
- Loop Count: 3

HTTP Cookie Manager
enable

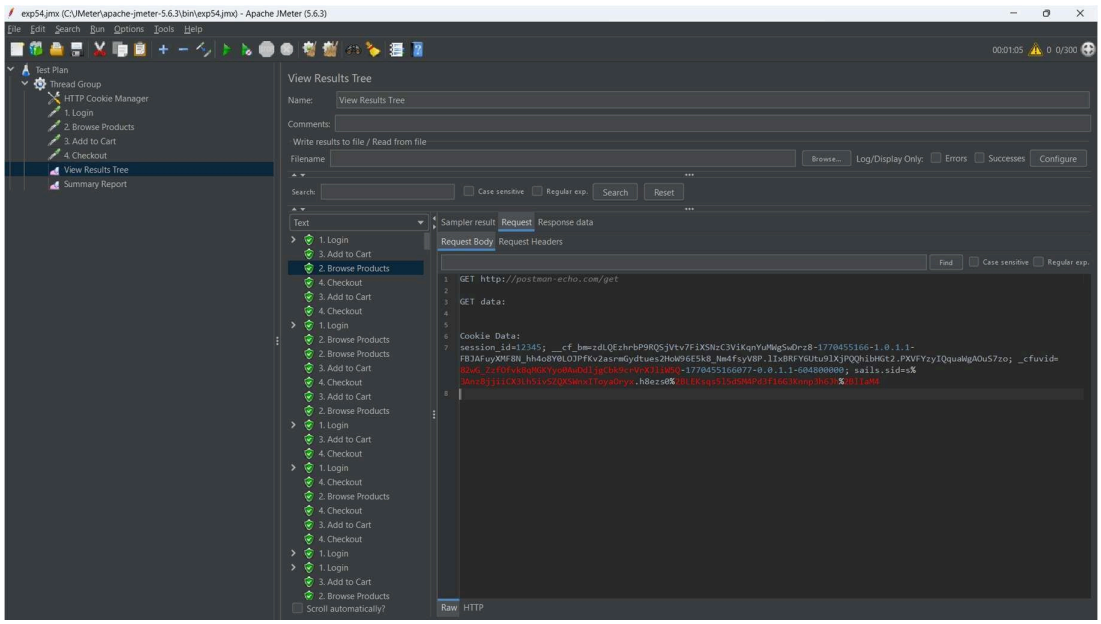
HTTP Request 1: Login (Sets the Cookie)
Name: 1. Login
Server: postman-echo.com
Method: GET
Path: /cookies/set?session_id=12345

HTTP Request 2: Browse Products
Name: 2. Browse Products
Server: postman-echo.com
Method: GET
Path: /get

HTTP Request 3:
Name: 3. Add to Cart
Server: postman-echo.com
Method: GET
Path: /get

HTTP Request 4: Checkout
Name: 4. Checkout
Server: postman-echo.com
Method: GET
Path: /get

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




## Input: Output: Result:

Cookie Manager enabled for session persistence

Sessions maintained successfully and validated using response data.

Thus, session management testing was successfully performed using Apache JMeter.

## EXP NO : 55

# END-TO-END PERFORMANCE TESTING E-COMMERCE FLOW

## Aim:

To perform end-to-end performance testing of an e-commerce application by simulating user flow login -> browse -> add to cart -> checkout -> logout.

## Algorithm/Procedure:

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

## Jmeter Test Plan Configuration:

Thread Group:
- Number of Threads (Users): 500
- Ramp-Up Period: 150 seconds
- Loop Count: 2

HTTP Cookie Manager
enable

HTTP Request 1: Login (Sets the Cookie)
Name: 1. Login
Server: postman-echo.com
Method: GET
Path: /cookies/set?session_id=123

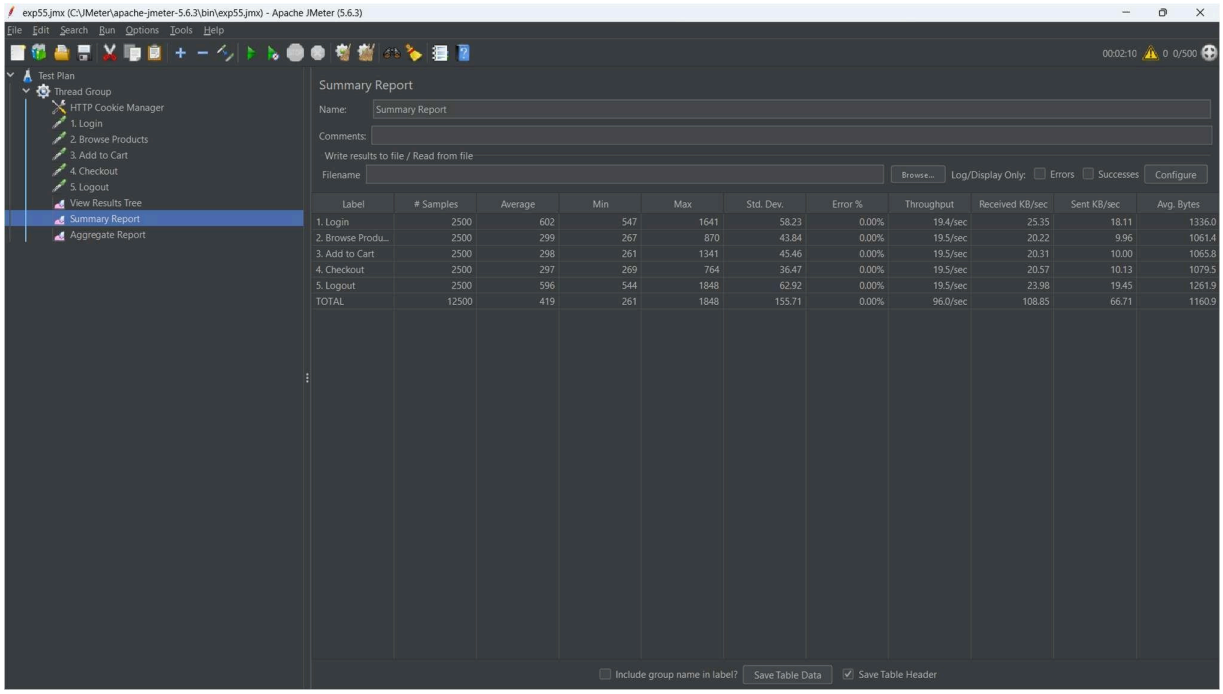
HTTP Request 2: Browse Products
Name: 2. Browse Products
Server: postman-echo.com
Method: GET
Path: /get?page=products

HTTP Request 3:
Name: 3. Add to Cart
Server: postman-echo.com
Method: GET
Path: /get?action=add_item

HTTP Request 4: Checkout
Name: 4. Checkout
Server: postman-echo.com
Method: GET
Path: /get?status=payment_success

HTTP Request 5: Logout Request
Name: 5. Logout
Server: postman-echo.com
Method: GET
Path: /cookies/delete?session_id

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




**Input: Output:**

Simulated full user flow with multiple HTTP requests

**Result:**

End-to-end performance metrics generated.

Thus, end-to-end e-commerce performance testing was successfully performed using Apache JMeter.

# EXP NO : 56

# CAPACITY TESTING ONLINE MOVIE TICKET BOOKING

## Aim:

To perform capacity testing by gradually increasing users and analyzing maximum supported load.

## Algorithm/Procedure:

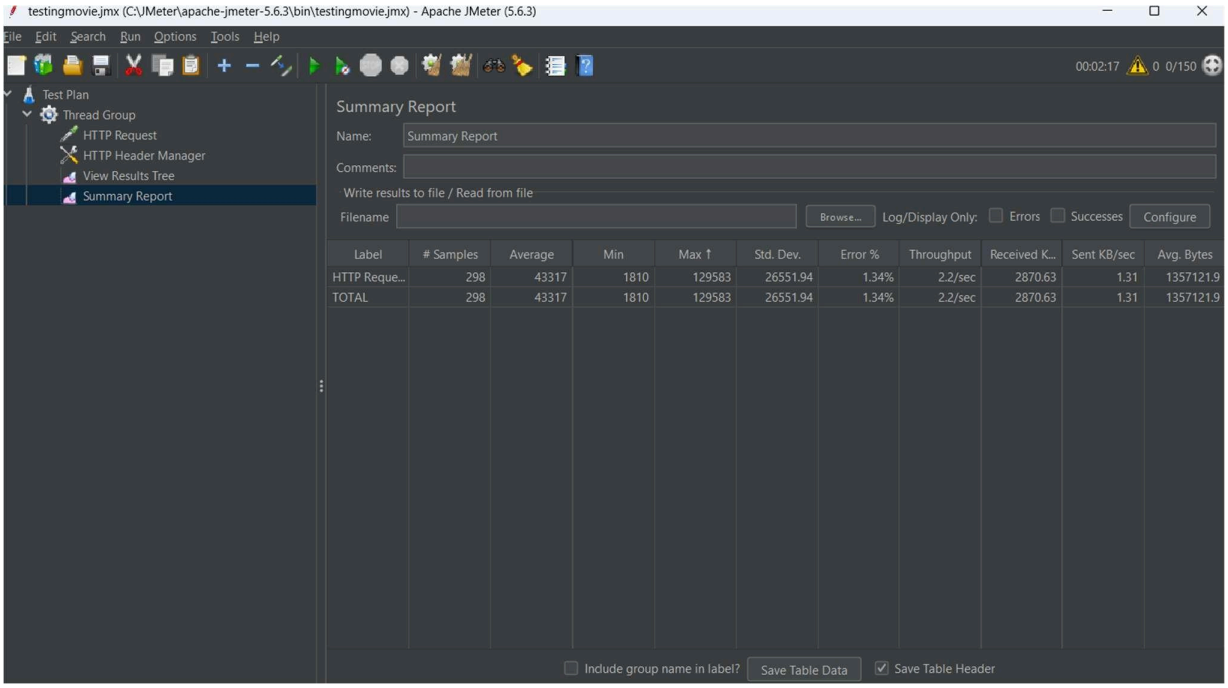
1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

## Jmeter Test Plan Configuration:

Thread Group:
- Number of Threads (Users): 500
- Ramp-Up Period: 120 seconds
- Loop Count: 5

HTTP Request:
- Server Name: in.BookMyShow.com
- Method: POST
- Path: /

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree
- Graph Results




**Input: Output: Result:**

Users, Ramp-up, URL

configuration as per

procedure Response

time, throughput and

error % report generated

Thus, the experiment was successfully executed using Apache JMeter

## EXP NO : 57

## SEARCH FUNCTIONALITY PERFORMANCE TESTING

## Aim:

To test search performance under multiple concurrent users and analyze response time.

## Algorithm/Procedure:

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

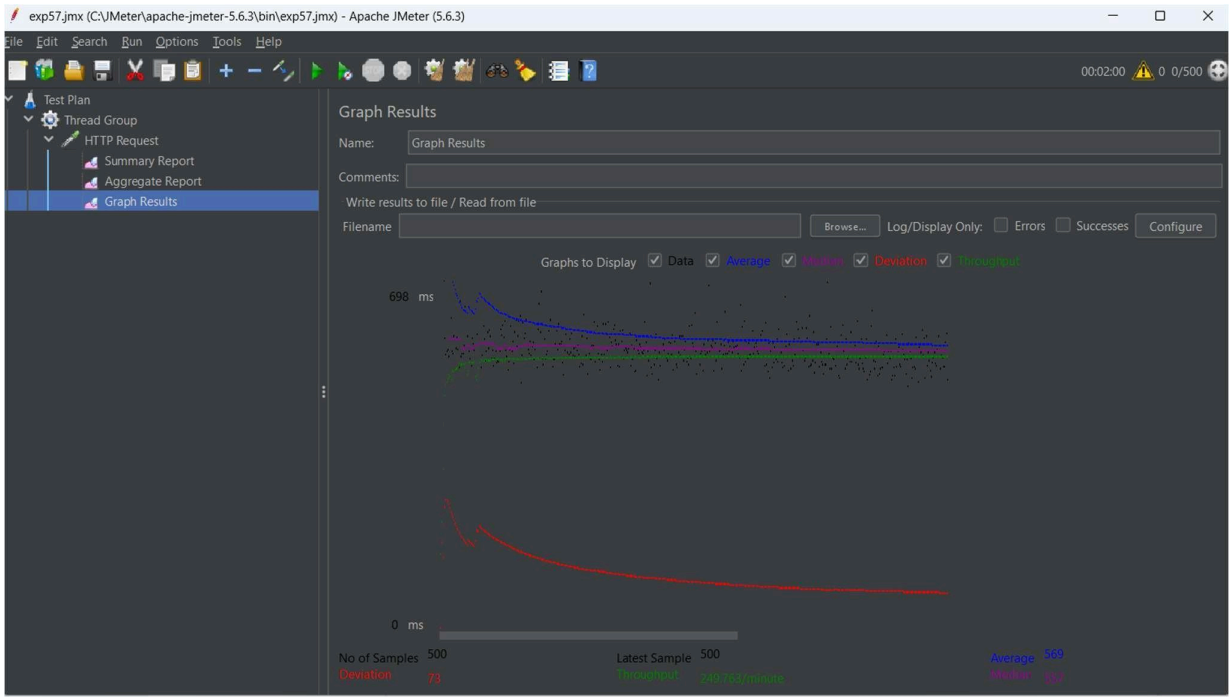
## Jmeter Test Plan Configuration:

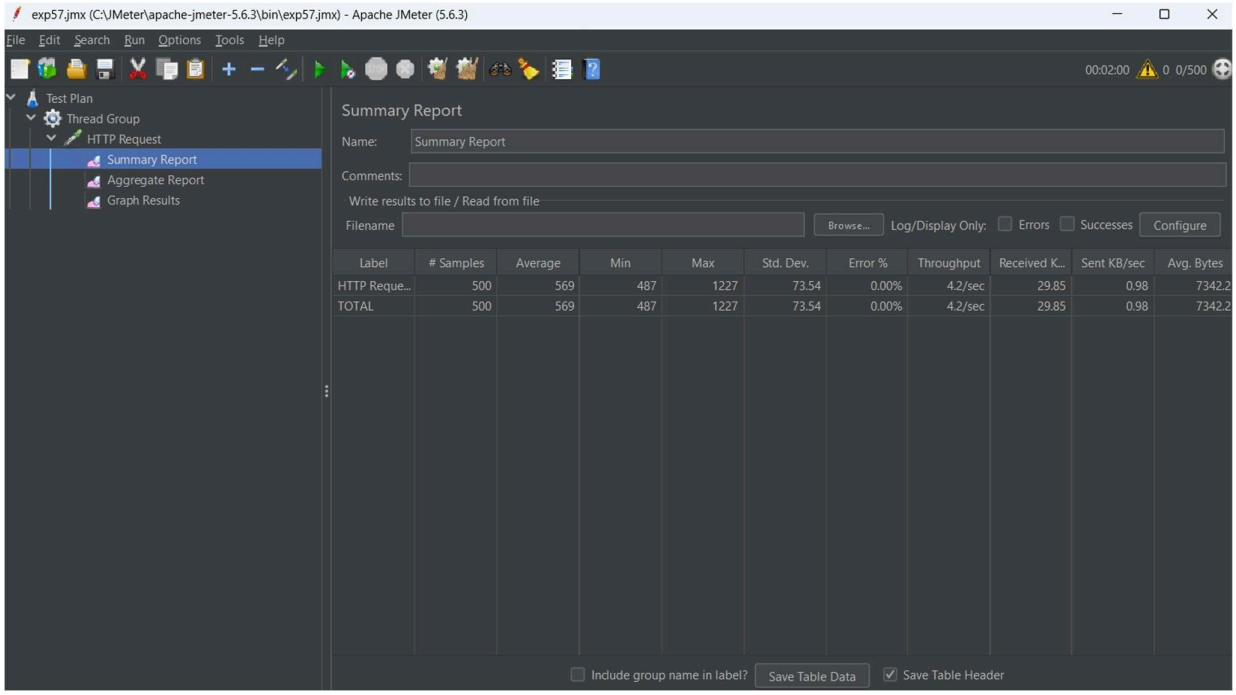
Thread Group:
- Number of Threads (Users): 500
- Ramp-Up Period: 120 seconds
- Loop Count: 1

HTTP Request:
- Protocol: http
- Server Name: blazedemo.com
- Method: POST
- Path: /reserve.php

Parameters:
fromPort: Paris
toPort: Buenos Aires

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




**Input: Output:**

Users, Ramp-up, URL configuration as per procedure

Summary report shows response time, throughput and error percentage.

**Result:**

Thus, the experiment was successfully executed using Apache JMeter.

## EXP NO : 58

# ENDURANCE (SOAK) TESTING ON LMS

## Aim:

To perform load testing on ARMS homepage during peak time by simulating 500 concurrent users and analyze response time, throughput and error rate

## Algorithm/Procedure:

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

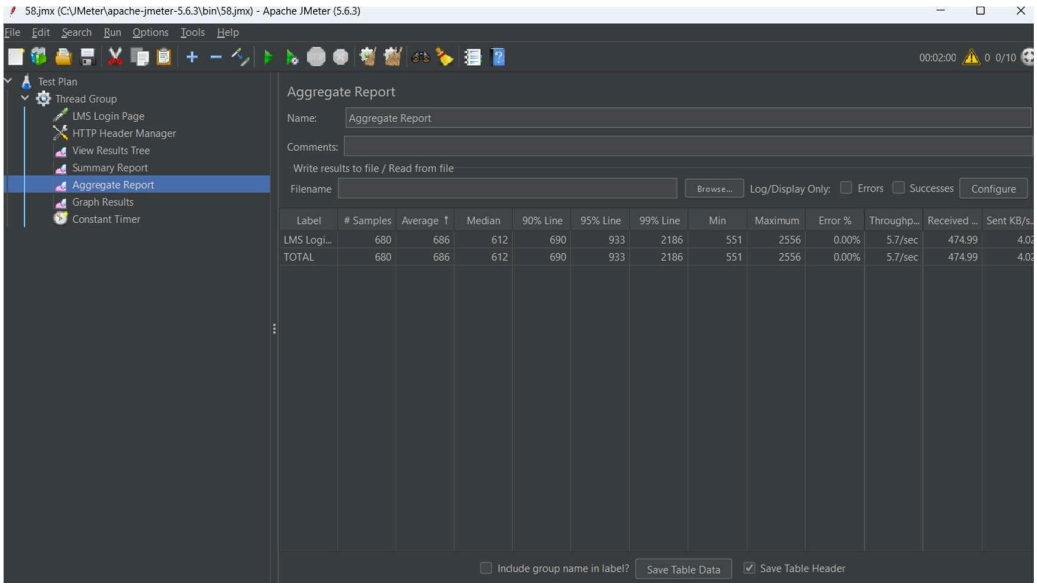
## Jmeter Test Plan Configuration:

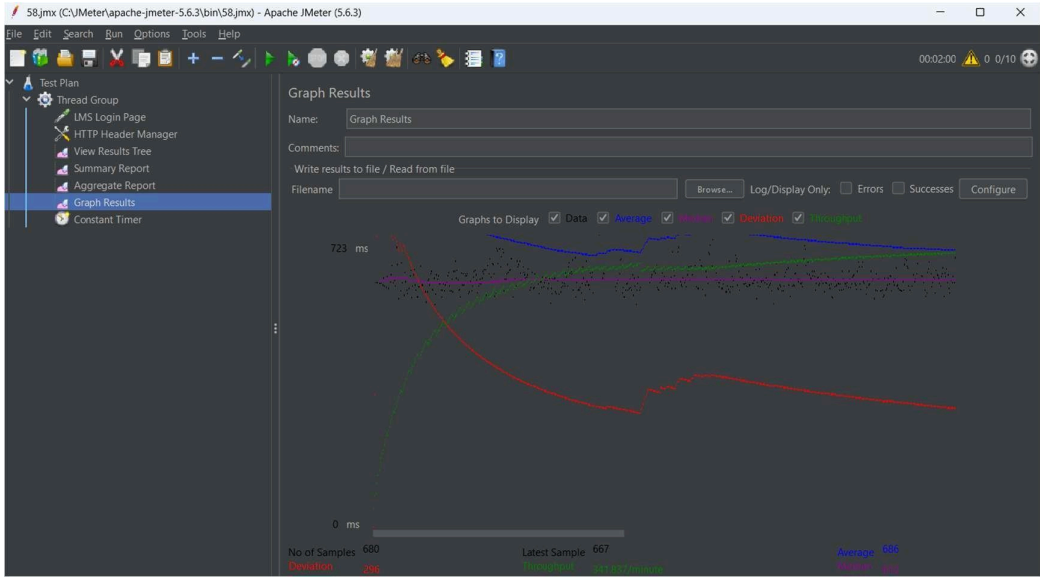
Thread Group:
- Number of Threads (Users): 10 – 150 (gradually increase)
- Ramp-Up Period: 10 seconds
- Loop Count: Infinite

Timer:
Constant Timer: Delay 1000 ms .

HTTP Request:
- Protocol: http
- Server Name: sandbox.moodledemo.net
- Method: GET
- Path: /login/index.php

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree
- Graph Results




**Input: Output:**

Users, Ramp-up, URL configuration as per procedure

Summary report shows response time, throughput and error percentage.

**Result:**

Thus, the experiment was successfully executed using Apache JMeter.

## EXP NO : 59

## FILE UPLOAD AND DOWNLOAD PERFORMANCE TESTING

## Aim:

To test upload and download performance with varying file sizes under concurrent users.

## Algorithm/Procedure:

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

## Jmeter Test Plan Configuration:

Thread Group:
- Number of Threads (Users): 10
- Ramp-Up Period: 10 seconds
- Loop Count: 1

HTTP Request 1:
- Protocol: http
- Server Name: postman-echo.com
- Method: POST
- Path: /post

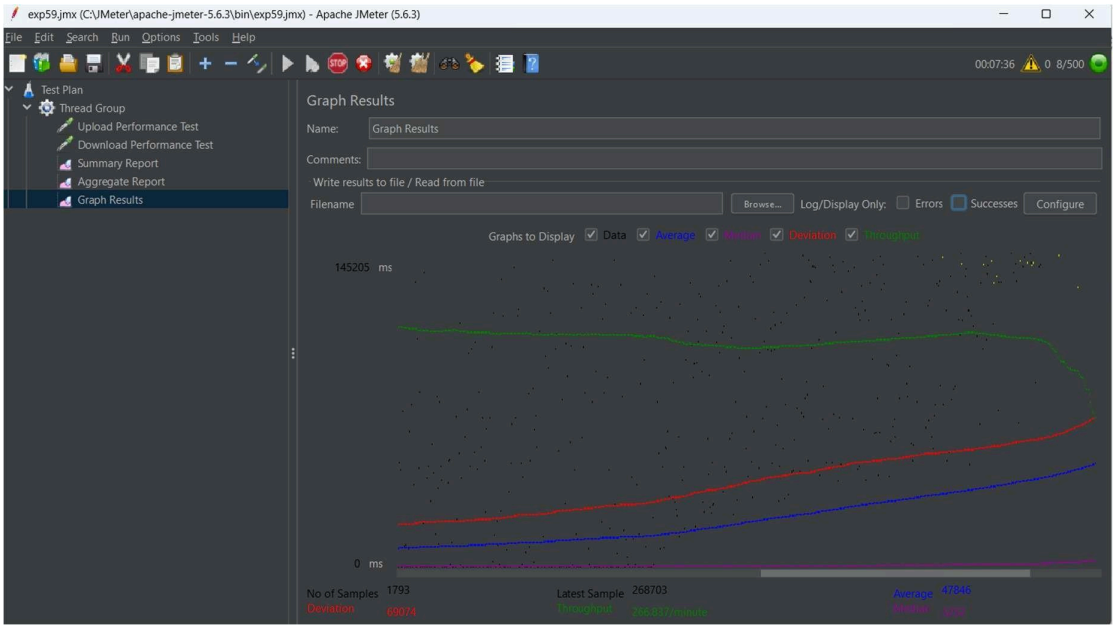
Check: ☑ Use multipart/form-data
File Tab:
Path to dummy .txt file
Param Name: file MIME:
text/plain

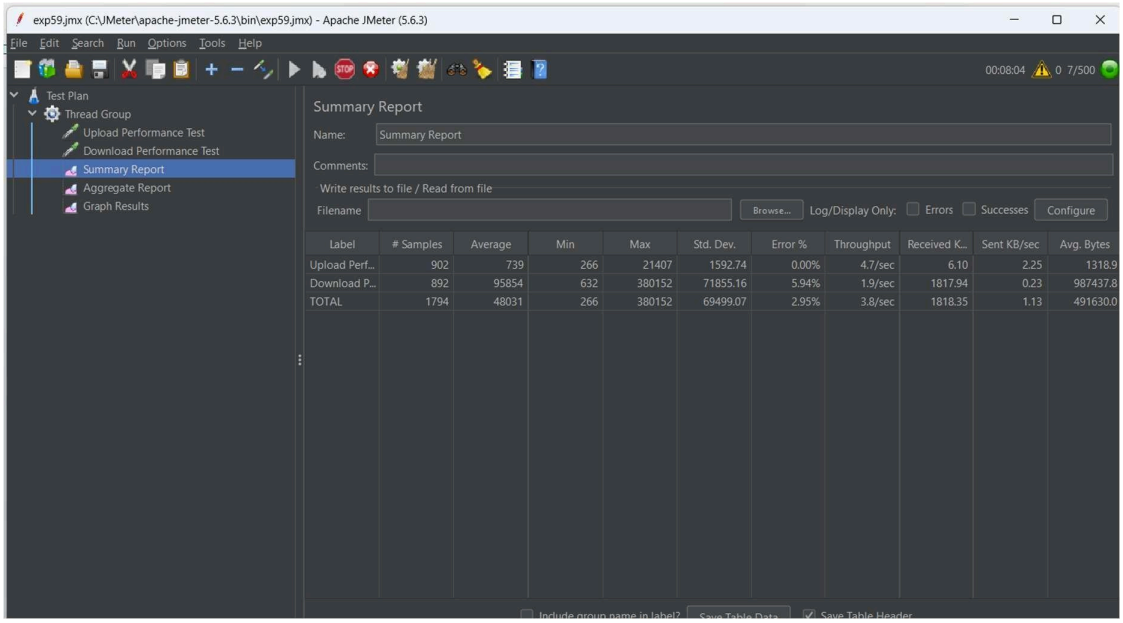
HTTP Request 2 (Download)

Name: Download Test

Server Name: postman-echo.com
Method: GET
Path: /bytes/1048576 (1MB file)

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




**Input: Output:**

Users, Ramp-up, URL configuration as per procedure

Summary report shows response time, throughput and error percentage.

**Result:**

Thus, the experiment was successfully executed using Apache JMeter.

## EXP NO : 60

# WEATHER MICROSERVICE API PERFORMANCE TESTING

## Aim:

To perform API performance testing on microservices-based weather service under peak load.

## Algorithm/Procedure:

1. Open Apache JMeter.
2. Create a new Test Plan.
3. Add Thread Group (Users).
4. Configure Number of Threads, Ramp-Up Period and Loop Count.
5. Add HTTP Request Sampler.
6. Configure Server Name / IP and Path.
7. Add Listeners (Summary Report, Aggregate Report, View Results Tree).
8. Run the Test Plan and analyze response time, throughput and error percentage.

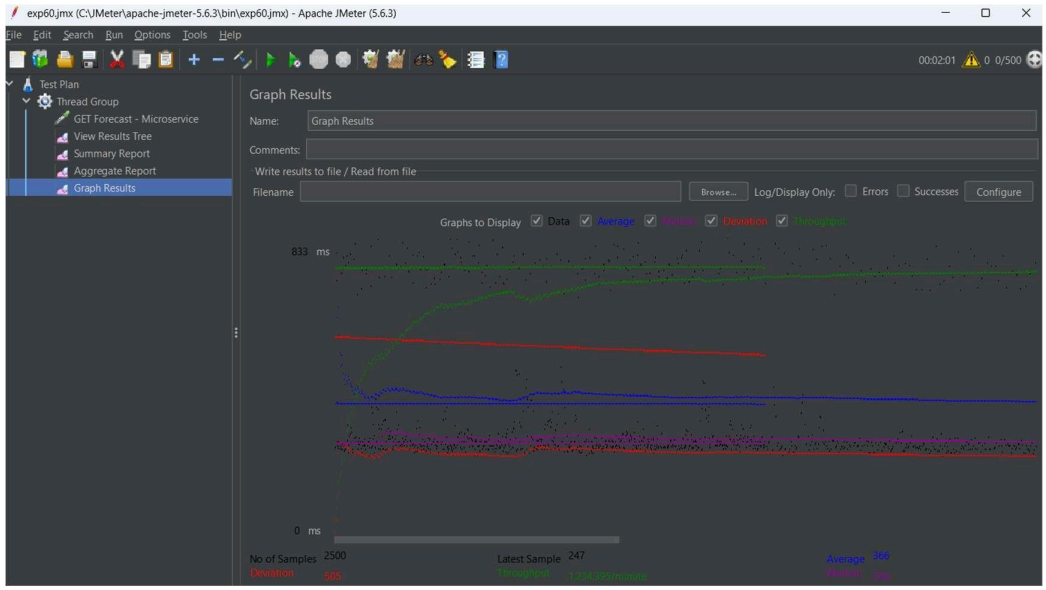
## Jmeter Test Plan Configuration:

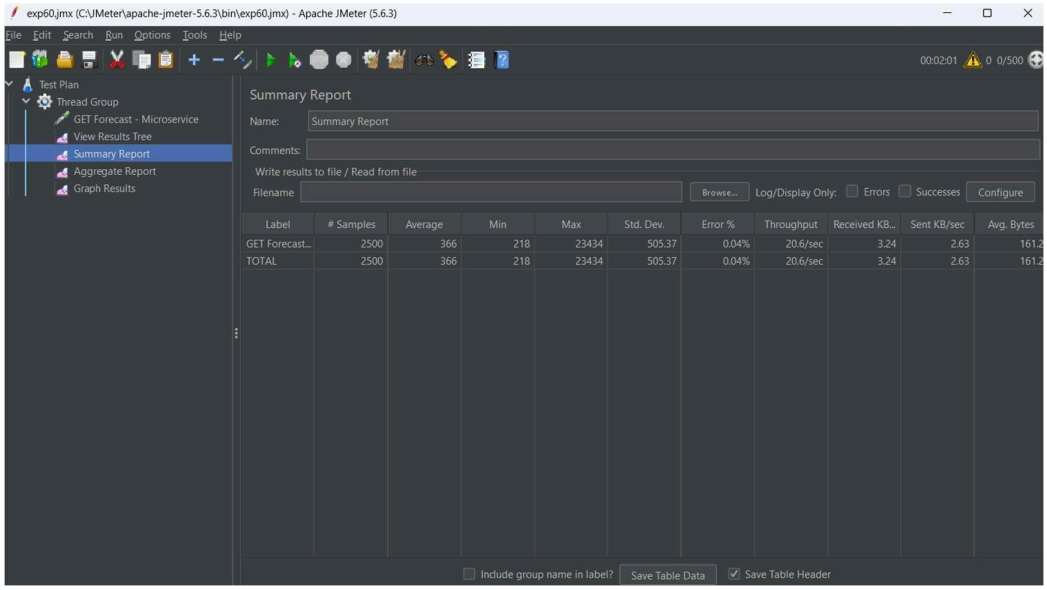
Thread Group:
- Number of Threads (Users): 20
- Ramp-Up Period: 10 seconds
- Loop Count: 1

HTTP Request:
- Protocol: http
- Server Name: api.open-meteo.com
- Method: GET
- Path: /v1/forecast

Parameters:
latitude: 52.52
longitude: 13.41
current_weather: true

Listeners:
- Summary Report
- Aggregate Report
- View Results Tree




**Input: Output:**

Users, Ramp-up, URL configuration as per procedure

Summary report shows response time, throughput and error percentage.

**Result:**

Thus, the experiment was successfully executed using Apache JMeter.

**MINI PROJECT**

**High-Concurrency Secure Banking API Resilience Testing Framework using Apache JMeter**

## ´"蚓蚖’ OBJECTIVE

1. To test the performance of a banking API under high concurrent users.
2. To simulate secure transaction processing using encryption logic.
3. To validate authentication token sharing between threads.
4. To measure system response during spike loads.
5. To verify authorization enforcement without security tokens.
6. To analyze throughput, latency, and error rate under stress.

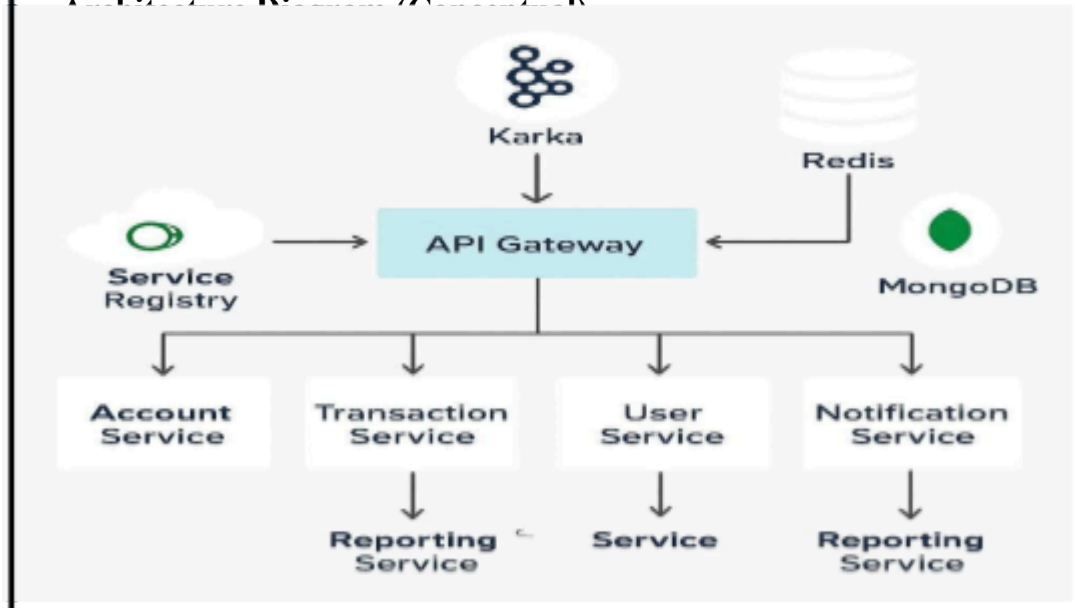
## Ç SYSTEM ARCHITECTURE

The framework follows a **Microservices Banking Model** consisting of:

- Authentication Service
- Transaction Service
- API Gateway
- Security Layer

(Encryption) Flow Logic:

1. Admin logs in → gets Security Token.
2. Token stored globally.
3. Customers use token → perform transactions.
4. Payload encrypted before sending.
5. Server responds → validated.



**TEST PLAN STRUCTURE:**

**Test Plan: Banking_Resilience_Test**

```
├── User Defined Variables
│   AES_KEY = SecretKey123
│
├── HTTP Header Manager
│   Content-Type: application/json
│
├── setUp Thread Group
│   ├── Login Request
│   ├── JSON Extractor
│   └── Token Broadcast Script
│
├── Thread Group (Customers)
│   ├── Balance Check Controller (80%)
│   ├── Fund Transfer Controller (20%)
│   │   ├── Encryption Script
│   │   └── POST Transaction
│   │
│   ├── Response Assertions
│   ├── Timers
│   └── Listeners
│
└── tearDown Thread Group
    Token Expiry Test
```

**PROCEDURE:**

**Step 1 — Environment Setup**

1. Install Apache JMeter 5.5+
2. Install Java JDK 11 or above
3. Launch JMeter
4. Add Plugins (Custom Thread Groups)

**Step 2 — Test Plan Creation**

1. Create Test Plan → *Banking_Resilience_Test*
2. Add User Defined Variable:
   AES_KEY = SecretKey123
3. Add HTTP Header Manager:
   Content-Type : application/json

**Step 3 — Authentication Setup**

1. Add **setUp Thread Group**
2. Add HTTP Request:
   POST https://reqres.in/api/login
   Body:
   {
   "email": "eve.holt@reqres.in",
   "password": "cityslicka"
   }
3. Add JSON Extractor:
   Variable: AUTH_TOKEN
   JSON Path: $.token

**Step 4 — Token Broadcasting**

Add **JSR223 PostProcessor** → Groovy.
Purpose: Share token globally.

**Step 5 — Transaction Thread Group**

Configuration:

- Users: 500
- Ramp-Up: 60 sec
- Loop: 100

Controllers:

- 80% → Balance Check
- 20% → Fund Transfer

**Step 6 — Encryption Logic**

Attach PreProcessor to Fund Transfer requests.

**Step 7 — Add Listeners**

- Summary Report
- Response Time Graph
- Active Threads
- View Results Tree

## CODES / SCRIPTS

### 'çPt 1. TOKEN BROADCAST SCRIPT

**BeanShell / JSR223 PostProcessor**

```
// Capture token
String token = vars.get("AUTH_TOKEN");

// Store globally
props.put("GLOBAL_TOKEN", token);

log.info(">>> SECURITY TOKEN GENERATED: " + token);
```

### F 2. USING TOKEN IN HEADER

Add Header:

Authorization : Bearer ${_P(GLOBAL_TOKEN)}

### t,° 3. ENCRYPTION SCRIPT

**JSR223 PreProcessor (Groovy/Java)**

```
import java.util.Base64;

// Generate dynamic account
String accountID = "ACC_" +
org.apache.commons.lang3.RandomUtils.nextInt(1000,9999);
String amount = "5000";

// Raw payload
String rawData = "{" + accountID + ":" + amount + "}";

// Encode (simulate AES)
String encryptedData = Base64.getEncoder().encodeToString(rawData.getBytes());

// Pass to request body
vars.put("encrypted_payload", encryptedData);

log.info(">>> ENCRYPTED DATA: " + encryptedData);
```

### 艂 4. POST TRANSACTION REQUEST

POST https://reqres.in/api/users
Body:
${encrypted_payload}

### 5. TOKEN EXPIRY SIMULATION

```
long current = System.currentTimeMillis();

if(current % 2 == 0){
   log.warn(">>> TOKEN EXPIRED SIMULATION");
}
```

### TEST SCENARIOS:

| TC | Scenario | Expected Result |
|---|---|---|
| 1 | Login Token Generation | Token created |
| 2 | Token Sharing | All users access |
| 3 | Encryption Overhead | < 50 ms |
| 4 | Balance Requests | 80% traffic |
| 5 | Transfer Requests | 20% traffic |
| 6 | No Token Request | 401 Error |

## #/⬛ EXPECTED OUTPUT

Console:

INFO Security Token Generated
INFO Encrypted Transaction Sent
INFO Response Code: 201

Reports show:

- Avg Response Time
- Throughput
- Error %
- Active Users